



# AgentOS-uCore 设计开发文档

基于 RISC-V uCore 的系统原生智能体运行机制

|      |                          |
|------|--------------------------|
| 项目名称 | AgentOS-uCore            |
| 队伍名称 | happy-legend             |
| 目标平台 | RISC-V 64 / uCore / QEMU |
| 文档版本 | 决赛设计开发文档                 |

|     |            |
|-----|------------|
| 姓名  | 联系方式（QQ）   |
| 王浩泮 | 2091576055 |
| 康俊豪 | 488718235  |

2026 年 08 月

## 摘 要

智能体工作流已经从单轮问答发展为持续的工具调用、文件处理和多角色协作。现有应用层框架能够安排模型和工具，却往往只在应用日志里保存角色、工作流代次、上下文因果关系和资源归属。当工具真正写入文件、发送消息或申请 CPU 服务时，内核看到的仍是普通进程、文件和 IPC，无法据此作出一致判断。

本项目基于 LearningOS/uCore 实现 AgentOS-uCore。系统沿用 RISC-V uCore 的进程、虚拟内存、VFS、系统调用和调度器，并把 Agent（智能体）登记为内核能够识别的工作负载。Agent 进程创建与地址空间设计把低频的身份、配额和生命周期状态留在 PCB，将高频读取的 Context 镜像和用户缓存映射到固定用户地址；Agent-OS 内核结构化交互接口以工具目录、schema（参数模式）、执行约定（Execution Contract）和任务通道承接工具请求；上下文路径、来源追溯（provenance）和执行记录保存请求、结果与数据来源；文件查询、事件等待和工作流 EEVDF 则分别处理属性检索、Agent Loop 唤醒和 CPU 服务分配。

在集成层，AgentOS 控制台按固定步骤执行科研恢复工作流；AgentOS Nexus 则在同一次 QEMU 启动中持续运行协调、系统、研究和分析四个业务角色。Nexus 通过中继、结构化工具、消息等待和 artifact（任务结果文件）分派工作、汇集结果；Guest（客户机）可以同时查看身份、上下文、文件、IPC、等待和调度状态。

以下性能结果固定对应冻结在源码提交 2b14fb 的可复现测量批次；当前版本后续功能改动另由功能回归验证，未重新计入这组数据。该批次使用 30 次相互独立的全新 QEMU 启动，保留 33 份原始串口输出、19 份 CSV 数据表和 7,498 条结构化记录。在 16 组独立工作流配对中，带索引核心查询的耗时均低于遍历查询，配对加速比中位数为 **3.118 倍**；在同样 16 组配对的端到端窗口中，索引路径仅有 **3/16 组** 更快，其耗时减去遍历路径的中位差值为 **+13.452 ms**，说明核心查询之外的准备与收尾开销尚未得到控制。文件查询网格覆盖目录规模与命中数的 12 种组合，任务通道保留 96 条包含 16 个操作的序列；EEVDF 的 504 条唤醒探针均在 0–1 个时钟节拍（tick）内获得服务。

| 设计主题                 | 内核机制                                | 可观察行为                                         |
|----------------------|-------------------------------------|-----------------------------------------------|
| Agent 进程创建与地址空间设计    | PCB 扩展、受信任映像、七页 Agent Context 区     | 普通进程与 Agent 进程共存；Guest 可直接读取六页内核镜像，并使用一页用户缓存。 |
| Agent-OS 内核结构化交互接口   | 工具目录、V2/V3 请求和固定大小返回结构              | 工具可发现，参数按 schema 解码；未知工具、类型错误和权限不足均返回明确状态。    |
| 工具调用协议               | Execution Contract、Batch、任务通道 SQ/CQ | 同步短调用、带约定调用和有界队列共用一套工具语义；重试、取消、超时与背压可区分。      |
| 上下文路径管理              | 128 条有界路径、只读镜像、分支和回滚                | 连续调用会形成可直接读取的路径；容量用尽后按 FIFO 淘汰，回滚只改变后续可见上下文。  |
| 面向 Agent 查询优化的文件系统扩展 | 元数据目录、选择性索引、摘要和实时订阅                 | Agent 按属性和摘要查找对象；查询结果写入选入用户缓存，并以遍历路径校验语义一致性。  |
| Agent Loop 内核运行机制    | 心跳、事件等待、消息路由、工作流 EEVDF              | 无事件时线程休眠，消息或 TIMER 到达后唤醒；多个工作流按实体而非线程数获得服务。   |
| 综合场景                 | 恢复控制台和 Nexus 四角色协作                  | 同一 RV64 Guest 中完成任务分派、失败后重排、报告审批与会话关闭。        |

表 1: AgentOS-uCore 的系统能力与可观察行为

# 目录

|                                  |    |
|----------------------------------|----|
| 摘要                               | ii |
| 第一章 项目背景                         | 1  |
| 1.1 项目背景与设计目标                    | 1  |
| 1.1.1 智能体对操作系统的新需求               | 1  |
| 1.1.2 基于 uCore 的系统扩展             | 1  |
| 1.1.3 机制与策略的分层                   | 1  |
| 1.1.4 相关机制与技术选择                  | 3  |
| 1.2 设计目标与阅读线索                    | 3  |
| 第二章 系统设计                         | 4  |
| 2.1 AgentOS-uCore 总体架构           | 4  |
| 2.1.1 以 uCore 为底座的产品分层           | 4  |
| 2.1.2 六项核心机制与请求流向                | 5  |
| 2.1.3 一次智能体工具调用                  | 5  |
| 2.1.4 上层 runtime                 | 5  |
| 2.2 跨模块状态约束                      | 6  |
| 第三章 系统实现                         | 7  |
| 3.1 Agent 进程创建与地址空间设计            | 7  |
| 3.1.1 从普通进程到 workflow 中的 Agent   | 7  |
| 3.1.2 身份字段与角色策略                  | 8  |
| 3.1.3 Agent Context 区的七页布局       | 8  |
| 3.1.4 生命周期代次与并发控制                | 9  |
| 3.1.5 随 workflow 创建的资源主体         | 9  |
| 3.1.6 验证结果                       | 9  |
| 3.2 Agent-OS 内核结构化交互接口           | 9  |
| 3.2.1 工具调用协议与工具目录                | 9  |
| 3.2.2 自适应的 V2 调用                 | 10 |
| 3.2.3 四种执行路径                     | 10 |
| 3.2.4 工具执行期间的资源使用租约              | 10 |
| 3.2.5 带类型信息的任务通道                 | 10 |
| 3.2.6 验证结果                       | 11 |
| 3.3 上下文路径管理、数据来源与 Workflow Fence | 11 |
| 3.3.1 从多轮调用到 Context Path        | 12 |
| 3.3.2 分支、回滚与有界淘汰                 | 12 |
| 3.3.3 数据来源传播                     | 12 |
| 3.3.4 工具清单与副作用检查                 | 12 |
| 3.3.5 执行记录环                      | 13 |
| 3.3.6 Workflow Fence             | 13 |
| 3.3.7 验证结果                       | 13 |
| 3.4 面向 Agent 查询优化的文件系统扩展         | 14 |
| 3.4.1 从路径访问到属性查询                 | 14 |
| 3.4.2 元数据与 inode 代次              | 14 |
| 3.4.3 遍历查询与选择性索引                 | 14 |
| 3.4.4 带类型信息的实时订阅与缺口重新同步          | 14 |
| 3.4.5 查询性能                       | 15 |
| 3.4.6 验证结果                       | 16 |
| 3.5 Agent Loop 内核运行机制            | 16 |
| 3.5.1 从轮询循环到内核等待                 | 16 |
| 3.5.2 心跳与模型请求关联号                 | 17 |
| 3.5.3 工作流资源账户                    | 17 |
| 3.5.4 按工作流汇总的 EEVDF              | 17 |
| 3.5.5 唤醒延迟与公平性                   | 17 |
| 3.5.6 验证结果                       | 18 |
| 3.6 AgentOS 控制台综合工作流             | 18 |
| 3.6.1 科研恢复场景                     | 18 |
| 3.6.2 遍历与索引路径的配对实验               | 19 |
| 3.6.3 核心查询与完整流程窗口                | 19 |
| 3.6.4 系统结果                       | 20 |
| 3.7 综合场景：AgentOS Nexus 多智能体协作平台  | 20 |
| 3.7.1 长驻会话与四角色协作                 | 20 |
| 3.7.2 一次交互回合                     | 20 |
| 3.7.3 N1 子任务协议                   | 22 |
| 3.7.4 结果文件传递                     | 22 |
| 3.7.5 按角色显示工具                    | 23 |

|       |                                      |    |
|-------|--------------------------------------|----|
| 3.7.6 | 调用级审批与取消                             | 23 |
| 3.7.7 | 内核观测信息                               | 23 |
| 3.7.8 | 运行验证                                 | 23 |
| 3.8   | 关键实现                                 | 24 |
| 3.8.1 | 启动、身份与 workflow 代次                   | 24 |
| 3.8.2 | 结构化工具调用与执行约定                         | 27 |
| 3.8.3 | 上下文、回滚边界与数据来源                        | 35 |
| 3.8.4 | 元数据查询与实时查询事件                         | 39 |
| 3.8.5 | 任务通道与等待路径                            | 45 |
| 3.8.6 | 资源使用租约与 workflow <b>EEVDF</b>        | 50 |
| 3.8.7 | <b>Guest runtime</b> （客户机运行时）与结果文件发布 | 58 |
| 第四章   | 项目测试                                 | 65 |
| 4.1   | 功能与性能测试                              | 65 |
| 4.1.1 | 测试组织                                 | 65 |
| 4.1.2 | 任务一至任务五的 <b>Guest</b> 综合验收           | 66 |
| 4.1.3 | 结构化工具与任务通道测量                         | 66 |
| 4.1.4 | 结果文件发布与冲突处理                          | 67 |
| 4.1.5 | 文件查询路径的 <b>I/O</b> 观测                | 67 |
| 4.1.6 | 测量数据与可重复性                            | 68 |
| 4.2   | 测试路径与结果                              | 69 |
| 4.2.1 | <b>Guest</b> 原生工作负载                  | 69 |
| 4.2.2 | 工具、执行约定与任务通道测试                       | 71 |
| 4.2.3 | <b>Nexus</b> 会话与 <b>Host Relay</b>   | 75 |
| 4.2.4 | 失效状态与恢复分支                            | 77 |
| 第五章   | 总结与展望                                | 78 |
| 5.1   | 阶段性结论                                | 78 |
| 5.2   | 当前技术限制                               | 78 |
| 5.3   | 后续工作                                 | 78 |
| 第六章   | 参考说明                                 | 79 |
| 6.1   | <b>uCore</b> 与系统基础                   | 79 |
| 6.2   | 协议、工具与 <b>Runtime</b>                | 79 |
| 6.3   | 测量资料                                 | 79 |
| 6.4   | 文献使用                                 | 79 |
| 第七章   | 参考文献                                 | 80 |

# 1. 项目背景

智能体的“计划-调用-等待-修正”循环会同时经过进程、文件、系统调用和调度器。若角色、 workflow 标识和来源关系只保存在应用层，内核看到的便只能是无差别的线程和文件描述符，因而无法在创建、副作用提交、资源占用或唤醒服务等时点作出一致判断。本章依次讨论 AgentOS-uCore 所要解决的问题、uCore 的扩展边界和设计目标。

## 1.1. 项目背景与设计目标

### 1.1.1. 智能体对操作系统的新需求

大模型工具调用已从单次问答发展为持续运行的智能体循环。典型工作流会反复执行“读取上下文、选择工具、等待结果、修正计划”，并将任务拆分后委派给多个专职智能体。这类负载具有三个突出特点：生命周期较长，工具副作用丰富，协作过程中会产生大量中间状态。

传统进程抽象记录 PID、内存、文件和 CPU 时间，却无法直接表达智能体的角色、工作流代次、上下文因果关系、工具模式和数据来源。若这些信息只保存在提示词、JSON 格式文本和应用日志中，内核便难以在文件、IPC 与资源操作实际发生前进行统一检查。单一角色还可以通过创建更多线程扩大调度份额，进而影响其他工作流获得服务的时间。

我们将 AgentOS-uCore 的目标概括为：**使操作系统能够识别智能体工作负载，并在文件、消息和资源等副作用发生时提供一致的身份、资源、因果与调度机制。**

### 1.1.2. 基于 uCore 的系统扩展

项目建立在 LearningOS/uCore 教学内核之上 [1]。uCore 提供 RISC-V 启动流程、进程、虚拟内存、VFS、异常处理、系统调用和基础调度机制。我们保留这些主干，只在现有 PCB、VFS 对象、调度队列和系统调用路径中补充智能体状态。所有新用户态接口均使用定长结构，并明确写入版本号和结构大小，再由内核和用户态共同检查布局。普通 uCore 进程仍沿用原有路径；智能体进程则通过受控的创建和 exec 路径取得内核身份。

文件元数据采用当前启动周期的内存附属记录，并与实际的 dev+inum+incarnation 绑定。这样既不改变原有 uCore 磁盘格式，也能分别维护索引、订阅和增量缺口后的重新同步。

### 1.1.3. 机制与策略的分层

文件、消息、资源和调度等必须由内核裁定的动作留在内核；目标拆分、历史整理和工具选择留在 Guest 用户态；传输加密、服务接口密钥和模型服务协议则留在 Host（宿主机）侧。这样，无论使用固定策略、严格回放还是实际模型服务，Guest 都经过同一条内核调用路径。

| 系统对象  | uCore 基础           | 新增机制                                 |
|-------|--------------------|--------------------------------------|
| 进程与线程 | PID、内存、文件表、运行状态    | 智能体身份、角色与能力位、控制标识、 workflow 生命周期     |
| 虚拟内存  | 页表、用户页与内核页         | 六页只读的上下文镜像与一页用户缓存                    |
| 文件系统  | inode、目录、读写与 fsync | 以本次启动为范围的元数据、选择性索引、带类型信息的实时查询        |
| 系统调用  | 参数拷贝、执行与返回         | 结构化工具目录、带类型信息的 V2 与 V3、任务通道的 SQ 与 CQ |
| 进程间通信 | 管道、信号与基本等待         | 消息路由、订阅与等待、心跳、模型请求关联号                |
| 调度与资源 | 线程运行队列             | workflow 调度实体、资源账户、EEVDF 的滞后量与虚拟截止时间 |

表 1.1: uCore 基础与 AgentOS-uCore 的扩展

| 层次         | 主要对象                                | 职责                              |
|------------|-------------------------------------|---------------------------------|
| Host 侧     | 命令行、观察程序、模型服务提供方 (Provider)、QEMU 串口 | 负责交互、传输、加密和消息格式转换               |
| Guest 用户态  | 协调智能体、专职智能体、N1、artifact 存储          | 负责拆分目标、维护任务状态、整理 artifact 并回传结果 |
| AgentOS 内核 | 身份、上下文、工具、VFS、IPC、资源、调度器            | 负责身份检查、权限判断、数据来源记录、等待、资源记账与调度   |

表 1.2: AgentOS-uCore 的三级职责划分



1.1.4. 相关机制与技术选择

资源记账借鉴 Linux 的 `cpuacct` 与 `rstat` 分层汇总思路 [2, 3]，执行记录借鉴 BPF 环形缓冲区的有序提交方式 [4]，调度采用 EEVDF 的滞后量与虚拟截止时间概念 [5]。任务通道采用 `io_uring` 的 SQ 与 CQ 通信形式 [6]，文件元数据和选择性索引参考 Haiku BFS[7]。智能体工具循环中“模型选择、应用执行、结果回传”的职责划分借鉴现有工具调用实践 [8, 9]。

在 `uCore` 中，身份、上下文、VFS 访问范围、资源账户和 EEVDF 都记录同一个 `id+generation`。因此，文件查询、工具调用和调度器在判断某个智能体归属时会得到同一答案。

1.2. 设计目标与阅读线索

项目没有另起一套脱离 `uCore` 的 Agent runtime（运行时），而是把赛题关注的能力放进现有内核对象。Agent 进程创建与地址空间设计负责回答“谁在运行、哪些数据需要内核保护、哪些数据应由用户态直接读取”；Agent-OS 内核结构化交互接口与工具调用协议负责表达请求及其结果；上下文路径和文件查询保留每轮行动所依据的信息；事件等待与工作流调度负责 Agent Loop 的休眠、唤醒和 CPU 服务。每条路径都有相应的 Guest（客户机）程序和 Host（宿主机）侧验证，性能比较只使用工作负载和结果指纹一致的样本。

| 智能体运行环节  | 内核提供的机制                              | 文中可核对的现象                                                                                                   |
|----------|--------------------------------------|------------------------------------------------------------------------------------------------------------|
| 创建与装入    | 受信任映像、智能体身份、控制标识、七页 Context 区、生命周期代次 | Agent 与普通进程共存； <code>create</code> 、 <code>fork</code> 、 <code>exec</code> 、 <code>exit</code> 后的身份和映射保持一致 |
| 发起工具调用   | 工具目录、V2/V3、参数 schema、执行约定、任务通道 SQ/CQ | Guest 可列举并调用工具；错误参数、过期请求、重试、取消和背压得到不同结果                                                                    |
| 积累并检索信息  | 上下文路径、来源追溯、文件元数据、摘要、索引与订阅            | 路径可直接读取并有界淘汰；按属性查询与遍历结果一致，索引查询减少候选扫描                                                                       |
| 等待下一轮与协作 | 消息路由、原子等待、心跳、资源账户、工作流 EEVDF          | 无事件时休眠，事件到达后唤醒；线程数增加不会等比例扩大工作流的服务份额                                                                        |

表 1.3: 从 Agent Loop 的运行环节到内核可观察行为



## 2. 系统设计

AgentOS-uCore 沿用 uCore 的进程、页表、VFS、异常处理、系统调用和调度主干，不以独立运行时（Runtime）取代内核对象。一个智能体发起工具调用时，身份检查、上下文记录、文件访问、消息投递、资源记账和调度选择都读取进程中保存的同一组 workflow 编号和代次。图示、数据结构和状态机明确规定这些模块何时读取字段、何时更新状态，避免同一请求在不同位置被认作不同工作流。

### 2.1. AgentOS-uCore 总体架构

#### 2.1.1. 以 uCore 为底座的产品分层

AgentOS-uCore 按部署职责分为五部分：uCore 基础内核、AgentOS 内核模块、Guest Runtime、Host Relay 和 Agent 应用。图2.1自下而上展示这五部分：uCore 提供进程、内存、VFS、系统调用和调度器等基础对象；AgentOS 在这些对象上补充智能体身份、工具、Context、文件查询、Loop 和工作流调度；Guest Runtime 封装统一 UAPI，Host Relay 连接串口、TLS 与模型 Provider，控制台、Nexus 与科研工作流则负责具体任务。

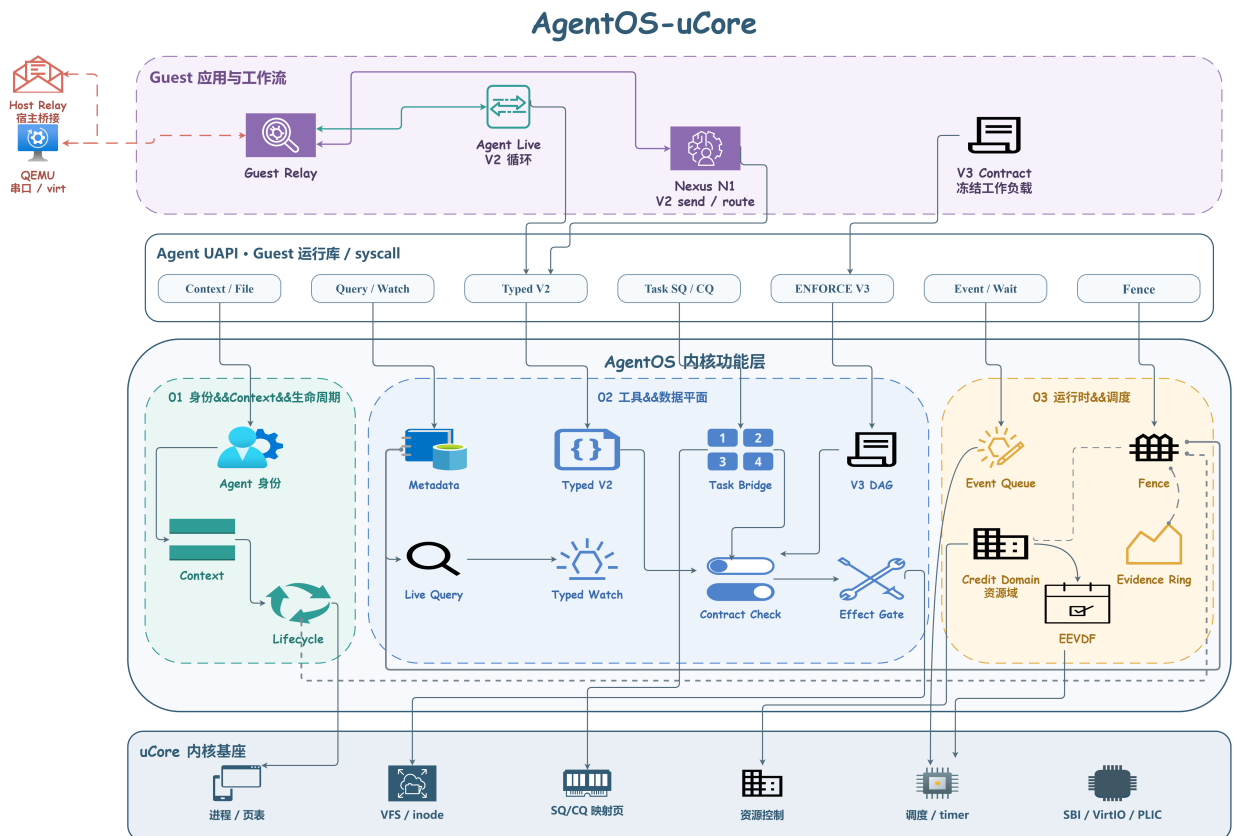


图 2.1: AgentOS-uCore 总体架构

底层 uCore 主干继续承担进程、页表、VFS、异常处理、系统调用和调度器的原有职责。AgentOS 在此基础上把相关机制分为三组：

1. 身份、上下文和资源：身份与生命周期登记智能体；上下文保存每次工具交互；资源

账户和 EEVDF 分别记录资源使用并分配 CPU 服务。

2. **工具、文件和消息**：带类型信息的工具调用检查 schema 和能力位；VFS 元数据与实时查询处理文件对象；消息路由传递跨智能体消息。
3. **来源记录和工作流结束**：上下文路径记录因果关系；来源追溯随文件、工具和 IPC 传播；执行记录环与 Workflow Fence（工作流栅栏）分别保存阶段记录和结束时的快照。

### 2.1.2. 六项核心机制与请求流向

| 模块             | 关键对象                        | 主要功能                                       |
|----------------|-----------------------------|--------------------------------------------|
| Agent 进程与地址空间  | 身份记录、七页 Context 区、控制标识、生命周期 | 创建受信任 Agent，绑定角色、能力位和工作流代次，并把高频状态映射给 Guest |
| 结构化交互与工具协议     | 工具目录、V2/V3、SQ/CQ            | 发现工具、检查带类型信息的参数，并支持动态调用、冻结执行约定和有界任务提交      |
| 上下文路径与来源追溯     | 路径镜像、分支、执行记录环               | 连接请求、结果、前驱记录和数据来源，并按配额淘汰旧记录                |
| 面向 Agent 的文件查询 | 元数据、摘要、索引、订阅器               | 为显式登记的文件提供属性查询、结构化结果、变更通知和缺口重新同步           |
| Agent Loop     | 事件队列、路由、心跳                  | 提供原子等待与唤醒、消息、模型请求关联、超时和取消                  |
| 工作流资源与调度       | 资源账户、EEVDF 实体               | 按工作流汇总资源记账，并根据滞后量和虚拟截止时间分配 CPU 服务          |

表 2.1: AgentOS 的核心模块

六项机制使用同一个工作流 id+generation。一次工具调用会同时取得调用者身份、前一条上下文记录、schema、数据来源、资源账户和调度实体。工作流代次变化后，已经回收槽位对应的订阅器和句柄不能再指向新对象。

### 2.1.3. 一次智能体工具调用

一次完整调用从 Guest 读取工具目录开始。Guest 按角色列出可用工具，再将模型结果编码为带类型信息的请求。请求进入内核后，AgentOS 依次拷入数据，检查版本和大小，检查身份与生命周期，解析工具并解码 schema，再检查执行约定、数据来源、资源和副作用，最后调用 VFS、IPC 或其他 uCore 子系统。返回前，上下文路径写入一条新记录，下一轮可引用其序号。

当智能体等待模型、文件事件或其他角色时，agent\_wait 将线程置于等待状态。事件入队与等待者安装按原子顺序执行，从而避免“发现队列为空后恰好错过唤醒”。

### 2.1.4. 上层 runtime

控制台按固定策略依次执行身份检查、上下文记录、文件查询、IPC 和恢复操作，用于功能演示和配对测量。Nexus 使用同一组系统调用维持会话、分派任务、整理结果文件并连接模型服务。两者访问同一批 AgentOS 内核对象，因此确定性测试和自然语言交互遵循相同的系统规则。

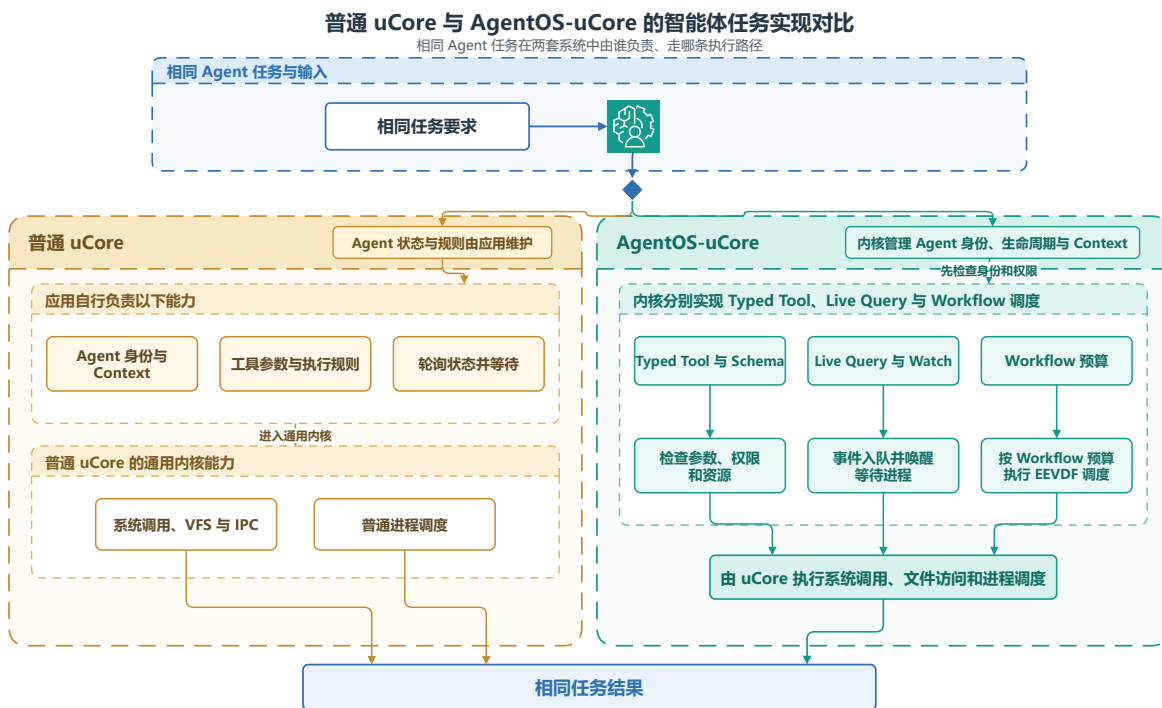


图 2.2: 同一科研恢复任务在普通 uCore 与 AgentOS-uCore 中的执行路径

图2.2的比较目标是说明：业务步骤不变时，AgentOS 把哪些原本散落在 runtime 中的约束交给了内核。普通 uCore 仍可用进程、文件和 IPC 串起科研恢复任务，但角色、上下文和等待关系只能由应用自行约定；AgentOS-uCore 在相同流程中加入可识别的工作流标识、结构化工具、选择性索引、事件等待和工作流调度，使身份检查、数据来源和资源归属贯穿每个阶段。

## 2.2. 跨模块状态约束

系统将工作流标识写为可复用槽位的 id+generation。模块使用该标识前，先确认对象仍在运行、访问范围相同且代次匹配；资源、元数据和执行记录也按该标识归属。这样，旧槽位关闭、回收并再次使用后，先前的请求不会被新工作流接收。

### 算法 1：工作流标识的跨模块校验

**输入：**调用进程、目标对象、请求携带的工作流编号和代次、访问范围。

**输出：**可继续执行的对象引用，或明确的 STALE、DENIED 结果。

1. 从调用进程读取内核保存的工作流编号、代次和能力位。
2. 检查工作流编号和代次的结构，确认生命周期仍处于活跃状态并且代次匹配。
3. 比对调用者、目标对象和请求的访问范围；如不一致，则拒绝提交副作用或传递事件。
4. 在工具、元数据、任务通道、执行记录或调度路径中，分别执行相应的参数和资源检查。
5. 只有在全部检查通过后，才发布状态、将事件入队或提交资源。

### 3. 系统实现

本章依据当前仓库中的内核和用户态源代码。内核代码主要位于 `os/agent_*.c`、`os/workflow_*.c` 和 `os/resource_controller.c`；用户态场景位于 `user/src/`，Nexus 还使用 `user/lib/agent_nexus.c` 和 `host_tools/agentos_relayd.py`。各节先说明要解决的问题和内核采取的动作，再给出可直接定位的源代码片段。

| 实现部分   | 主要源文件                                                                                                                                                                          | 关键责任                               |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| 启动与身份  | <code>os/main.c</code> 、<br><code>os/agent_identity.c</code> 、<br><code>os/workflow_lifecycle.c</code>                                                                         | 初始化智能体子系统，分配角色、控制标识和工作流生命周期。       |
| 上下文与执行 | <code>os/agent_context.c</code> 、<br><code>os/agent_core.c</code> 、<br><code>os/agent_execution_contract.c</code>                                                              | 写入上下文，解析 V2 与 V3 请求，并按执行约定检查副作用。   |
| 任务与来源  | <code>os/agent_task_channel.c</code> 、 <code>os/agent_provenance.c</code> 、 <code>os/agent_evidence_ring.c</code>                                                              | 管理任务通道的 SQ 与 CQ、来源标签和工作流执行记录的阶段快照。 |
| 文件与事件  | <code>os/agent_metadata_*.c</code> 、 <code>os/agent_file_state.c</code> 、 <code>os/fs.c</code> 、<br><code>os/agent_live_query_events.c</code> 、<br><code>os/agent_ipc.c</code> | 维护元数据目录、查询与订阅、结果文件发布、消息和等待。        |
| 资源与调度  | <code>os/resource_controller.c</code> 、 <code>os/workflow_credit_domain.c</code> 、<br><code>os/workflow_scheduler.c</code>                                                     | 处理资源使用租约、额度、工作流级选择和资源记账。           |
| 集成场景   | <code>user/src/labdemo_ucore.c</code> 、 <code>user/src/agent_nexus_ucore.c</code>                                                                                              | 驱动恢复流程和四角色常驻协作。                    |

表 3.1: AgentOS-uCore 的实现源代码分工

#### 3.1. Agent 进程创建与地址空间设计

##### 3.1.1. 从普通进程到工作流中的 Agent

AgentOS 区分普通进程、协调 Agent 和专职 Agent。VFS、IPC、工具执行器和调度器读取同一份身份记录，因此同一个进程在各处拥有相同的角色和能力位。内核将 Agent 身份写入 PCB，再根据受信映像清单和父角色授权完成创建。用户态可以读取身份信息，权限判断始终以内核副本为准。普通进程仍走 uCore 原有的创建与装入路径，两类进程可以在同一系统中并存。

创建受控工作流时，内核先分配生命周期，再写入身份、角色、能力位和控制标识，随

后建立 **Agent Context** 页、VFS 访问范围和资源账户。任一步骤失败，内核都会回收已经分配的对象。创建完成后，各子系统都通过同一组 workflow 编号和代次找到该 **Agent** 所属的工作流。

用户态的 `agent_create()` 和 `agent_create_role()` 进入内核的 **Agent** 创建路径：`agent_info()` 从 **PCB** 返回身份、角色、配额、**Loop** 状态和 **Context** 位置，`agent_workflow_lifecycle_info()` 再给出完整生命周期键。创建接口负责建立对象，查询接口只复制内核已经保存的元信息，两者都不会让用户态自行填写权威身份。

`agent_worker_create()` 不会新建 workflow 根生命周期。它只在已有生命周期中创建一个携带受控待执行映像的子进程；调用者只要在同一 workflow 内具有 **ORCHESTRATE** 即可调用，不要求它必须是控制者。子进程仍须沿通常的 `exec()` 路径装入映像，所以该调用返回时，子进程尚未在新映像中运行。`agent_scope_delegate_fd()` 只接受 **FD\_INHERIT\_DELEGATE**，当前实现用于管道。该接口不会立即把文件描述符放入某个访问范围，而是在调用线程上登记一次性凭据；下一次创建受控子进程时，内核制作创建快照才会使用该凭据。子进程通过 `filedup()` 获得文件描述符副本，父进程继续保留原描述符，随后内核清除这张凭据。

### 3.1.2. 身份字段与角色策略

| 字段                                | 含义                     | 使用模块                                                       |
|-----------------------------------|------------------------|------------------------------------------------------------|
| <code>agent_id</code>             | 当前启动周期内的智能体编号          | 工具、上下文、观测                                                  |
| <code>control_id</code><br>角色与能力位 | 内核分配的控制身份<br>角色及其具体权限集 | 控制者绑定、IPC 路由、句柄校验<br>工具、文件、消息与 <b>artifact</b> （工作流结果文件）操作 |
| 工作流<br><code>id+generation</code> | 工作流标识和可复用槽位的代次         | 生命周期、订阅、任务通道、资源、调度                                         |
| VFS 访问范围                          | 工作流的文件访问范围             | 元数据、查询、 <b>artifact</b>                                    |
| 执行与存储账户                           | 工作流资源归属                | 进程、线程、文件、块、页与缓冲区                                           |

表 3.2: 智能体身份的核心字段

内核的角色策略将每个角色映射为能力位掩码和调度权重。哨兵角色主要读取进程、元数据和调度状态；调查角色只能读取受限内容；报告角色可以创建报告结果文件；协调角色可以分派任务并创建角色。子智能体继承同一工作流的访问范围，其能力位同时受父角色授权和映像清单限制。

### 3.1.3. Agent Context 区的七页布局

每个 **Agent** 在用户地址空间的固定 **ABI** 地址拥有七页 **Agent Context** 区。前六页由内核维护，并以只读方式映射到 **Guest**，依次提供身份头、最近返回、上下文记录、路径索引和稳定摘要；第七页可由用户态读写，用于保存查询结果和其他高频策略数据。这样，身份、能力位、访问范围等权威状态仍受内核保护，**Guest** 读取 **Context** 和管理查询缓存时又不必为每个字段反复进入内核。

上下文最多保存 128 条记录。每条记录包含序号、请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识和记录摘要。内核按发布序号更新镜像；**Guest** 既可



以调用查询与快照接口，也可以直接读取同一结构。PCB 保存身份、配额、Loop 状态和路径位置等低频元信息，具体路径则通过 Context 区向用户态发布，这一划分兼顾了内核裁定与高频读取。

### 3.1.4. 生命周期代次与并发控制

workflow槽位可在回收后复用，内核每次重新分配时都会推进代次。订阅器、资源账户、VFS 附属记录、任务句柄和 artifact 都同时保存标识和代次。

生命周期使用普通操作、退出操作和 Workflow Fence（工作流栅栏）三类并发控制。普通系统调用增加普通操作计数；exit 或拆除过程增加退出操作计数；Workflow Fence 在两类操作全部完成后生成阶段快照。当控制者已经关闭、成员数、活跃普通操作数和退出操作数都归零，并且栅栏已经释放时，回收程序会清除该生命周期的上下文、订阅器和执行记录环。普通 VFS 访问范围会被完整回收；对于 preserve\_on\_retire 持久输出访问范围，只要存储账户进入 DRAINING 或 FREE，内核即可回收生命周期代次，同时保留登记项、文件和存储额度。

### 3.1.5. 随工作流创建的资源主体

工作流创建时，内核同时分配执行账户和存储账户。工作流中的进程、线程、文件对象、文件系统块、inode、智能体状态页和物理页都记入同一账户。这样，资源使用租约和工作流 EEVDF 都按工作流计算，子进程和多线程也始终记在原工作流名下。

### 3.1.6. 验证结果

agenttrust\_ucore 覆盖受信 exec、角色授权、能力位收缩以及 fork、exec、exit 与普通进程共存；agentscope\_ucore 覆盖访问范围与代次复用；agentfinal\_ucore 覆盖上下文固定映射、快照、分支、回滚和容量淘汰。所有场景都在 RV64 uCore Guest 中通过真实系统调用运行。

## 3.2. Agent-OS 内核结构化交互接口

### 3.2.1. 工具调用协议与工具目录

Agent 既要发现可用能力，也要把工具名称、参数和返回状态交给内核可靠解析。AgentOS 因此没有让 Guest 传入任意 JSON 文本，而是定义了带版本和大小字段的工具调用协议。内核维护 25 项工具目录，每个描述项写明工具编号、名称、schema（参数模式）、标志和副作用类别；启动时检查编号与名称是否唯一，并检查 schema 是否完整。

请求使用工具名称或编号加一组 typed 键值参数，响应返回状态码、Context 序号、结果类别和定长结果字段。版本不符、参数类型错误、工具不存在、能力不足和生命周期过期分别返回可区分的状态；用户态不需要从一段错误文本中猜测失败原因。目录项和请求结构都带 version 与 size，后续扩展可以在明确拒绝旧布局的前提下进行。

sys\_tool\_list 向 Guest 返回定长描述项。Guest 再按角色和能力位列出模型可见的工具，并补充用途、参数和结果说明。等待、上下文和模型请求等 runtime（运行时）接口只供 Guest 使用，模型只能请求当前角色允许的产品动作。

### 3.2.2. 自适应的 V2 调用

V2 请求明确携带版本号、请求大小、工具名称、工具编号、参数数量和带类型信息的参数数组。当前标量类型为 `uint64` 与字符串，适合 `uCore` 的紧凑参数解析。内核先把完整参数拷入内核缓冲区，依次检查参数名、类型、数量和长度，再进入共享执行器。

V2 适合根据上一轮结果改变下一步操作的智能体循环。例如，协调智能体可以先查询失败阶段，再根据候选数决定读取摘要还是创建研究子任务。每次调用都会独立检查身份、能力位、访问范围、`schema`、数据来源和资源状态。

### 3.2.3. 四种执行路径

| 路径            | 输入形式                                        | 主要用途         | 运行语义                                                |
|---------------|---------------------------------------------|--------------|-----------------------------------------------------|
| 紧凑批处理         | 最多 64 个 <code>agent_op</code>               | 固定顺序和上下文压力场景 | 在一次系统调用内顺序执行，逐项返回状态                                 |
| V2            | 带类型信息的键值参数                                  | 依据上一轮结果选择工具  | 每次都检查 <code>schema</code> 、能力位和访问范围                 |
| ENFORCE V3    | V2 前缀与 <code>Execution Contract</code> 绑定信息 | 预先定义的高保障 DAG | 冻结节点、前驱、尝试次数、截止时间和副作用清单                             |
| 任务通道的 SQ 与 CQ | 16 槽 SQ 与 16 槽 CQ                           | 有界提交、完成和背压   | 仅主线程建立通道，绑定提交线程并校验代次；每个已接受任务恰有一条完成 CQE，取消不会额外产生 CQE |

表 3.3: 四种工具执行路径

紧凑批处理用于同步执行短操作序列。V3 则在工作流生命周期内冻结一份带版本的执行约定，其中最多包含 24 个按拓扑顺序排列的节点。每个节点关联工具、`schema` 摘要、所需能力位、输入和输出 `artifact` 类型、截止时间、尝试次数、来源与副作用清单，以及执行和存储额度。

V3 请求同时绑定执行约定代次、节点、尝试次数、来源节点、前驱上下文序号和 32 字节指纹。无论成功还是失败，完成结果都会写入固定完成槽；合法重试直接读取原结果，避免再次产生副作用。

### 3.2.4. 工具执行期间的资源使用租约

高风险工具可以在执行前从工作流账户预留执行和存储额度。资源使用租约依次经过 `ADMITTED`→`ACTIVE`→`DEACTIVATED`→`SETTLED`。分配器在发布对象前取得 `nonce` 声明，发布成功的对象继续占用同一份已用额度，析构时再结算。这样，多个对象在同一轮工具调用中同时达到资源峰值时，账户仍能准确记录用量。

### 3.2.5. 带类型信息的任务通道

任务通道只能由智能体主线程建立，并映射 16 槽 SQ 和 16 槽 CQ，两者各占一个 `Guest` 映射页；复制的请求和完成记录、权威资源状态则分别保存在两页内核私有页中。SQE 与



CQE 均为 128 字节。进入通道和资源绑定会记录提交线程标识及身份代次。内核先拷入完整 SQE，再检查进入接口的 ABI、递增请求编号、执行约定、节点、尝试次数、schema、截止时间、取消链接和带类型信息的句柄。提交者、所有者或生命周期不匹配导致 STALE 时，输出只保留 ABI 版本、结构大小和 status，其余字段全部置零；控制标识代次不匹配时，进入接口返回当前通道状态。

内核私有的头尾指针才是权威状态。CQ\_FULL 只表示 CQ 暂无可写槽位，是背压，不表示已接受任务已经完成；对应任务仍待发布，直到 CQ 出现可用槽位。SQE 的环或槽位代次不一致，或者发生其他协议错误，都会使通道进入需要重新同步的状态；提交者必须以 AGENT\_TASK\_CHANNEL\_ENTER\_F\_RESYNC 和空 sq\_tail 显式复位，随后才能继续提交。每个已接受任务在成功、拒绝、失败、取消或超时后发布一条完成 CQE。CANCEL 命令使用独立请求编号，只引用目标任务，不会另行生成 CQE；同步策略拒绝仍会消费取消编号，并由进入接口返回 DENIED，目标任务继续保持原运行状态。目标任务已经确认或不存在时，同样会消费取消编号，进入接口返回当前代次和水位线。Task Bridge 既接受空输入，用于验证通道、截止时间、取消和完成语义；也允许 ECHO Provider（服务提供者）读取 UTF-8 资源 snapshot，检查带类型信息的输入能否进入同一个工具执行器。

任务输入也可以由 IMPORT 控制请求从当前进程的只读普通文件描述符导入。请求进入时，内核先固定文件引用和当次 workflow/contract cut；实际读取通过带租约的 readi 从 offset 0 多探测一个字节。只有文件恰好在给出的 1-63 字节处结束、内容不含 NUL 且 UTF-8 合法时，才建立资源。导入提交前，内核在同一条事务 lane 中再次核对 Context 序号、哈希和来源标签，避免把文件内容与另一时刻的 Context 拼成一条来源记录。

导入过程不改变文件描述符的共享 offset，也不把 structfile\* 留在通道中；内核只在私有资源槽保存不可变的 inline snapshot、SHA-256、最新 Context 序号和 UNTRUSTED\_FILE\_DATA 标签。IMPORT 只登记输入，不另行追加 Context 记录。定向 Guest 还让另一线程与导入并发执行 unlink/close，用来检查固定引用和事务边界能够走通。

资源句柄由 slot、type 和 generation 共同标识。BORROWED 输入执行后仍保持 LIVE，可由后续任务继续引用；OWNED 输入在完成后自动消费，显式 RELEASE 也会使旧句柄变为 STALE。ECHO Provider 从内核 snapshot 复制 payload，不再依赖导入文件的后续内容或 offset。若 IMPORT 已成功而最终结果无法复制回用户态，内核会撤销尚未暴露的资源，避免只有内核知道的槽位泄漏。

### 3.2.6. 验证结果

agenttoolabi\_ucore 覆盖目录发现、名称与编号一致性、未知参数、类型错误和返回缓冲区；agentcontract\_ucore 覆盖 DAG、截止时间、重试、取消、来源 Context 与来源检查；agenttask\_ucore 除了 SQ/CQ 已满、重新同步、取消和完成记录保留，还检查 UTF-8 文件 snapshot、BORROWED/OWNED 生命周期、显式释放、generation 防 ABA 以及并发 unlink/close 下的事务路径。最终 copyout 失败后的回滚分支由结构检查、mutation 测试和资源表模型核对。任务通道的测量数据另保存了 96 条每条 16 个操作的序列，第 4 章集中分析其分布。

## 3.3. 上下文路径管理、数据来源与 Workflow Fence

### 3.3.1. 从多轮调用到 Context Path

Agent 的下一步决策往往取决于前几轮工具结果。上下文路径管理把这些请求和结果按前驱关系组织为 **Context Path**：每次工具调用即将执行可能产生副作用的操作时，AgentOS 建立一条上下文记录，并由附属记录保存归一化的 `agent_op`、结果和 `provenance`（来源追溯）信息。附属记录不照搬完整的 V2 参数、V3 绑定或任务通道 `SQE`，而是保留后续决策真正需要的请求、结果、因果关系和数据来源。

记录序号只在同一次清空周期内递增。记录包含请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识、短载荷或结果以及记录哈希；执行清空后，序号从 1 重新开始。`path_parent_sequence` 指向当前活动头记录，后继索引用于查询分支。

上下文追加依次经过准备、填充、发布和提交；`detail` 会在记录标为奇数前写入。镜像头发布记录数、头记录、最早记录、最新记录、可见头记录、丢弃数和回滚数。**Guest** 读取记录后重新核验序号和哈希，并再次确认摘要未变化，便可得到一致快照。

用户态通过 `context_push()` 追加记录，也可以用 `context_query()` 复制活动路径；高频读取时则直接检查 **Context** 镜像。`context_rollback()` 把可见头移到仍在窗口内的历史记录，`context_clear()` 清空当前路径。系统调用与直接映射读取使用相同的序号和摘要，因此两种读取方式可以互相核对。

### 3.3.2. 分支、回滚与有界淘汰

上下文路径最多保留 128 条记录。容量用尽后，内核按序号淘汰最早记录，并更新最早记录、最新记录和丢弃计数。活动路径只引用仍然保留的前驱记录。

回滚将 `visible_head` 移到仍然存在的上下文序号，并为后续调用分配新的分支代次。已经产生的文件、IPC 和模型服务副作用保持不变。需要幂等的副作用由 V3 尝试缓存和具体工具协议处理。因此，上下文分支只改变后续决策看到的路径；手工备注、回滚和清空都不会预留或提交执行记录。

### 3.3.3. 数据来源传播

AgentOS 使用六类可组合的数据来源标签，标签随上下文、文件、工具返回、IPC 和 `artifact` 传播。

文件、工具、邮箱和任务资源按保守的按位或规则传播来源标签。生成摘要、重新计算哈希和在智能体之间整理 `artifact` 时会增加新来源，同时保留既有标签。因此，文件数据可以供模型查看，但不会自动变成控制授权。

### 3.3.4. 工具清单与副作用检查

每个内核工具都有安全清单，其中列出可接受的输入来源标签、输出新增标签、所需能力位，以及文件、元数据、IPC、进程、权限、`artifact` 和订阅的副作用掩码。启用 `ENFORCE V3` 后，生命周期代次、执行约定中冻结的边、`schema`、能力位、工具清单和数据来源必须同时匹配。失败请求会在工具产生副作用前返回结构化状态。直接调用或 V3 调用被拒绝时通常记录来源上下文、工具和原因；但是关键记录的预留或提交失败时，调用会返回 `NO_SPACE` 或 `RETRY`，因此不能认为每次拒绝都一定有执行记录票号。

| 标签                        | 含义                      | 典型来源                    |
|---------------------------|-------------------------|-------------------------|
| KERNEL_FACT               | 内核生成的结构化事实              | PID、资源、调度器、能力位          |
| TRUSTED_USER_CONTR<br>OL  | 经产品界面提交的用户控制信息          | 新交互轮次、参数绑定<br>审批        |
| AGENT_DERIVED             | 智能体根据已有输入推导的数据          | 摘要、计划、报告                |
| UNTRUSTED_FILE_DAT<br>A   | 来自文件的内容或元数据             | 查询、读取、摘要哈希              |
| UNTRUSTED_TOOL_OUT<br>PUT | 工具执行返回                  | 带类型信息的 V2 或<br>V3 结果    |
| CROSS_AGENT_DATA          | 经 IPC 或 artifact 在角色间传递 | 任务结果、协调者整理的<br>artifact |

表 3.4: 上下文的数据来源标签

### 3.3.5. 执行记录环

每个活动 workflow 按需分配四个智能体状态页：三页普通记录环，共 48 个 256 字节槽位；一页关键记录环，共 16 个槽位。直接调用或 V3 调用的标准完成或拒绝路径会预留递增的执行记录票号，填充后再提交；手工备注、回滚和清空不使用这套预留和提交规则。当较早编号仍在准备时，读取者会停在当前位置，从而保持全局发布顺序。

普通成功的上下文记录写入普通记录环，获准副作用和成功建立记录的拒绝写入关键记录环。拒绝记录无法预留或提交时，直接调用或 V3 调用返回 NO\_SPACE 或 RETRY，而不是伪造关键记录编号。复用环形空间前，已提交分段会汇入内部根记录。该结构为 Workflow Fence 提供按执行记录票号排序的执行记录。

### 3.3.6. Workflow Fence

控制者通过 `agent_run(count=0, AGENT_RUN_F_FENCE)` 发起 Workflow Fence。内核检查协调者与控制标识的关系，使生命周期进入屏障状态，排空元数据、实时查询延迟工作和 VFS 回收，检查执行与存储账户中的待结算额度归零，最后密封执行记录环。

非空的屏障请求只携带版本号、`struct_size`、必须为零的 `flags`、必须为零的 `reserved`、挑战值和请求编号，**不携带**前序根记录。请求和回执可以同时为 NULL，此时内核执行匿名且不等待结果的屏障。前序根记录只由内核在阶段快照完成后写入返回回执。提供回执时，生成的 320 字节回执绑定前序根记录、挑战值、屏障序号、执行记录票号范围、事件与缺口计数、元数据代次、额度周期、精确已用额度摘要和有序分段摘要。请求编号非零且提供回执时，内核在内部状态提交后复制结果；同一个请求编号再次复制时返回缓存结果。该回执表示当前启动周期内的工作流阶段快照，其标志同时记录元数据是否易失以及覆盖状态。

### 3.3.7. 验证结果

`agentfinal_ucore` 连续执行 64 次工具调用，并覆盖快照、分支、回滚和窗口淘汰。数据来源场景把文件来源传入受保护的副作用，验证副作用前的拒绝与关键记录环记录。屏障场景覆盖挑战值、重试缓存、待结算额度、元数据代次和结果复制重试。

### 3.4. 面向 Agent 查询优化的文件系统扩展

#### 3.4.1. 从路径访问到属性查询

科研和恢复工作流程经常需要回答两个问题：当前哪些对象满足条件，哪些对象刚刚发生变化。传统路径访问适合已知文件名的场景，却不能直接表达“查找某个阶段、状态或类别的结果文件”。AgentOS 在 uCore VFS 之上增加按启动周期隔离的元数据目录，为显式登记的文件维护项目、运行、阶段、状态、类别、依赖关系和内容摘要，使 Agent 可以按属性与摘要查询对象，也可以订阅后续变化。

#### 3.4.2. 元数据与 inode 代次

拥有 META\_WRITE 的智能体调用 `agent_file_meta_set()` 写入元数据，字段与真实 `dev+inum+incarnation` 绑定。更新通过 `update_mask` 完成，删除使用显式 DELETE 动作。文件创建和重命名仍沿用 uCore VFS，只有 workflow 明确关心的对象才会进入元数据目录。

对象代次用于区分 inode 编号复用前后的对象。VFS 以增量方式维护它，并随查询和订阅结果返回。查询或订阅命中记录后，内核会复核元数据目录代次、生命周期、访问范围和完整条件。unlink、内容大小变化和对象失效都会更新已登记的附属记录。

#### 3.4.3. 遍历查询与选择性索引

元数据目录为 `status`、`stage` 和 `kind` 维护选择性索引。带类型信息的查询只携带条件字段、查询标志和最大命中数；访问范围由调用者身份限定，查询计划由内核选择。结果返回命中数、候选记录数、扫描记录数、全部命中数、实际返回数、truncated、查询计划、计划原因和文件系统代次。调用者可以用 SCAN 强制遍历，也可以用 USE\_INDEX 允许内核使用索引。启用索引时，内核按固定的 `status->stage->kind` 顺序选择第一个已给定字段，不比较字段基数；索引路径仍会对候选记录检查完整条件，因此遍历与索引查询返回相同结果。最大返回数为 8，`total_hits` 仍报告全部命中数；当前 ABI 没有分页游标。

查询结果是带属性、摘要、digest 和对象代次的定长结构。Guest 可以把输出地址直接放在 Agent Context 区的第七页用户缓存中，由自己的策略决定保留多久；内核只管理这页的映射和配额，不在背后维护另一份不可见的查询缓存。

#### 3.4.4. 带类型信息的实时订阅与缺口重新同步

`agent_live_watch()` 安装一份完整的 `agent_file_query`。订阅器只保存查询条件、访问范围和代次，不保存持久结果集。每次元数据变化时，内核重新计算变更前后是否满足条件：对象从未命中变为命中时产生 ENTER，对象持续命中且记录改变时产生 UPDATE，对象从命中变为未命中或被删除时产生 LEAVE。事件以 `AGENT_EVENT_FILE_QUERY` 写入目标智能体的上下文和有界队列。

队列或待处理表无法保存全部增量时，内核会推进 `resync_generation` 并发送 `RESYNC_REQUIRED`。正常连续的文件系统代次变化仍只发布 ENTER、UPDATE、LEAVE，不会触发重新同步。恢复时，智能体保留旧订阅，先以相同条件安装不带 `ACK_RESYNC` 的替换订阅，再执行有界查询。只有 `truncated=0` 的快照才能作为完整基线；随后才在旧订阅



上以对应代次执行 ACK\_RESYNC 并取消旧订阅，替换订阅覆盖整个切换时段。单次查询最多返回 8 个命中项，当前没有分页游标；快照一旦截断就不能重新建立基线或确认，调用者必须收紧查询条件，分页留待未来 ABI。重新同步不能把 Workflow Fence 当作通用屏障。关闭工作流只会使生命周期进入关闭状态，不会排空待处理项；屏障会排空删除标记、内容增量并尝试投递标记事件。只要该工作流或访问范围中的重新同步尚未由 ACK\_RESYNC 确认，即使标记事件已投递，屏障仍返回 RETRY，最终回收才会清理残留对象。

3.4.5. 查询性能

实验使用目录规模 24、64、96 与精确命中数 1、2、4、8，组成 12 个实测网格。每个网格保留 15 个 AB/BA 内部配对，并记录遍历和 indexed 查询的延迟、候选数和检查记录数。本节及后续性能图表固定对应冻结在源码提交 2b14fb1f74b9bd093e6de939a16554620835699e 的测量批次；当前版本后续功能改动另由功能回归验证，未重新计入这组数据。

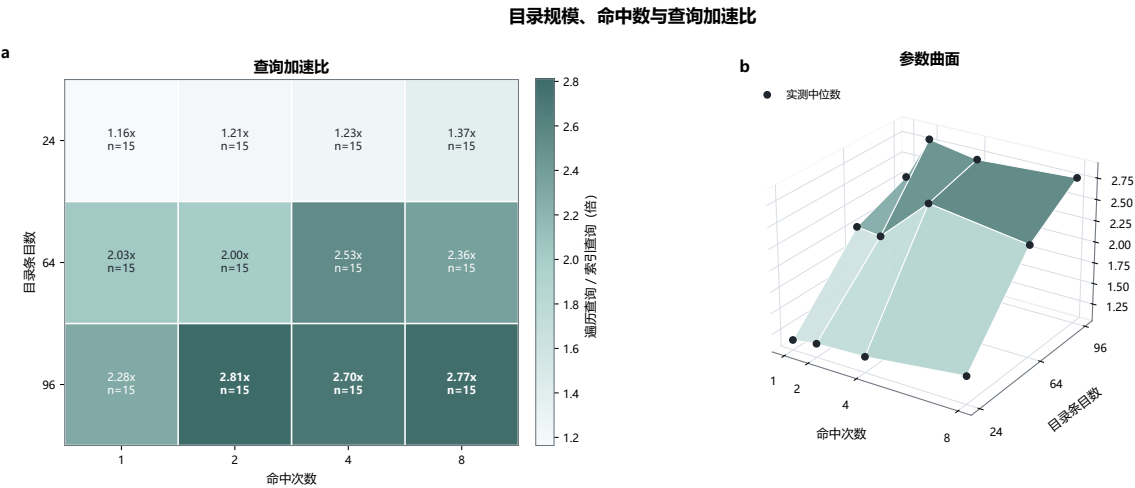


图 3.1: 目录规模、命中数与 indexed 查询加速比

图3.1给出的 12 个格点中，indexed 路径的中位加速比为 1.164–2.808 倍；目录规模从 24 增至 64、96 时，indexed 更快的内部配对数分别为 43/60、60/60、58/60，整体收益随规模增大而上升。其原因是 status->stage->kind 的固定选择顺序先缩小候选集，再逐项复核完整条件，而不是改变查询语义。命中数与加速比并不单调，24 项目录中仍有 17/60 个配对不占优；同时这些样本均使用 ready index，未计入建索引和维护索引的成本。因此，当前设计适合目录较大且筛选条件明确的已登记对象，不能把曲面上的最大倍数当作所有目录的固定收益；后续应把索引维护开销和低选择性查询纳入同一条路径评估。

图3.2显示，indexed 查询的中位耗时随目录规模由 98.3 微秒升至 108.3 微秒；首次 SCAN 为 388.5 微秒，后续 SCAN 为 255.1 微秒。首个 SCAN 不是冷启动，28 条诊断记录均标为 cache=ready，它反映的是该查询阶段的首次完整遍历而非 Host 页面缓存重新装载。固定索引选择减少了中位延迟及其对目录规模的敏感性，但 64 项目录的后续 SCAN 仍出现 25.7505 ms/query 的尖峰，说明尾延迟尚未稳定。该结果支持把 indexed 查询作为有明确筛选字段时的默认路径，同时保留 SCAN 作为语义等价的回退，并继续定位完整遍历和目录检查造成的长尾。

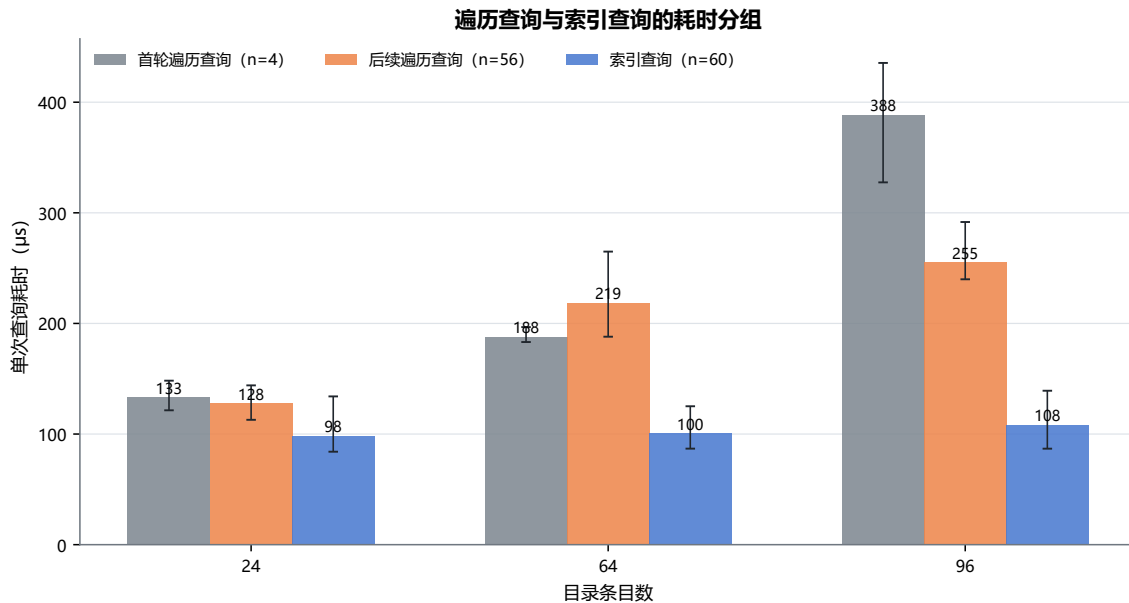


图 3.2: SCAN 与 indexed 查询的耗时分组

### 3.4.6. 验证结果

agentfs\_ucore 覆盖元数据设置、更新、删除、摘要、摘要哈希、对象代次和访问范围；agentbench\_ucore 使用同一数据集比较遍历与索引查询的结果集、候选数和延迟。实时查询场景覆盖 ENTER、UPDATE、LEAVE、队列缺口、RESYNC\_REQUIRED 以及替换订阅、快照、确认的安全切换。

## 3.5. Agent Loop 内核运行机制

### 3.5.1. 从轮询循环到内核等待

Agent Loop 会等待用户输入、模型回复、文件变化和其他 Agent 的消息。持续轮询不仅频繁进入内核，还会让大量等待线程占用调度时间。AgentOS 使用有界事件队列、订阅和等待接口，让没有事件的线程真正睡眠；消息、文件事件或 TIMER 到达后，内核再把相应线程唤醒。

每个智能体拥有容量为 16 的事件队列，其中保留一部分内核槽位，同一事件来源最多排队 4 条事件。路由表同样具有固定容量。发送 MESSAGE 时，内核检查 MESSAGE\_SEND 能力位、活动工作流、发送者、目标路由和订阅状态。公开的 agent\_event 只返回发送者、目标 PID、关联号等事件字段；control\_id 和数据来源标签保存在内核附属记录中。

Agent 使用 agent\_watch() 登记关心的事件，以 agent\_wait() 休眠等待，再用 agent\_unwatch() 取消订阅。心跳由 agent\_heartbeat\_set() 和 agent\_heartbeat\_stop() 调整。Loop 的下一步决策仍在 Guest 中完成，内核只负责保存订阅、管理等待状态，并在事件或 TIMER 到达时提供可靠唤醒。

非 TIMEOUT 的 agent\_wait 会先检查已排队事件。选中事件时，内核建立保留项；没有事件时，内核以线程身份代次作为等待键安装等待者并让线程睡眠。事件入队和等待者状态使用相同锁序，确保到达事件与等待代次对应。TIMEOUT 路径在本地直接返回，不建立保留项，也不写入上下文。agent\_wait\_cancel() 只向等待路径注入 AGENT\_EVENT\_CA



NCELLED，它不是工具或任务通道的 CANCEL 操作；消费事件后才更新循环状态和上下文。

线程进入工具执行、等待事件和结束本轮处理时，内核分别维护 RUNNING、WAITING 和 IDLE 状态，并把同一进程中各线程的状态汇总到 PCB。agent\_info() 可读取这一状态，观察程序也会把它与唤醒和调度记录对应起来。任务结束后，Guest 沿普通 exit 路径退出；生命周期在成员、在途操作和资源引用排空后统一回收 Context、订阅与队列对象。

### 3.5.2. 心跳与模型请求关联号

Agent 可以设置心跳间隔。定时器到期时，内核生成 TIMER 事件并唤醒 Agent；Guest 再根据当前状态决定检查文件、发送消息或继续等待。停止心跳会阻止后续 TIMER 产生，但已经进入事件队列的 TIMER 仍按普通事件消费，避免悄然改写队列历史。

LLM\_REQUEST 与 LLM\_RESPONSE 复用消息与等待机制。请求会登记生命周期、请求者和中继程序的 PID、控制标识、关联号和截止时间。同一请求者的关联号严格递增，并且只有成功入队后才更新。中继程序返回匹配响应时，内核原子消费待处理请求并生成 LLM\_DONE。只有同一关联号仍处于活动待处理状态时才返回 DUPLICATE；复用已经完成或过期的关联号则返回 CONFLICT。待处理请求使用 120 秒内核时钟 TTL，完成历史区分已消费请求和超时请求。

### 3.5.3. 工作流资源账户

资源账户维护已用额度  $U$ 、待结算额度  $P$ 、空余额度  $F$  和总额  $U + P + F$ 。当前账户的可用额度先在本地状态中转移：

reserve:F→P      commit:P→U      cancel:P→F      release:U→F

账户补充额度时会检查全局限制、资源类别和账户硬限制。系统压力、上下文切换、账户关闭和 Workflow Fence 都会收缩  $F$ 。Workflow Fence 在同一把锁内执行 trim+snapshot，要求  $P = 0$ ，并导出精确  $U$ 、账户代次、额度周期和向量摘要。

### 3.5.4. 按工作流汇总的 EEVDF

传统线程级公平会让多线程工作流得到更多调度候选位。AgentOS 将同一资源账户中的线程合并为一个工作流实体。调度器使用工作流虚拟运行时间 (vruntime) 计算滞后量，满足  $vruntime \leq global\_vtime$  的实体进入可服务集合，再从中选择虚拟截止时间最早的实体。延迟类别、事件和截止时间可以缩短服务请求。一次分派完成后，内核根据实际服务周期更新工作流虚拟运行时间；睡眠实体的滞后量会向全局虚拟时间衰减。

只有一个工作流实体时，调度器使用快速路径。实体表容量为 4，其中包含一个 BOOT\_SEALED 启动参与者，因此最多可同时纳入三个新建工作流。实体信息不完整或容量超出时，调度器回到原 uCore 路径，并增加回退计数。

### 3.5.5. 唤醒延迟与公平性

唤醒与公平性测量使用 6 次全新 QEMU 启动，覆盖并发度为 1、2、3、4 的服务窗口和精确唤醒探针。504 条探针直接读取调度器跟踪中的 ready\_age，其中 425 条为 0 个时钟节拍 (tick)，79 条为 1 个 tick。uCore 的 tick 为 10 ms；这些记录表明等待者能够在当前采样

粒度内及时得到服务，也说明事件等待没有把 workflow 滞留在睡眠状态。0 tick 只代表未跨过下一个时钟节拍，不能解读为唤醒没有实际开销。

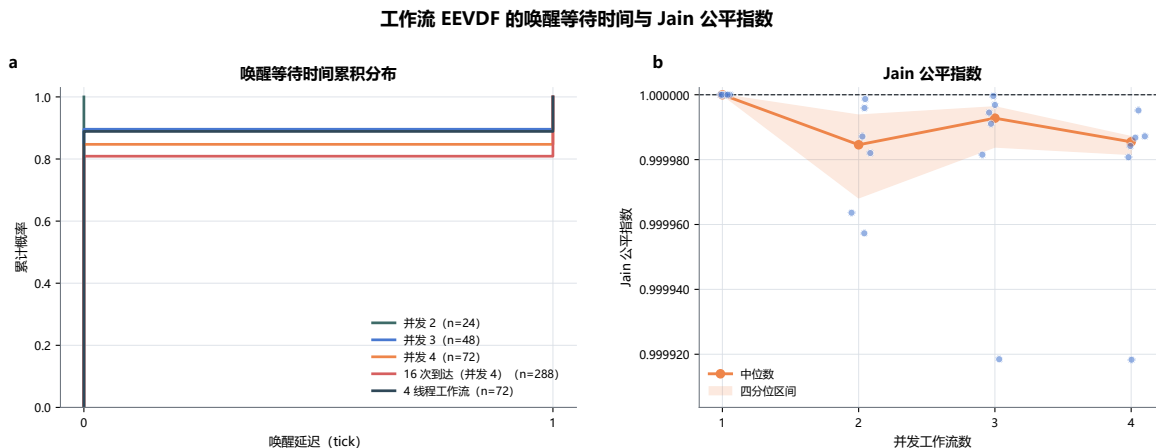


图 3.3: 工作流 EEVDF 的唤醒 tick 与调度公平性

图3.3中，425/504 个探针为 0 tick、79/504 个为 1 tick；并发度为 1、2、3、4 时，Jain 指数中位数分别为 1.000000、0.999985、0.999993 和 0.999985。工作流 EEVDF 将同一工作流内的线程折叠为一个调度实体，因而能抑制线程数放大带来的服务倾斜。该结论受单核、10 ms 的 tick 粒度和最多四个实体分批纳入的条件限制：在独立的四线程放大场景中，Jain 中位数为 0.9980725，该工作流取得 26.895% 的服务份额，高于理想均分的 25%，并非绝对无偏。数据清单还标记了 boot 05 的 3 条 sample marker：串口拼接使它们缺少 wake-bucket 尾字段，但 service 值完整，其中并发度 3 和 4 的两条仍进入 Jain 计算；保守剔除相应测试批次后，两组中位数分别为 0.9999911 和 0.9999842，定性结论不变。精确唤醒分布来自独立的逐探针表，不受这 3 条记录影响。设计上应把当前实现视为对小规模并发工作流的公平保护；补采该组记录、扩大实体表、缩小观测粒度并在多核负载下复核，才是将该策略推广到更一般运行条件的前提。

### 3.5.6. 验证结果

agentloop\_ucore 覆盖原子等待、唤醒、超时、取消、心跳和消息路由；agentsched\_ucore 与 agent\_eevdf\_ucore 覆盖快速路径、实体容量、线程放大、睡眠衰减、回退、唤醒直方图和 Jain 公平性。

## 3.6. AgentOS 控制台综合工作流

### 3.6.1. 科研恢复场景

AgentOS 控制台用六组内核机制执行一个确定性的科研恢复工作流。labdemo\_ucore 创建协调、哨兵、调查和恢复四个角色，建立工作流身份、MESSAGE 路由、上下文、元数据和资源账户。场景登记 prepare、align、analyze、report、archive 五个阶段及其依赖关系，再查询失败节点、读取摘要、向其他智能体发送恢复请求，完成 align 恢复和 report 更新。

固定策略为每个阶段指定相同的业务工作量，工具结果仍由 AgentOS 内核执行和检查。因此，该场景同时覆盖身份、上下文、实时查询、IPC 等待和恢复写入，并能在不同

实现路径之间进行等量比较。

### 3.6.2. 遍历与索引路径的配对实验

综合场景保留遍历实现和索引实现，两者功能等价。每次独立 QEMU 启动按 A/B 或 B/A 顺序运行，两侧产生相同的工作负载和结果指纹。核心窗口覆盖查询、恢复写入、fsync 和复核，查询区段单独记录纯查询时间。

本章性能数字固定对应冻结在源码提交 2b14fb 的测量批次；当前版本后续功能改动另由功能回归验证，未重新计入这组数据。

测量数据来自 16 次独立工作流启动，AB 与 BA 各占一半。16 个核心查询区间中，索引路径的延迟均更低；遍历和索引路径的核心中位数分别为 34.713 ms 与 13.294 ms，配对加速比中位数为 3.118 倍。

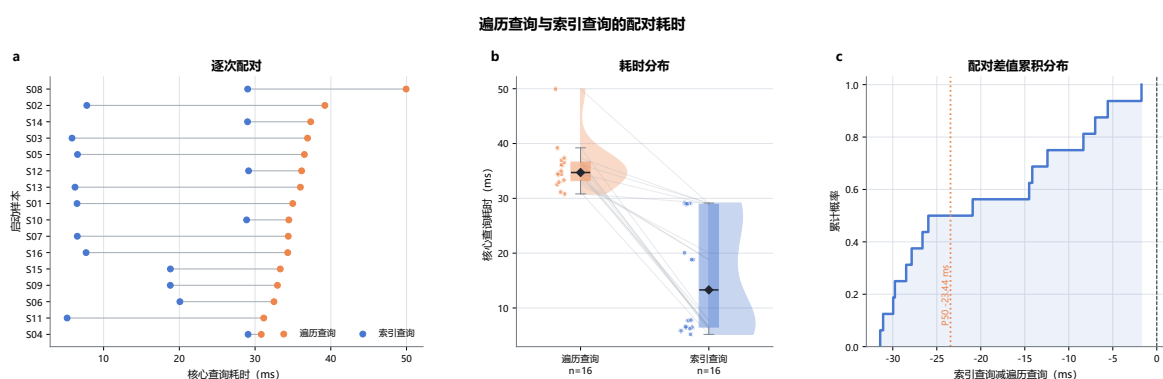


图 3.4: 遍历查询与索引查询的配对耗时

图3.4显示 16/16 个核心查询配对均由 indexed 路径更快完成，中位加速比为 3.118 倍。选择性索引先减少候选记录，再执行与遍历路径相同的完整条件复核，因此收益来自减少无关扫描，而不是省略检查。AB 与 BA 的中位加速比分别为 1.454 倍和 5.481 倍，indexed 路径的四分位区间为 6.5185–28.9128 ms，表明收益幅度仍受启动顺序和运行状态影响，不能外推为固定吞吐提升。该结果支持在恢复流程的核心查询段优先选择 indexed 路径，同时要求完整流程继续分别观察查询以外的开销。

### 3.6.3. 核心查询与完整流程窗口

核心窗口用于观察 AgentOS 查询路径，完整流程窗口在此基础上还包含场景启动、文件与元数据准备、工作流收尾和统计输出。在完整流程窗口中，索引路径只在 16 个配对中的 3 个取得更低延迟，indexed-traversal 的中位数为 +13.452 ms。这说明核心窗口以外的总开销已经抵消索引收益；当前计时没有把这些外围步骤逐段拆开，尚不能确定其中哪一步占主导。

图3.5把核心查询段与完整流程分开后，核心段的 indexed-traversal 中位差为 -23.4415 ms，16/16 个配对更快；端到端中位差却为 +13.452 ms，只有 3/16 个配对更快，整体约慢 1.90%。indexed 路径在核心查询以外的余量在 16/16 个配对中更长，中位多 33.477 ms，说明当前系统虽然已经缩短了索引查询本身，但完整流程的准备、统计和清理没有同步受益，最终抵消了核心路径的改进。这是现阶段系统实现的明确不足。下一步需要把外围步骤分别计时，找出占主导的环节后再优化；长期会话能否摊薄这部分成本尚未测量，不能作为现有结论。

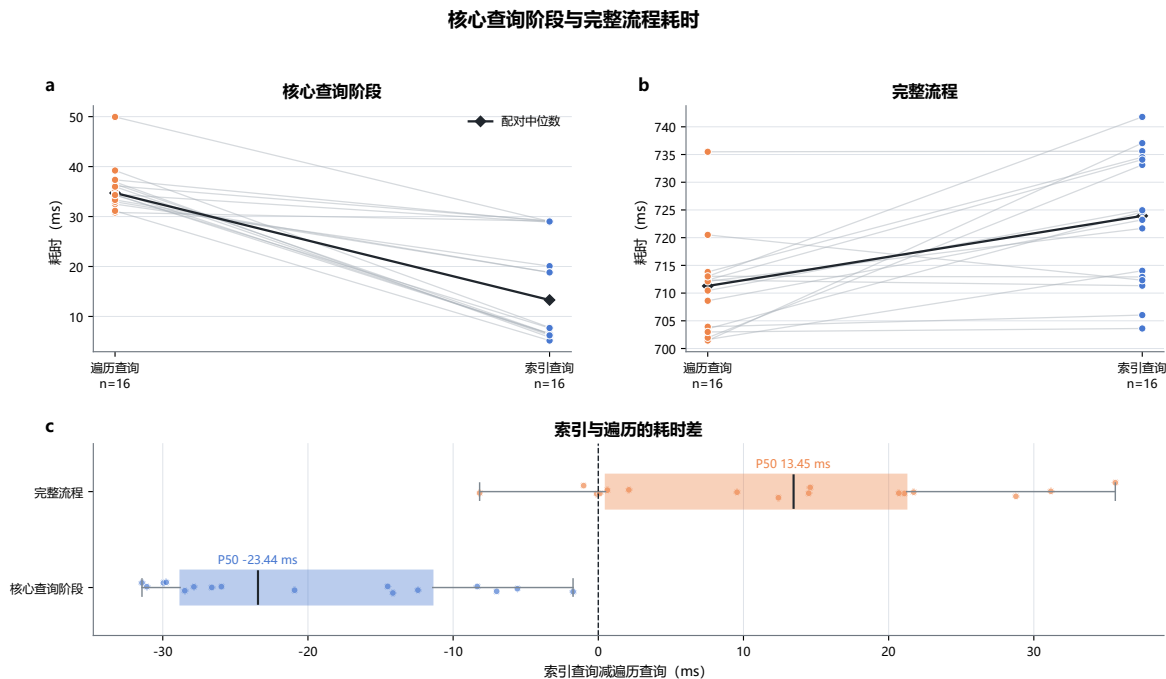


图 3.5: 核心查询阶段与完整流程耗时

### 3.6.4. 系统结果

综合 workflow 依次执行智能体创建、上下文记录、带类型信息的工具调用、实时查询、消息等待、恢复写入和执行记录。输出包含业务指纹、各阶段状态、检查记录数、核心与完整流程时间以及工作量计数。遍历查询和索引查询都在同一 AgentOS-uCore 内核与同一 Guest 场景中运行，并以相同的用户态约定比较两种查询实现。

## 3.7. 综合场景：AgentOS Nexus 多智能体协作平台

### 3.7.1. 长驻会话与四角色协作

AgentOS Nexus 是构建在 AgentOS-uCore 之上的交互 runtime（运行时）。一轮 Nexus 会话在同一 workflow 中常驻协调、系统、研究和分析四个业务 Agent。每个角色都有独立的 PID、Agent 编号、控制标识、角色、能力位和 Context；四个角色共享 workflow 生命周期、VFS 访问范围、资源账户和 EEVDF 实体。这样，Agent 进程、工具调用、上下文路径、文件查询和 Agent Loop 不再是彼此分开的演示点，而是在任务分派、结果整理和报告审批中连续发生。

另有一个 Guest Relay（客户机中继）负责 QEMU 串口帧。Host Relay（宿主机中继）持有本地套接字、传输加密配置、服务接口密钥和模型服务格式适配器；Guest Relay 分别转发模型请求、控制消息和运行指标帧。业务文件和结果文件保存在 Guest VFS，协调智能体根据实际工具返回决定下一步任务。

### 3.7.2. 一次交互回合

图3.6展示了从用户目标到最终结果的一次完整运行过程。

1. 命令行将新目标作为一次交互轮次提交给协调智能体。

| 产品角色  | 内核角色 | 职责                      | 主要权限             |
|-------|------|-------------------------|------------------|
| 协调智能体 | 协调者  | 维护会话、调用模型、创建子任务、核验返回    | 分派任务、受控工具、整理结果文件 |
| 系统智能体 | 哨兵   | 读取当前启动的进程、上下文、文件大小和调度状态 | 读取元数据、进程和调度状态    |
| 研究智能体 | 调查者  | 读取已登记的本地资料和历史测量结果文件     | 读取内容、查询和摘要哈希     |
| 分析智能体 | 报告者  | 合并系统和研究结果并生成报告          | 读取、写入和发布报告结果文件   |

表 3.5: Nexus 的四个业务角色

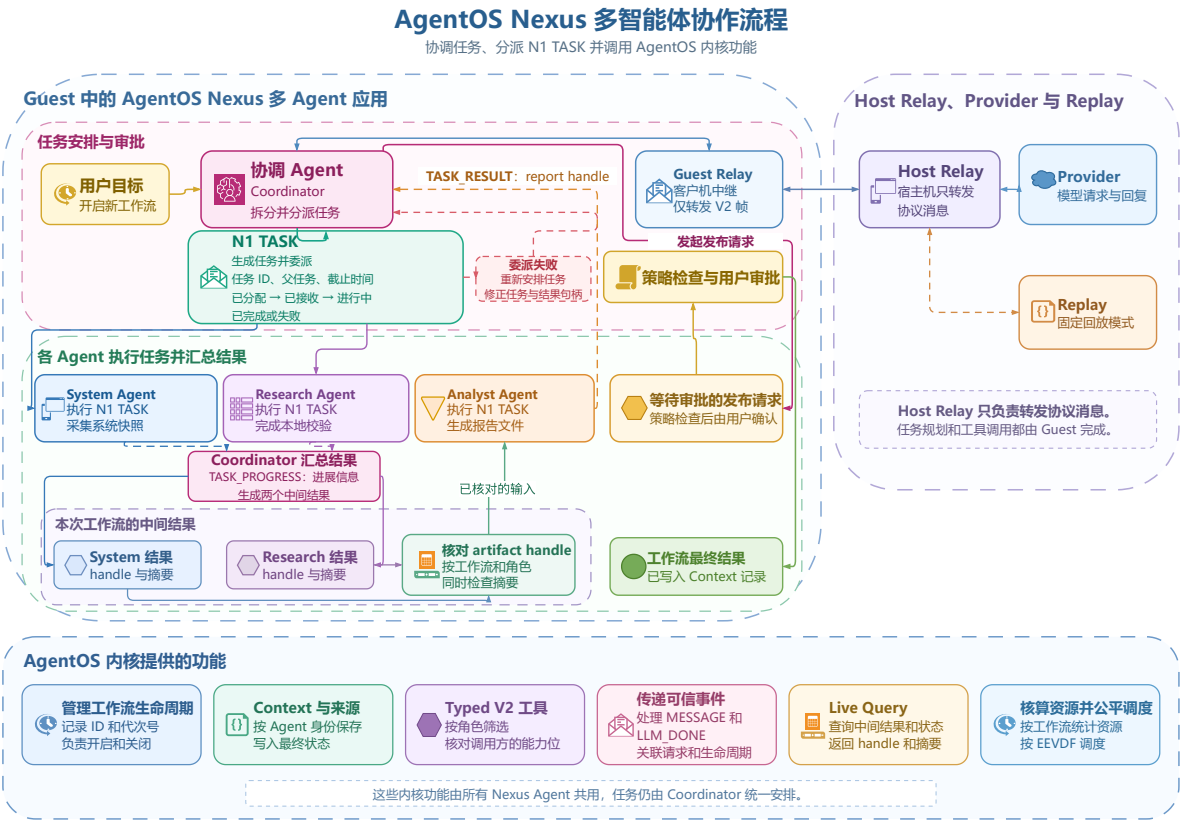


图 3.6: Nexus 多智能体协作流程



2. 协调智能体整理有限历史和按角色筛选的工具集，发出 `LLM_REQUEST` 后进入 `WAITING_LLM`。
3. **Guest Relay** 将请求交给实时模型服务或固定回放提供方（**ReplayProvider**）；响应到达后，**AgentOS** 唤醒协调智能体。
4. 模型需要专门信息时，协调智能体使用 `N1` 创建系统、研究或分析子任务。
5. 子任务智能体以自身身份、上下文和能力位执行带类型信息的 `V2` 工具，等待期间进入 `WAITING_EVENT`。
6. 系统和研究智能体以 `TASK_PROGRESS` 进展信息回传协调智能体；协调智能体把较长结果整理为 workflow 范围内的结果文件。分析智能体自行发布报告结果文件，并以 `TASK_RESULT` 返回报告句柄；协调智能体取得该句柄后提交经过审批的发布请求。
7. 工具结果、结构化失败和审批结果都会返回模型，使下一轮能够继续执行或重新安排任务。

### 3.7.3. N1 子任务协议

`N1` 是 **Guest** 用户态使用的固定 44 字节小端消息格式。它以 `N1:` 加无填充 **Base64URL** 的形式，通过带类型信息的 `V2 send_message` 传输。每条任务包含生命周期标识、代次号、任务标识、父任务标识、发送者、接收者、截止时间、状态和有界值。状态机包含 `TASK_ASSIGN`、`TASK_ACCEPT`、`TASK_PROGRESS`、`TASK_RESULT`、`TASK_FAILED` 和 `TASK_CANCEL`。

**AgentOS** 内核检查 `MESSAGE` 层的发送者、接收者、路由、访问范围、数据来源、队列和唤醒。**Nexus runtime** 负责 `N1` 解码、状态转换校验、截止时间和单个在途任务。较长的目标、工具结果和报告通过结果文件句柄引用，因此控制消息保持紧凑。

### 3.7.4. 结果文件传递

结果文件清单绑定生命周期、句柄代次、类别、任务、父任务、生产者、所有者、整理者、权限掩码、载荷大小、数据来源和来源追溯。**Guest** 用流式 **SHA-256** 覆盖完整载荷，读取者同时校验清单和摘要哈希。

发布时，**Nexus** 把完整 `header` 和 `payload` 交给 `agent_file_publish()`。内核先在工作流资源账户中分配一页 `snapshot`，创建尚未进入目录的 `inode`，写入完整文件并执行 `checkpoint`；数据块和最终 `inode` 形态持久后，才通过一次正式 `dirlink` 把文件名接入目录，并提交第二次 `checkpoint`。因此，工作流中的其他 **Agent** 只能看到“不存在”或“`header` 与 `payload` 均完整”两种状态，不会读到先有文件名、后补内容的中间文件。

如果目录接入已经发出而底层 `I/O` 无法判断是否提交，内核返回 `INDETERMINATE`，不会回收可能已经被目录引用的 `inode`。**Nexus** 随后只读取正式路径，逐字节核对 `header`、`payload` 和 `EOF`；内容完全一致时把本次发布收敛为成功，否则保留失败结果。`DUPLICATE` 也使用同样的精确核对，已有的不同内容不会被覆盖。

系统和研究智能体不在任务完成时直接发布结果文件，而是通过 `TASK_PROGRESS` 进展信息回传协调智能体；协调智能体负责整理对应结果文件，并保留逻辑生产者和整理者。分析智能体读取两份上游结果文件后，以自身身份发布报告类别结果文件，并以 `TASK_RESULT` 返回报告句柄；协调智能体取得句柄后才提交经过审批的发布请求。来源标

签取各输入的并集，因此报告可以同时保留文件、工具和跨智能体来源。

### 3.7.5. 按角色显示工具

Nexus 启动时从 AgentOS 内核读取 25 项工具目录，检查名称、编号和 schema，再按业务角色、内核角色、能力位和副作用类别筛选。tool\_search 按类别分页返回详细说明。

默认读取面包含 pid\_info、ctx\_stat、query\_process、get\_system\_status、read\_context、query\_file、read\_file\_summary、read\_file\_digest、dependency\_query 和 read\_message。delegate\_task、read\_artifact 和 publish\_report 是 Nexus 的用户态功能，协调智能体将它们组织为 N1、结果文件和带类型信息的 V2 操作。

会话保留最近 4 个完整交互轮次，每个轮次最多执行 8 次模型请求，更早内容写入结构化摘要。每次模型请求都携带当前可见的 schema 和上一次稳定工具结果，使工具输出可以直接参与后续决策。

### 3.7.6. 调用级审批与取消

协调智能体取得报告句柄后，publish\_report 的审批对象包含会话、交互轮次、请求、关联号、工具编号、规范参数摘要、nonce、发出时钟和过期时钟。协调智能体所在的 Guest 收到决定后逐字段比较；通过时，再执行精确选择器的带类型信息 V2 artifact\_update。ReplayProvider（固定回放服务）包含一次拒绝发布，报告结果文件仍然可读，会话继续返回最终结果。

Ctrl-C 取消当前交互轮次，常驻智能体和 QEMU 继续运行；/quit 关闭会话。关闭时依次停止观察程序的生产者、排空内核观测事件、等待观察程序退出、关闭遥测写入器；中继程序在 EOF 后输出 SESSION\_CLOSED。

### 3.7.7. 内核观测信息

Nexus 观察程序通过 agent\_timeline\_wait/read、agent\_audit\_query、agent\_info 和上下文快照生成两类视图。kernel\_audit 显示 MESSAGE 入队、消费时的生命周期、身份、发送者、接收者、关联号和状态；kernel\_snapshot 显示四个业务智能体的上下文序号、等待、唤醒间隔、调度分派、预算、虚拟运行时间和能力位掩码。

协调智能体在分配子任务和收到子任务回复时主动读取共享审计游标，观察线程使用同一把互斥锁访问该游标。左侧窗口显示交互与工具结果，右侧窗口同步显示 AgentOS 对应的消息、等待、唤醒和调度状态。

### 3.7.8. 运行验证

自动验收使用 ReplayProvider。每条固定响应都与 Guest 当次产生的规范 MODEL\_REQUEST SHA-256 绑定，并在当前 RV64 镜像的真实 QEMU 启动中执行。基准场景包含 3 个连续交互轮次、4 个业务智能体身份、系统快照、研究句柄过期失败后的重新安排任务、分析智能体合并报告、一次审批拒绝和完整会话关闭。实时模式复用同一个 Guest、N1、结果文件和带类型信息的工具路径，并通过 Host 模型服务适配器对接实时模型服务。

### 3.8. 关键实现

下列代码把设计章节中的机制逐一对应到当前代码树。代码框保留相对路径和物理行号，便于直接核对内核实现。所选片段覆盖启动、身份、调用、执行约定、上下文、文件事件、任务通道、结果文件发布、资源和调度，说明一个智能体工作流如何创建、执行并结束。

#### 3.8.1. 启动、身份与工作流代次

内核入口先初始化进程、控制台和页表等基本设施，再调用 `agentinit()`。文件系统就绪后，系统启用智能体存储和文件代次机制。随后加载的初始用户程序直接使用已建立的生命周期、元数据和资源机制，不需要在用户态临时拼接旁路状态。

代码 1: uCore 启动序列中的 AgentOS 初始化

c

```

19 void main()
20 {
21     clean_bss();
22     printf("hello world!\n");
23     proc_init();
24     console_init();
25     agentinit();
26     kinit();
27     kvm_init();
28     if (kalloc_physical_policy_init(PHYSICAL_PAGE_SYSTEM_RESERVE,
29                                     PHYSICAL_PAGE_ORDINARY_LIMIT) < 0)
30         panic("physical page policy");
31     trap_init();
32     plicinit();
33     virtio_disk_init();
34     binit();
35     fsinit();
36     timer_init();
37     agent_storage_init();
38     fs_epoch_runtime_enable();
39     load_init_app();
40     infof("start scheduler!");
41     /* 仅启动期使用的映像加载完成后，才开放运行时输入输出准入。 */
42     bio_policy_start();
43     virtio_disk_runtime_start();
44     scheduler();

```

生命周期表用 `id+generation` 共同定位工作流；槽位复用时，代次单调增加。创建工作流时，内核先准备记录，再一次写入创建者、访问范围、成员数和代次。因此，旧代次的订阅、任务或资源句柄会在下次检查时失效，不会误指向新工作流。

代码 2: 工作流生命周期的创建与代次绑定

c

```

56 static struct workflow_lifecycle_record *
57 workflow_lifecycle_find_locked(struct workflow_lifecycle_key key)
58 {
59     uint slot;
60     struct workflow_lifecycle_record *record;

```

```

61
62     if (!workflow_lifecycle_key_valid(key) ||
63         key.id > WORKFLOW_LIFECYCLE_CAP)
64         return 0;
65     slot = key.id - 1;
66     record = &workflow_lifecycles[slot];
67     if (!record->used || record->generation != key.generation)
68         return 0;
69     return record;
70 }
71
72 static struct workflow_lifecycle_key
73 workflow_lifecycle_record_key(uint slot,
74                             const struct workflow_lifecycle_record *record)
75 {
76     struct workflow_lifecycle_key key = {
77         .id = slot + 1,
78         .generation = record->generation,
79     };
80
81     return key;
82 }
83
84 int workflow_lifecycle_create(uint scope_id,
85                             struct workflow_lifecycle_key *key)
86 {
87     int enabled;
88     int result = -1;
89     uint allocated_slot = WORKFLOW_LIFECYCLE_CAP;
90     uint64 next_generation = 0;
91
92     if (key == 0 || scope_id < VFS_SCOPE_FIRST_DYNAMIC ||
93         scope_id >= FS_OWNER_SCOPE_FLAG)
94         return -1;
95     *key = workflow_lifecycle_none();
96     enabled = intr_save();
97     for (uint i = 0; i < WORKFLOW_LIFECYCLE_CAP; i++)
98         if (workflow_lifecycles[i].used &&
99             workflow_lifecycles[i].scope_id == scope_id)
100             goto out;
101     for (uint i = 0; i < WORKFLOW_LIFECYCLE_CAP; i++) {
102         struct workflow_lifecycle_record *record =
103             &workflow_lifecycles[i];
104         uint64 generation;
105
106         if (record->used ||
107             workflow_lifecycle_generations[i] == ~0ULL)
108             continue;
109         generation = workflow_lifecycle_generations[i] + 1;
110         if (generation == 0 ||
111             !agent_identity_lease_lifecycle_contains(i, generation))
112             continue;
113         workflow_lifecycle_generations[i] = generation;
114         record->used = 1;
115         record->scope_id = scope_id;
116         record->generation = generation;
117         record->controller_control_id = 0;

```

```

118     record->next_context_branch = 0;
119     record->fence_sequence = 0;
120     record->members = 1;
121     record->active_operations = 0;
122     record->departing_operations = 0;
123     record->fence_gate = 0;
124     record->closing = 0;
125     *key = workflow_lifecycle_record_key(i, record);
126     allocated_slot = i;
127     next_generation = generation == ~0ULL ? 0 : generation + 1;
128     result = 0;
129     break;
130 }
131 out:
132     intr_restore(enabled);
133     if (allocated_slot < WORKFLOW_LIFECYCLE_CAP)
134         agent_identity_lease_lifecycle_note_next(
135             allocated_slot, next_generation);
136     return result;

```

角色规则不是用户态的字符串约定。内核按角色取得能力位掩码和调度权重，再在授权入口检查父角色、目标角色和能力位的合法关系。因此，工具、VFS、IPC 和调度器都依据同一份身份信息作出判断。

### 代码 3：角色策略表查询与调度权重取得

c

```

128 const struct agent_role_policy *
129 agent_identity_role_policy(int role)
130 {
131     for (uint i = 0; i < NELEM(role_policies); i++)
132         if (role_policies[i].role == role)
133             return &role_policies[i];
134     return 0;
135 }
136
137 int
138 agent_identity_role_valid(int role)
139 {
140     return agent_identity_role_policy(role) != 0;
141 }
142
143 int
144 agent_identity_role_sched_weight(int role)
145 {
146     const struct agent_role_policy *policy =
147         agent_identity_role_policy(role);
148
149     return policy ? policy->sched_weight : 50;
150 }

```

### 代码 4：父角色授权与进程访问范围校验

c

```

190 int
191 agent_identity_authority_check(struct proc *p, int role)

```



```

192 {
193     if (!agent_identity_role_valid(role))
194         return AGENT_STATUS_BAD_PARAM;
195     if (p == 0 ||
196         (p->is_agent ?
197          (!exec_policy_process_allows_role(p, p->agent_role) ||
198           !exec_policy_process_allows_role(p, role)) :
199          (!exec_policy_process_bootstrap(p) ||
200           !exec_policy_process_allows_role(p, role))) ||
201         (p->agent_role_grant_mask & AGENT_ROLE_GRANT_BIT(role)) == 0)
202         return AGENT_STATUS_DENIED;
203     return AGENT_STATUS_OK;
204 }
205
206 uint
207 agent_identity_proc_scope(struct proc *p)
208 {
209     if (p == 0 || !p->is_agent ||
210         p->agent_control_state != AGENT_CONTROL_OPEN ||
211         p->vfs_scope_id < VFS_SCOPE_FIRST_DYNAMIC ||
212         p->vfs_scope_id >= FS_OWNER_SCOPE_FLAG ||
213         !vfs_proc_lifecycle_active(p) ||
214         !vfs_scope_active(p->vfs_scope_id))
215         return VFS_SCOPE_NONE;
216     return p->vfs_scope_id;

```

### 算法 2：带代次的工作流进入检查

1. 内核从当前进程控制块读取 workflow 编号和代次，并验证 id 与 generation 均非零。
2. 内核在生命周期表中同时查找这两个字段；仅槽位相同不能通过校验。
3. 内核进入操作保护区后才执行系统副作用；屏障或回收已关闭该路径时返回重试或过期错误。
4. 异步对象在完成、取消或回收时再次比较代次，再释放生命周期引用。

### 3.8.2. 结构化工具调用与执行约定

V3 调用先检查用户内存范围、ABI 版本和大小、保留字段、参数个数和生命周期绑定。请求中的执行约定、节点、尝试号、schema 摘要和输入指纹会复制到内核记录。因此，一次工具调用不再是孤立的函数调用，而是执行约定 DAG 中可核验的节点执行。

#### 代码 5：V3 请求的 ABI、生命周期和执行约定绑定检查

c

```

2590     if (copyin(p->pagetable, (char *)&req, reqaddr, sizeof(req)) < 0 ||
2591         user_range_check(p->pagetable, respaddr, sizeof(resp), PTE_W) < 0)
2592         return -1;
2593     memset(&resp, 0, sizeof(resp));
2594     resp.version = AGENT_CALL_VERSION_V3;
2595     resp.size = sizeof(resp);
2596     resp.request_id = req.request_id;
2597     lifecycle = vfs_proc_lifecycle(p);
2598     resp.contract.lifecycle.id = lifecycle.id;
2599     resp.contract.lifecycle.reserved = 0;

```

```

2600     resp.contract.lifecycle.generation = lifecycle.generation;
2601     resp.contract.generation =
2602         agent_execution_contract_generation(lifecycle);
2603     resp.node_id = req.node_id;
2604     resp.attempt_id = req.attempt_id;
2605     memmove(resp.input_fingerprint, req.input_fingerprint,
2606             sizeof(resp.input_fingerprint));
2607     resp.source_context_sequence = req.source_context_sequence;
2608     memset(&binding, 0, sizeof(binding));
2609     binding.lifecycle.id = req.contract.lifecycle.id;
2610     binding.lifecycle.generation = req.contract.lifecycle.generation;
2611     binding.contract_generation = req.contract.generation;
2612     binding.node_id = req.node_id;
2613     binding.attempt_id = req.attempt_id;
2614     memmove(binding.input_fingerprint, req.input_fingerprint,
2615             sizeof(binding.input_fingerprint));
2616     binding.source_context_sequence = req.source_context_sequence;
2617     binding.source_control_id = req.source_control_id;
2618     binding.source_pid = req.source_pid;
2619     binding.source_reserved = req.source_reserved;
2620     memmove(binding.schema_digest, req.schema_digest,
2621             sizeof(binding.schema_digest));
2622     binding.input_artifact_type = req.input_artifact_type;
2623     binding.source_node_id = req.source_node_id;
2624     binding.input_mode = AGENT_EXECUTION_INPUT_INLINE;
2625     memset(&op, 0, sizeof(op));
2626     op.version = AGENT_CALL_VERSION_V3;
2627     op.tool_id = req.tool_id > 0 ? req.tool_id :
2628         (0x20000000 | (int)((req.node_id + 1) & 0xffffU));
2629     op.request_id = req.request_id;
2630     op.arg0 = req.contract.generation;
2631     op.arg1 = ((uint64)req.node_id << 32) | req.attempt_id;
2632     safestrncpy(op.payload, "v3_rejected", sizeof(op.payload));
2633     enforced = p->is_agent &&
2634         agent_execution_contract_enforced(lifecycle);
2635     memset(error, 0, sizeof(error));
2636     if (req.version != AGENT_CALL_VERSION_V3) {
2637         status = AGENT_STATUS_BAD_VERSION;
2638         safestrncpy(error, "bad_request_version", sizeof(error));
2639     } else if (req.size != sizeof(req)) {
2640         status = AGENT_STATUS_BAD_SIZE;
2641         safestrncpy(error, "bad_request_size", sizeof(error));
2642     } else if (req.flags != 0 || req.source_reserved != 0) {
2643         status = AGENT_STATUS_BAD_PARAM;
2644         safestrncpy(error, "unsupported_fields", sizeof(error));
2645     } else if (req.param_count > AGENT_TOOL_PARAM_MAX) {
2646         status = AGENT_STATUS_BAD_SIZE;
2647         safestrncpy(error, "too_many_params", sizeof(error));
2648     } else if (req.contract.lifecycle.id == 0 ||
2649         req.contract.lifecycle.generation == 0 ||
2650         req.contract.generation == 0) {
2651         status = AGENT_STATUS_BAD_PARAM;
2652         safestrncpy(error, "bad_contract_binding", sizeof(error));
2653     } else if (!p->is_agent) {
2654         status = AGENT_STATUS_NOT_AGENT;
2655         safestrncpy(error, "not_agent", sizeof(error));
2656     } else if (req.contract.lifecycle.reserved != 0 ||

```

```

2657     req.contract.lifecycle.id != lifecycle.id ||
2658     req.contract.lifecycle.generation != lifecycle.generation) {
2659     status = AGENT_STATUS_STALE;
2660     denial_reason = AGENT_EXECUTION_REASON_STALE_LIFECYCLE;
2661     safestrncpy(error, "stale_lifecycle", sizeof(error));
2662 } else {
2663     status = agent_tool_protocol_resolve(req.tool_id, req.tool_name,
2664                                         &tool, error, sizeof(error));
2665     if (status != AGENT_STATUS_OK)
2666         denial_reason = AGENT_EXECUTION_REASON_TOOL_MISMATCH;
2667 }

```

解析阶段先按工具名称或编号查找定义，再逐项从用户内存读取带类型信息的参数。每个参数都必须符合目录规则规定的名称、类型、长度和唯一性，最后还要确认必填参数全部出现。用户态即使构造出形式正确的结构，也不能绕过内核目录中的 schema。

## 代码 6: V2 参数 schema 解码器

c

```

548 int agent_tool_protocol_decode_v2(pagetable_t pagetable,
549                                  struct agent_request_v2 *request,
550                                  struct agent_tool_match *match,
551                                  struct agent_op *op, char *error,
552                                  int error_size)
553 {
554     uint seen = 0, count = rule_count(match->tool_id);
555     const struct param_rule *tool_rules = rules_for_tool(match->tool_id);
556
557     memset(op, 0, sizeof(*op));
558     op->version = AGENT_CALL_VERSION_V1;
559     op->tool_id = match->tool_id;
560     op->request_id = request->request_id;
561     for (uint i = 0; i < request->param_count; i++) {
562         struct agent_param_v2 param;
563         uint text_length = 0;
564         int index = -1;
565
566         if (copyin(pagetable, (char *)&param,
567                  request->params + (uint64)i * sizeof(param),
568                  sizeof(param)) < 0)
569             return -1;
570         if (param.version != AGENT_PARAM_VERSION)
571             REJECT(AGENT_STATUS_BAD_VERSION, "bad_param_version");
572         if (param.size != sizeof(param) ||
573             string_length(param.key, AGENT_PARAM_KEY_SIZE, 0) < 0)
574             REJECT(AGENT_STATUS_BAD_SIZE, "bad_param_size");
575         for (uint j = 0; j < count; j++)
576             if (!strncmp(param.key, rule_key(&tool_rules[j]),
577                        AGENT_PARAM_KEY_SIZE)) {
578                 index = j;
579                 break;
580             }
581         if (index < 0)
582             REJECT(AGENT_STATUS_UNKNOWN_PARAM, "unknown_param");
583         if (seen & (1U << index))
584             REJECT(AGENT_STATUS_DUPLICATE, "duplicate_param");

```

```

585     seen |= 1U << index;
586     if (param.type != rule_type(&tool_rules[index]))
587         REJECT(AGENT_STATUS_BAD_TYPE, "bad_param_type");
588     if (param.type == AGENT_PARAM_UINT64) {
589         if (param.value_size != sizeof(param.value.uint64_value))
590             REJECT(AGENT_STATUS_BAD_SIZE, "bad_value_size");
591     } else if (param.value_size == 0 ||
592                param.value_size > AGENT_PARAM_STRING_SIZE ||
593                string_length(param.value.string_value, param.value_size,
594                              &text_length) < 0 ||
595                text_length + 1 != param.value_size)
596         REJECT(AGENT_STATUS_BAD_SIZE, "bad_value_size");
597     switch (rule_target(&tool_rules[index])) {
598     case PARAM_ARG0: op->arg0 = param.value.uint64_value; break;
599     case PARAM_ARG1: op->arg1 = param.value.uint64_value; break;
600     default: safestrcpy(op->payload, param.value.string_value,
601                        sizeof(op->payload)); break;
602     }
603 }
604 for (uint i = 0; i < count; i++)
605     if (rule_required(&tool_rules[i]) && !(seen & (1U << i)))
606         REJECT(AGENT_STATUS_BAD_PARAM, "missing_param");
607 return AGENT_STATUS_OK;

```

参数校验通过后，调用先进入生命周期操作保护区，再调用 `agent_execute_one()`。该入口汇总调用结果、输出 `artifact` 类型、拒绝原因、来源标签和执行记录票号；受规则约束的拒绝会尝试写入安全记录。关键记录无法预留或提交时，直接调用和 V3 调用的拒绝会返回 `NO_SPACE` 或 `RETRY`，因此不是每次拒绝都会得到执行记录票号。

### 代码 7：V3 调用的解码、检查与执行进入

c

```

2678     if ((req.param_count == 0 && req.params != 0) ||
2679         (req.param_count != 0 && req.params == 0)) {
2680         status = AGENT_STATUS_BAD_PARAM;
2681         safestrcpy(error, "bad_param_pointer", sizeof(error));
2682         goto rejected;
2683     }
2684     if (req.param_count != 0 &&
2685         user_range_check(p->pagetable, req.params,
2686                          (uint64)req.param_count *
2687                          sizeof(struct agent_param_v2), PTE_R) < 0) {
2688         status = AGENT_STATUS_DENIED;
2689         safestrcpy(error, "bad_param_range", sizeof(error));
2690         goto rejected;
2691     }
2692     memset(&decode_request, 0, sizeof(decode_request));
2693     decode_request.version = AGENT_CALL_VERSION_V2;
2694     decode_request.size = sizeof(decode_request);
2695     decode_request.tool_id = req.tool_id;
2696     decode_request.param_count = req.param_count;
2697     decode_request.request_id = req.request_id;
2698     decode_request.params = req.params;
2699     safestrcpy(decode_request.tool_name, req.tool_name,
2700               sizeof(decode_request.tool_name));
2701     status = agent_tool_protocol_decode_v2(

```

```

2702     p->pagetable, &decode_request, &tool, &op, error, sizeof(error));
2703     if (status != AGENT_STATUS_OK) {
2704         status = AGENT_STATUS_DENIED;
2705         denial_reason = AGENT_EXECUTION_REASON_CONTRACT_INVALID;
2706         goto rejected;
2707     }
2708     if (workflow_lifecycle_operation_enter(lifecycle) < 0) {
2709         status = AGENT_STATUS_RETRY;
2710         safestrcpy(error, "workflow_fenced", sizeof(error));
2711         goto out;
2712     }
2713     operation_entered = 1;
2714     if (agent_execute_one(p, &op, &result, agent_ticks(), &binding, 0, 0,
2715         &outcome) < 0)
2716         result.status = AGENT_STATUS_NO_SPACE;
2717     status = result.status;
2718     resp.sequence = result.sequence;
2719     resp.value0 = result.value0;
2720     resp.value1 = result.value1;
2721     resp.value2 = result.value2;
2722     resp.output_artifact_type = outcome.output_artifact_type;
2723     resp.decision_reason = outcome.decision_reason;
2724     resp.completion_flags = outcome.completion_flags;
2725     resp.output_provenance_labels = outcome.output_provenance_labels;
2726     resp.evidence_ticket = outcome.evidence_ticket;
2727     safestrcpy(resp.result, result.result, sizeof(resp.result));
2728     goto out;

```

## 代码 8：受控拒绝的安全记录与返回封装

c

```

2730 rejected:
2731     if (enforced && op.tool_id > 0) {
2732         if (op.request_id == 0) {
2733             /* Keep user correlation zero while giving Evidence a valid key. */
2734             op.request_id = (1ULL << 63) |
2735                 ((uint64)(uint)p->pid << 32) |
2736                 (uint)(p->agent_call_count + 1);
2737         }
2738         status = agent_tool_call_v3_security_denial(
2739             p, &binding, &op, status, denial_reason,
2740             &result, &outcome);
2741         resp.sequence = result.sequence;
2742         resp.output_artifact_type = outcome.output_artifact_type;
2743         resp.decision_reason = outcome.decision_reason;
2744         resp.output_provenance_labels =
2745             outcome.output_provenance_labels;
2746         resp.evidence_ticket = outcome.evidence_ticket;
2747         safestrcpy(resp.result, result.result, sizeof(resp.result));
2748         error[0] = 0;
2749     }
2750 out:
2751     if (operation_entered)
2752         workflow_lifecycle_operation_leave(lifecycle);
2753     if (enforced &&
2754         (status == AGENT_STATUS_DENIED || status == AGENT_STATUS_STALE) &&

```



```

2755     resp.evidence_ticket == 0) {
2756         status = AGENT_STATUS_RETRY;
2757         error[0] = 0;
2758         safestrcpy(resp.result, "security_evidence_unavailable",
2759                 sizeof(resp.result));
2760     }
2761     resp.status = status;
2762     if (status != AGENT_STATUS_OK && error[0] != 0)
2763         safestrcpy(resp.result, error, sizeof(resp.result));
2764     return copyout(p->pagetable, respaddr, (char *)&resp, sizeof(resp));

```

执行约定的接纳检查会逐项比对绑定的生命周期、执行约定代次、节点编号、截止时间、工具编号和资源额度。任务通道的内部请求还限制为“不产生输出”的预检形式；被拒绝的任务不会因此发布伪造的带类型信息输出。

### 代码 9：执行约定节点的代次、资源与工具一致性检查

c

```

1157     if (!workflow_lifecycle_key_equal(binding->lifecycle, current)) {
1158         agent_execution_result_error(
1159             result, AGENT_STATUS_STALE, "stale_lifecycle",
1160             AGENT_EXECUTION_REASON_STALE_LIFECYCLE, claim);
1161         goto denied_counted;
1162     }
1163     if (!agent_execution_record_enforced(record, current)) {
1164         agent_execution_result_error(
1165             result, AGENT_STATUS_DENIED, "execution_contract_missing",
1166             AGENT_EXECUTION_REASON_CONTRACT_REQUIRED, claim);
1167         goto denied_counted;
1168     }
1169     if (record->state == AGENT_EXECUTION_CONTRACT_RETIRING) {
1170         agent_execution_result_error(
1171             result,
1172             (binding->internal_flags &
1173              AGENT_EXECUTION_BINDING_INTERNAL_F_TASK_CHANNEL) != 0 ?
1174             AGENT_STATUS_DENIED : AGENT_STATUS_CANCELLED,
1175             "contract_retiring",
1176             AGENT_EXECUTION_REASON_CONTRACT_RETIRING, claim);
1177         goto denied_counted;
1178     }
1179     if (binding->contract_generation == 0 ||
1180         binding->contract_generation != record->generation) {
1181         agent_execution_result_error(
1182             result, AGENT_STATUS_STALE, "stale_contract",
1183             AGENT_EXECUTION_REASON_STALE_CONTRACT, claim);
1184         goto denied_counted;
1185     }
1186     if (binding->node_id >= record->node_count) {
1187         agent_execution_result_error(
1188             result, AGENT_STATUS_DENIED, "unknown_contract_node",
1189             AGENT_EXECUTION_REASON_UNKNOWN_NODE, claim);
1190         goto denied_counted;
1191     }
1192     node = &record->nodes[binding->node_id];
1193     runtime = &record->runtime[binding->node_id];
1194     claim->output_artifact_type = node->output_artifact_type;

```

```

1195     claim->charge_class = node->charge_class;
1196     contract_deadline = node->deadline_tick != 0 &&
1197         (record->deadline_tick == 0 ||
1198         node->deadline_tick < record->deadline_tick) ?
1199         node->deadline_tick : record->deadline_tick;
1200     claim->deadline_tick = request_deadline_tick != 0 ?
1201         request_deadline_tick : contract_deadline;
1202     claim->manifest.accepted_input_labels = node->accepted_input_labels;
1203     claim->manifest.output_add_labels = node->output_add_labels;
1204     claim->manifest.required_capabilities = node->required_capabilities;
1205     claim->manifest.side_effect_mask = node->side_effect_mask;
1206     for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++) {
1207         claim->exec_amounts[kind] = node->exec_envelope[kind];
1208         claim->storage_amounts[kind] = node->storage_envelope[kind];
1209     }
1210     if ((binding->internal_flags &
1211         AGENT_EXECUTION_BINDING_INTERNAL_F_TASK_CHANNEL) != 0) {
1212         uint expected_policy =
1213             AGENT_EXECUTION_PREFLIGHT_F_OUTPUT_NONE_ONLY |
1214             (request_deadline_tick != 0 ?
1215             AGENT_EXECUTION_PREFLIGHT_F_HARD_DEADLINE : 0);
1216
1217         if (admission_policy_flags != expected_policy ||
1218             (contract_deadline != 0 &&
1219             (admission_policy_flags &
1220             AGENT_EXECUTION_PREFLIGHT_F_HARD_DEADLINE) == 0) ||
1221             node->output_artifact_type != AGENT_ARTIFACT_NONE) {
1222             /* A rejected Task request never publishes a typed output. */
1223             claim->output_artifact_type = AGENT_ARTIFACT_NONE;
1224             agent_execution_result_error(
1225                 result, AGENT_STATUS_DENIED,
1226                 "task_admission_policy",
1227                 AGENT_EXECUTION_REASON_CONTRACT_INVALID, claim);
1228             goto denied_counted;
1229         }
1230     } else if (admission_policy_flags != 0) {
1231         agent_execution_result_error(
1232             result, AGENT_STATUS_DENIED,
1233             "unexpected_admission_policy",
1234             AGENT_EXECUTION_REASON_CONTRACT_INVALID, claim);
1235         goto denied_counted;
1236     }
1237     if ((runtime->flags &
1238         AGENT_EXECUTION_RUNTIME_F_DEPENDENCY_TERMINAL) != 0) {
1239         claim->dependency_failed = 1;
1240         claim->retry_forbidden = 1;
1241         claim->decision_reason =
1242             AGENT_EXECUTION_REASON_DEPENDENCY_FAILED;
1243     }
1244     if (node->tool_id != (uint)op->tool_id) {
1245         agent_execution_result_error(
1246             result, AGENT_STATUS_DENIED, "contract_tool_mismatch",
1247             AGENT_EXECUTION_REASON_TOOL_MISMATCH, claim);
1248         goto denied_counted;
1249     }
1250     if (!agent_execution_digest_equal(binding->schema_digest,

```

真正触发副作用之前，内核再次确认节点仍在运行，请求号、尝试号、摘要与当前声明一致，并处理超时或取消的胜出分支。这个检查缩短了“执行约定通过”和“副作用实际发生”之间的时间差，避免状态过期后继续产生内核可见副作用。

#### 代码 10：副作用开始前的执行约定检查

c

```

1781 enum agent_execution_effect_admission
1782 agent_execution_contract_effect_begin(struct agent_execution_claim *claim)
1783 {
1784     struct agent_execution_contract_record *record;
1785     struct agent_execution_node_runtime *runtime;
1786     int enabled;
1787
1788     if (claim == 0 || !claim->active)
1789         return AGENT_EXECUTION_EFFECT_STALE;
1790     if (claim->legacy)
1791         return AGENT_EXECUTION_EFFECT_ALLOWED;
1792     if (claim->slot < 0 || claim->slot >= WORKFLOW_LIFECYCLE_CAP ||
1793         claim->node_id >= AGENT_EXECUTION_CONTRACT_MAX_NODES)
1794         return AGENT_EXECUTION_EFFECT_STALE;
1795     enabled = intr_save();
1796     record = &agent_execution_contracts[claim->slot];
1797     if (!agent_execution_record_matches(record, claim->lifecycle) ||
1798         record->generation != claim->contract_generation ||
1799         claim->node_id >= record->node_count) {
1800         intr_restore(enabled);
1801         return AGENT_EXECUTION_EFFECT_STALE;
1802     }
1803     runtime = &record->runtime[claim->node_id];
1804     if (runtime->state != AGENT_EXECUTION_NODE_RUNNING ||
1805         runtime->request_id != claim->request_id ||
1806         runtime->attempt_id != claim->attempt_id ||
1807         !agent_execution_digest_equal(runtime->request_digest,
1808             claim->request_digest)) {
1809         intr_restore(enabled);
1810         return AGENT_EXECUTION_EFFECT_STALE;
1811     }
1812     if ((runtime->flags &
1813         AGENT_EXECUTION_RUNTIME_F_TIMEOUT_REQUESTED) != 0) {
1814         claim->decision_reason =
1815             AGENT_EXECUTION_REASON_DEADLINE_EXPIRED;
1816         intr_restore(enabled);
1817         return AGENT_EXECUTION_EFFECT_STALE;
1818     }
1819     if ((runtime->flags &
1820         AGENT_EXECUTION_RUNTIME_F_CANCEL_REQUESTED) != 0) {
1821         claim->decision_reason =
1822             AGENT_EXECUTION_REASON_CANCEL_REQUESTED;
1823         if ((runtime->flags &
1824             AGENT_EXECUTION_RUNTIME_F_FORCE_CANCEL) != 0)
1825             claim->retry_forbidden = 1;
1826         intr_restore(enabled);
1827         return AGENT_EXECUTION_EFFECT_CANCELLED;
1828     }
1829     runtime->flags |= AGENT_EXECUTION_RUNTIME_F_EFFECT_STARTED;
1830     intr_restore(enabled);

```

```
1831     return AGENT_EXECUTION_EFFECT_ALLOWED;
```

### 算法 3：一次 V3 工具调用的内核决策顺序

1. 内核验证固定大小 ABI、用户地址、智能体身份和工作流代次。
2. 内核查找工具描述符并按 schema 解码；失败时构造结构化拒绝结果，关键记录无法建立时返回 NO\_SPACE 或 RETRY。
3. 接纳检查核对执行约定 DAG、输入来源、能力位、截止时间和资源额度。
4. 开始副作用前，内核再次确认当前声明；执行后写入上下文、执行记录、来源信息和完成结果。

### 3.8.3. 上下文、回滚边界与数据来源

上下文附属记录只保存归一化的 agent\_op、结果和归因，不完整保留 V2 参数、V3 绑定或任务 SQE。镜像页按 prepare--begin--end 顺序发布；detail 在记录标为奇数前写入，序号只在一次清空后的时期内单调增加，清空后从 1 重新开始。因此，Guest 看到的只读页是稳定快照，而不是写到一半的记录。

#### 代码 11：上下文附属记录的写入与发布窗口

c

```
1152 static int
1153 agent_context_append_flags(struct proc *p, struct agent_op *op,
1154                          struct agent_result *latest, uint64 tick,
1155                          uint64 flags, int authority_effect,
1156                          int causal_audit,
1157                          struct agent_evidence_context_reservation *reservation,
1158                          struct agent_evidence_security_reservation *
1159                          security_reservation,
1160                          uint64 *evidence_ticket_out)
1161 {
1162     struct agent_context_append_receipt receipt;
1163     struct agent_context_detail detail;
1164     struct agent_context_record record;
1165     struct thread *t = curr_thread();
1166     uint64 slot;
1167
1168     if (p == 0 || op == 0 || latest == 0 || p->agent_ctx_base == 0 ||
1169         p->context_path_capacity == 0 || !agent_context_layout_ok() ||
1170         p->agent_current_span_id == 0 ||
1171         p->agent_current_span_owner == 0 || t == 0 || t->process != p ||
1172         p->agent_context_lane_owner_tid != t->tid ||
1173         p->agent_context_lane_depth == 0 ||
1174         latest->sequence != p->agent_call_count)
1175         return -1;
1176     if ((reservation != 0 && security_reservation != 0) ||
1177         ((reservation != 0 || security_reservation != 0) &&
1178          (evidence_ticket_out == 0 ||
1179           agent_observe_recording_suppressed(p))) ||
1180         (reservation != 0 && !reservation->active) ||
1181         (security_reservation != 0 && !security_reservation->active))
1182         return -1;
```

```

1183     slot = p->context_path_head % p->context_path_capacity;
1184
1185     memset(&record, 0, sizeof(record));
1186     record.sequence = latest->sequence;
1187     record.request_id = latest->request_id;
1188     record.cause_sequence = p->agent_current_cause_sequence;
1189     record.span_id = p->agent_current_span_id;
1190     record.branch_generation = p->context_branch_generation;
1191     record.path_parent_sequence = p->context_path_visible_head;
1192     record.arg0 = op->arg0;
1193     record.value0 = latest->value0;
1194     record.value1 = latest->value1;
1195     record.value2 = latest->value2;
1196     record.tick = tick;
1197     record.flags = agent_provenance_context_flags(
1198         p, latest->request_id, latest->tool_id, flags);
1199     if (strlen(op->payload) >= sizeof(record.payload) ||
1200         strlen(latest->result) >= sizeof(record.result))
1201         record.flags |= AGENT_CONTEXT_RECORD_F_TRUNCATED;
1202     record.tool_id = latest->tool_id;
1203     record.status = latest->status;
1204     safestrncpy(record.payload, op->payload, sizeof(record.payload));
1205     safestrncpy(record.result, latest->result, sizeof(record.result));
1206     record.prev_hash = p->agent_context_chain_hash;
1207     record.record_hash = agent_context_record_hash(&record);
1208     if (agent_context_append_receipt_prepare(p, &record, slot, &receipt) < 0)
1209         return -1;
1210
1211     memset(&detail, 0, sizeof(detail));
1212     detail.sequence = latest->sequence;
1213     detail.flags = record.flags;
1214     memmove(&detail.op, op, sizeof(*op));
1215     memmove(&detail.result, latest, sizeof(*latest));
1216     if (agent_context_store(p, slot, &detail,
1217         p->agent_current_span_owner,
1218         p->agent_current_cause_pid,
1219         p->agent_current_cause_control) < 0)
1220         return -1;
1221     agent_context_publish_begin(p);
1222     agent_context_write_record(p, slot, &record);
1223     if (p->context_path_count < p->context_path_capacity) {
1224         if (p->context_path_count == 0)
1225             p->context_path_oldest = latest->sequence;
1226         p->context_path_count++;
1227     } else {
1228         p->context_path_oldest =
1229             latest->sequence - p->context_path_capacity + 1;
1230     }
1231     p->context_path_latest = latest->sequence;
1232     p->context_path_head = (slot + 1) % p->context_path_capacity;
1233     if (record.cause_sequence != 0)
1234         p->agent_provenance_edges++;
1235     if (agent_context_append_receipt_commit(p, &record, &receipt) < 0)
1236         panic("Agent context append receipt");
1237     p->agent_context_chain_hash = record.record_hash;
1238     p->agent_current_cause_sequence = latest->sequence;
1239     p->context_cause_branch_generation = record.branch_generation;

```

```

1240 p->agent_current_cause_pid = p->pid;
1241 p->agent_current_cause_control = p->agent_control_id;
1242 p->agent_current_span_id = record.span_id;
1243 agent_provenance_context_committed(
1244     p, record.request_id, record.tool_id, record.flags);
1245 if (agent_context_write_latest(p, latest) < 0 ||
1246     agent_context_write_header_locked(p) < 0)
1247     panic("Agent context direct publish");
1248 /* Reserved terminal and denial records stay behind an odd sequence until
1249  * their canonical Evidence ticket is committed. The release below is the
1250  * only point at which direct readers may accept the new Context snapshot. */
1251 if (reservation != 0)
1252     *evidence_ticket_out =
1253         agent_observe_commit_context_reserved_ticket(
1254             p, &record, authority_effect, causal_audit,
1255             reservation);
1256 else if (security_reservation != 0)
1257     *evidence_ticket_out =
1258         agent_observe_commit_security_reserved_ticket(
1259             p, &record, security_reservation);
1260 agent_context_publish_end(p);
1261 if (reservation != 0 || security_reservation != 0) {
1262     agent_observe_publish_context_ticket(
1263         p, &record, authority_effect, causal_audit,
1264         *evidence_ticket_out);

```

执行入口在执行约定接纳之前为上下文追加记录做准备；输入来源标签随执行约定声明合并到当前进程。面对截止时间、依赖失败和策略拒绝，代码会尝试生成完成记录或安全记录，让调试和测试区分“未执行”“拒绝”和“执行后失败”。执行记录无法建立时，直接调用和 V3 调用的拒绝仍以 NO\_SPACE 或 RETRY 表达失败。手工注记、回滚和清空不使用执行记录的预留与提交票号规则。

## 代码 12：执行入口中的上下文准备与来源合并

c

```

1577 agent_execute_one(struct proc *p, struct agent_op *op,
1578                 struct agent_result *res, uint64 tick,
1579                 const struct agent_execution_binding *binding,
1580                 uint64 request_deadline_tick, uint admission_policy_flags,
1581                 struct agent_execution_outcome *outcome)
1582 {
1583     struct agent_execution_claim claim;
1584     struct agent_provenance_request provenance_request;
1585     struct agent_provenance_decision provenance_decision;
1586     struct agent_evidence_context_reservation evidence_reservation = { 0 };
1587     struct resource_phase_lease phase_lease = { 0 };
1588     struct resource_account_handle storage = resource_account_none();
1589     enum agent_execution_admission admission;
1590     enum agent_execution_effect_admission effect_admission;
1591     int bound = binding != 0;
1592     int phase_started = 0;
1593     uint64 phase_generation = 0;
1594     uint64 terminal_tick = tick;
1595     int result = 0;
1596     int status;

```



```

1597
1598     if (outcome != 0)
1599         memset(outcome, 0, sizeof(*outcome));
1600     op->payload[AGENT_OP_PAYLOAD_SIZE - 1] = 0;
1601     agent_result_init(res, op);
1602     if (agent_lifecycle_context_lane_enter(p) < 0) {
1603         res->status = AGENT_STATUS_NO_SPACE;
1604         agent_result_text(res, "context_interrupted");
1605         return -1;
1606     }
1607     if (agent_context_append_prepare(p, p->agent_call_count + 1) < 0) {
1608         res->status = AGENT_STATUS_NO_SPACE;
1609         agent_result_text(res, "context_sync_unavailable");
1610         agent_lifecycle_context_lane_leave(p);
1611         return -1;
1612     }
1613     admission = agent_execution_contract_admit(
1614         p, binding, op, request_deadline_tick, admission_policy_flags,
1615         tick, &claim, res, outcome);
1616     if (bound && admission == AGENT_EXECUTION_ADMISSION_EXECUTE &&
1617         claim.input_provenance_labels != 0 &&
1618         agent_provenance_merge_current(
1619             p, claim.input_provenance_labels) != AGENT_STATUS_OK)
1620         panic("contract input provenance merge");
1621     if (outcome != 0 && admission != AGENT_EXECUTION_ADMISSION_CACHED) {
1622         outcome->output_artifact_type = claim.output_artifact_type;
1623         outcome->terminal_tick = terminal_tick;
1624     }
1625     if (admission == AGENT_EXECUTION_ADMISSION_DENIED) {
1626         if (res->status == AGENT_STATUS_DENIED ||
1627             res->status == AGENT_STATUS_STALE) {
1628             result = agent_execution_security_denial(
1629                 p, &claim, op, bound, res, outcome);
1630             agent_execution_publish_policy_denial(
1631                 p, op, res, &claim);
1632             goto out;
1633         }
1634         p->agent_call_count++;
1635         res->sequence = p->agent_call_count;
1636         result = agent_context_append(p, op, res, tick, 0);
1637         if (result < 0)
1638             p->agent_call_count--;
1639         goto out;
1640     }
1641     if (admission == AGENT_EXECUTION_ADMISSION_CACHED) {
1642         if (outcome != 0)
1643             outcome->completion_flags |= AGENT_RESPONSE_V3_F_CACHED;
1644         goto out;
1645     }
1646     if (claim.deadline_expired || claim.dependency_failed) {
1647         if (agent_evidence_context_reserve(
1648             p, &evidence_reservation) < 0) {
1649             res->status = AGENT_STATUS_RETRY;
1650             agent_result_text(res, "evidence_reservation_unavailable");
1651             agent_execution_contract_abort(&claim);
1652             goto out;
1653         }

```

```

1654     p->agent_call_count++;
1655     res->sequence = p->agent_call_count;
1656     res->status = claim.deadline_expired ?
1657         AGENT_STATUS_TIMEOUT : AGENT_STATUS_CANCELLED;
1658     agent_result_text(
1659         res, claim.deadline_expired ?
1660             "contract_deadline" : "dependency_failed");
1661     if (outcome != 0) {
1662         outcome->decision_reason = claim.decision_reason;
1663         outcome->output_provenance_labels =
1664             agent_provenance_current_labels(p);
1665     }
1666     result = agent_execution_append_terminal(
1667         p, op, res, tick, 0, &claim, outcome,
1668         &evidence_reservation);

```

回滚接口只恢复上下文路径的可见记录和分支状态，不会撤销已经发生的文件写入、消息发送或外部服务调用。因此，上下文回滚用于修正后续推理和可见因果关系，不承担通用事务回滚的职责。

### 3.8.4. 元数据查询与实时查询事件

文件查询先确定执行方式：调用者可以明确要求遍历、关闭索引，或者在启用索引时让内核按固定的 `status->stage->kind` 顺序选择第一个给定字段。选择时不比较基数。随后，内核在目录读取快照中扫描候选记录，检查访问范围和筛选条件，并定期让出内核工作预算。结果同时返回命中总数、实际返回数、truncated、是否使用索引、原因、查询时钟数和文件系统代次，供用户态解释本次查询。

#### 代码 13：元数据查询的索引计划与快照执行

c

```

143 static void
144 agent_query_plan_build(const struct agent_file_query *q,
145     struct agent_file_query_result *r,
146     struct agent_query_plan *plan)
147 {
148     memset(plan, 0, sizeof(*plan));
149     plan->limit = q->max_hits;
150     if (plan->limit <= 0 || plan->limit > AGENT_FILE_QUERY_MAX_HITS)
151         plan->limit = AGENT_FILE_QUERY_MAX_HITS;
152     r->plan = AGENT_FILE_QUERY_PLAN_SCAN;
153     r->index_bucket = -1;
154     if (q->flags & AGENT_FILE_QUERY_SCAN) {
155         plan->force_scan = 1;
156         plan->reason |= AGENT_FILE_QUERY_REASON_FORCED_SCAN;
157         return;
158     }
159     if ((q->flags & AGENT_FILE_QUERY_USE_INDEX) == 0) {
160         plan->reason |= AGENT_FILE_QUERY_REASON_INDEX_OFF;
161         return;
162     }
163     for (int i = AGENT_QUERY_INDEX_FIRST; i < AGENT_QUERY_FIELDS; i++) {
164         const struct agent_query_alias *alias = &agent_query_aliases[i];
165         const char *value = (const char *)q + alias->offset;

```

```

166
167     if (!value[0])
168         continue;
169     plan->index = AGENT_CATALOG_INDEX_STATUS +
170         i - AGENT_QUERY_INDEX_FIRST;
171     plan->use_index = 1;
172     plan->index_key = value;
173     r->plan = AGENT_FILE_QUERY_PLAN_STATUS_INDEX +
174         i - AGENT_QUERY_INDEX_FIRST;
175     plan->reason |= AGENT_FILE_QUERY_REASON_STATUS_INDEX <<
176         (i - AGENT_QUERY_INDEX_FIRST);
177     return;
178 }
179 plan->reason |= AGENT_FILE_QUERY_REASON_NO_INDEX_KEY;
180 }
181
182 static void agent_query_result_reset(struct agent_file_query_result *r)
183 {
184     memset(r, 0, sizeof(*r));
185 }
186
187 int
188 agent_metadata_query_execute_snapshot(
189     uint scope, const struct agent_file_query *q,
190     struct agent_file_query_result *r)
191 {
192     struct agent_catalog_read_snapshot snapshot;
193     struct agent_file_meta meta;
194     struct agent_query_plan plan;
195     uint owner;
196     uint work = 0;
197     int bucket = -1;
198     int copied;
199     uint64 start;
200
201     agent_query_result_reset(r);
202     agent_query_plan_build(q, r, &plan);
203     start = agent_file_state_now();
204     copied = agent_metadata_catalog_read_begin(
205         scope, plan.index, plan.index_key, plan.force_scan,
206         &snapshot, &bucket);
207     if (copied < 0)
208         return copied;
209     if (plan.use_index)
210         r->index_bucket = bucket;
211     for (int slot = agent_metadata_catalog_read_next(&snapshot, -1);
212          slot >= 0;
213          slot = agent_metadata_catalog_read_next(&snapshot, slot)) {
214         r->scanned_records++;
215         copied = agent_metadata_catalog_read_copy(
216             &snapshot, slot, &meta, &owner);
217         if (copied < 0)
218             return copied;
219         if (copied > 0 && agent_object_scope_visible(scope, owner)) {
220             r->candidate_records++;
221             if (agent_metadata_query_matches(scope, owner, q, &meta)) {
222                 r->total_hits++;

```

```

223         if (r->returned < plan.limit) {
224             agent_file_state_project_hit(
225                 &r->hits[r->returned++],
226                 &meta, owner);
227         } else {
228             r->truncated = 1;
229         }
230     }
231 }
232 if (++work == AGENT_QUERY_READ_GRANULE) {
233     work = 0;
234     (void)kernel_work_checkpoint_cleanup(
235         KERNEL_WORK_OPERATION_UNITS);
236 }
237 }
238 if (!agent_metadata_catalog_read_end(&snapshot))
239     return AGENT_CATALOG_STALE;
240 r->used_index = plan.use_index;
241 r->index_rebuild_records = 0;
242 r->query_ticks = agent_file_state_now() - start;
243 r->plan_reason = plan.reason;
244 r->fs_generation = snapshot.fs_generation;
245 return r->returned;

```

实时查询根据元数据变化前后的状态生成 ENTER、UPDATE、LEAVE 三类事件。订阅只保存筛选条件和代次，不保存结果集；每个对象都会先检查工作流和访问范围是否可见。只有投递失败或队列出现缺口才需要重新同步，正常的文件系统代次变化仍持续投递三类增量。安全恢复时，智能体保留旧订阅，先按相同条件安装不带 ACK\_RESYNC 的替代订阅，取得 truncated=0 的有界快照后，才在旧订阅上确认缺口并取消订阅；替代订阅覆盖切换窗口。Workflow Fence 会排空延迟工作；只要工作流或访问范围内的黏性重新同步尚未确认，即使标记已投递也返回 RETRY，最后的回收才会清理残留。

#### 代码 14：实时查询的 ENTER、UPDATE、LEAVE 事件投递

c

```

1250 struct workflow_lifecycle_key source_key =
1251     agent_live_query_proc_key(source);
1252 uint source_scope = agent_identity_proc_scope(source);
1253 uint64 fid;
1254 int delivered = 0;
1255
1256 if (!agent_live_query_domain_valid(key, owner_scope) ||
1257     identity == 0 || !identity->used || identity->fid <= 0)
1258     return 0;
1259 if (generation == 0)
1260     generation = agent_file_state_scope_generation(owner_scope);
1261 fid = (uint64)(uint)identity->fid;
1262 for (struct proc *target = pool; target < &pool[NPROC]; target++) {
1263     struct workflow_lifecycle_key target_key;
1264     struct proc *attributed = 0;
1265     uint target_scope;
1266     uint legacy_changes;
1267     uint typed_changes;
1268     int enabled = intr_save();
1269

```

```

1270     if (!agent_live_query_target_visible(target, key, owner_scope) ||
1271         !agent_live_query_target_valid(
1272             target, &target_key, &target_scope)) {
1273         intr_restore(enabled);
1274         continue;
1275     }
1276     legacy_changes = agent_live_query_target_changes(
1277         target, owner_scope, before, after);
1278     typed_changes = agent_live_query_typed_target_changes(
1279         target, owner_scope, before, after);
1280     if (legacy_changes == 0 && typed_changes == 0) {
1281         intr_restore(enabled);
1282         continue;
1283     }
1284     if (source != 0 && source_scope == target_scope &&
1285         agent_live_query_key_equal(source_key, target_key))
1286         attributed = source;
1287     intr_restore(enabled);
1288     for (uint stream = 0; stream < 2; stream++) {
1289         uint changes = stream == 0 ? legacy_changes : typed_changes;
1290         int event_type = stream == 0 ? AGENT_EVENT_FILE_STATUS :
1291             AGENT_EVENT_FILE_QUERY;
1292
1293         for (int change = AGENT_LIVE_QUERY_ENTER;
1294             change <= AGENT_LIVE_QUERY_LEAVE; change++) {
1295             char payload[AGENT_EVENT_PAYLOAD_SIZE];
1296             int result;
1297
1298             if ((changes & (1U << change)) == 0)
1299                 continue;
1300             agent_live_query_payload(payload, change, key);
1301             result = agent_ipc_deliver_live_event(
1302                 target, attributed, event_type, fid,
1303                 generation, payload, 0);
1304             if (result > 0) {
1305                 delivered += result;
1306                 continue;
1307             }
1308             if (result < 0) {
1309                 enabled = intr_save();
1310                 agent_live_query_proc_resync_mark_locked(
1311                     target, generation,
1312                     1U << event_type);
1313                 intr_restore(enabled);
1314                 (void)agent_live_query_proc_resync_flush(
1315                     target);
1316             }
1317         }
1318     }
1319     return delivered;
1320 }
1321

```

## 代码 15: 实时查询的重新同步确认与 Workflow Fence 入口

c

```

1323 void
1324 agent_live_query_watch_installed(struct proc *p)
1325 {
1326     struct workflow_lifecycle_key key;
1327     uint64 generation;
1328     uint scope_id;
1329     int enabled = intr_save();
1330
1331     if (!agent_live_query_target_valid(p, &key, &scope_id) ||
1332         !agent_live_query_has_file_watch(p))
1333         goto out;
1334     generation = agent_file_state_scope_generation(scope_id);
1335     generation = MAX(generation,
1336         agent_live_query_domain_generation_locked(key, scope_id));
1337     agent_live_query_proc_resync_mark_locked(
1338         p, generation, 1U << AGENT_EVENT_FILE_STATUS);
1339     (void)agent_live_query_proc_resync_flush(p);
1340 out:
1341     intr_restore(enabled);
1342 }
1343
1344 void
1345 agent_live_query_watch_removed(struct proc *p)
1346 {
1347     int enabled;
1348     uint slot;
1349
1350     if (p == 0 || p < pool || p >= &pool[NPROC])
1351         return;
1352     enabled = intr_save();
1353     if (!agent_live_query_has_file_watch(p)) {
1354         slot = p - pool;
1355         agent_live_query_proc_resync_clear(
1356             &agent_live_query_proc_resync[slot]);
1357     }
1358     intr_restore(enabled);
1359 }
1360
1361 void
1362 agent_live_query_proc_reset(struct proc *p)
1363 {
1364     int enabled;
1365
1366     if (p == 0 || p < pool || p >= &pool[NPROC])
1367         return;
1368     enabled = intr_save();
1369     for (uint slot = 0; slot < AGENT_LIVE_QUERY_SUBSCRIPTION_CAP; slot++)
1370         if (agent_live_query_subscriptions[slot].used &&
1371             agent_live_query_subscriptions[slot].target == p)
1372             agent_live_query_subscription_remove_locked(
1373                 &agent_live_query_subscriptions[slot], 0);
1374     agent_live_query_proc_resync_clear(
1375         &agent_live_query_proc_resync[p - pool]);
1376     intr_restore(enabled);

```



```

1377 }
1378
1379 int
1380 agent_live_query_resync_ack(struct workflow_lifecycle_key key,
1381                          uint scope_id, uint64 generation)
1382 {
1383     int acknowledged = 0;
1384     int enabled;
1385
1386     if (!agent_live_query_domain_valid(key, scope_id) || generation == 0)
1387         return 0;
1388     enabled = intr_save();
1389     for (uint slot = 0; slot < AGENT_LIVE_QUERY_DOMAIN_CAP; slot++) {
1390         struct agent_live_query_domain_resync *state =
1391             &agent_live_query_domain_resync[slot];
1392
1393         if (!state->used || state->generation > generation ||
1394             !agent_live_query_domain_equal(
1395                 state->key, state->scope_id, key, scope_id))
1396             continue;
1397         memset(state, 0, sizeof(*state));
1398         acknowledged = 1;
1399     }
1400     if (scope_id == VFS_SCOPE_SYSTEM &&
1401         agent_live_query_global_resync_generation != 0 &&
1402         agent_live_query_global_resync_generation <= generation) {
1403         agent_live_query_global_resync_generation = 0;
1404         acknowledged = 1;
1405     }
1406     intr_restore(enabled);
1407     return acknowledged;
1408 }
1409
1410 static int
1411 agent_live_query_proc_resync_pending_domain(
1412     struct workflow_lifecycle_key key, uint scope_id)
1413 {
1414     for (struct proc *target = pool; target < &pool[NPROC]; target++) {
1415         struct agent_live_query_proc_resync *pending =
1416             &agent_live_query_proc_resync[target - pool];
1417
1418         if (!agent_live_query_proc_resync_valid(pending, target))
1419             continue;
1420         if (scope_id == VFS_SCOPE_SYSTEM ||
1421             agent_live_query_domain_equal(
1422                 pending->key, pending->scope_id, key, scope_id))
1423             return 1;
1424     }
1425     return 0;
1426 }
1427
1428 int
1429 agent_live_query_fence_drain(struct workflow_lifecycle_key key, uint scope_id)
1430 {
1431     int retry = 0;
1432
1433     if (!agent_live_query_domain_valid(key, scope_id) ||

```

```

1434     !agent_metadata_txn_owned(0))
1435     return AGENT_STATUS_RETRY;
1436     for (uint slot = 0; slot < AGENT_LIVE_QUERY_TOMBSTONE_CAP; slot++) {
1437         int pending;
1438         int enabled = intr_save();
1439
1440         pending = agent_live_query_tombstones[slot].used &&
1441             agent_live_query_domain_equal(
1442                 agent_live_query_tombstones[slot].key,
1443                 agent_live_query_tombstones[slot].scope_id,
1444                 key, scope_id);
1445         intr_restore(enabled);
1446         if (!pending)
1447             continue;
1448         if (agent_live_query_tombstone_process(slot) < 0)
1449             retry = 1;
1450     }
1451     for (uint slot = 0; slot < AGENT_FILE_META_MAX; slot++) {
1452         int pending;
1453         int enabled = intr_save();
1454
1455         pending = agent_live_query_content_pending[slot].used &&
1456             agent_live_query_domain_equal(
1457                 agent_live_query_content_pending[slot]
1458                     .receipt.lifecycle,
1459                 agent_live_query_content_pending[slot]
1460                     .receipt.scope_id,
1461                 key, scope_id);
1462         intr_restore(enabled);
1463         if (!pending)
1464             continue;
1465         if (agent_live_query_content_process(slot) < 0)
1466             retry = 1;
1467     }
1468     for (struct proc *target = pool; target < &pool[NPROC]; target++) {

```

### 3.8.5. 任务通道与等待路径

任务通道设置只允许智能体主线程使用；SQ 和 CQ 各占一个 Guest 映射页，复制后的请求和完成记录，以及权威资源状态分别放在两页内核私有页。进入操作和资源绑定会记录发起线程编号和身份代次。进入内核后，代码仍检查进入 ABI、CQ 确认、重新同步标志和 SQ 边界。发起者、创建者或生命周期不匹配导致的 STALE 只保留 ABI 版本、大小和 status，其他输出为零；控制代次不匹配时，结果填充函数返回当前通道状态。CQ\_FULL 只是完成队列背压，不会把已接纳的请求直接变为完成记录。目标已确认或不存在时，取消操作仍会消费取消编号，结果填充函数返回当前代次和水位线，也不会额外生成取消 CQE。环形队列或槽位协议错误进入黏性重新同步后，发起者必须以空 SQ 尾指针的显式重新同步进入操作复位。这里的队列是受执行约定和执行记录约束的内核对象，不是普通用户态环形数组。

## 代码 16: 任务通道进入操作的代次、重新同步与 SQ 边界检查

c

```

2012 agent_task_channel_enter(struct proc *p, struct thread *issuer,
2013                          const struct agent_task_channel_enter *enter,
2014                          struct agent_task_channel_enter_result *result,
2015                          const struct agent_task_channel_ops *ops)
2016 {
2017     struct agent_task_channel_state *state;
2018     struct agent_task_private_header *header;
2019     uint submitted = 0;
2020     uint completed = 0;
2021     uint limit;
2022     int status = AGENT_TASK_CHANNEL_OK;
2023     int enabled;
2024
2025     if (result == 0)
2026         return AGENT_TASK_CHANNEL_BAD_REQUEST;
2027     memset(result, 0, sizeof(*result));
2028     result->version = AGENT_TASK_CHANNEL_VERSION;
2029     result->size = sizeof(*result);
2030     result->status = AGENT_TASK_CHANNEL_BAD_REQUEST;
2031     if (p == 0 || issuer == 0 || enter == 0 ||
2032         enter->version != AGENT_TASK_CHANNEL_VERSION ||
2033         enter->size != sizeof(*enter) ||
2034         (enter->flags & ~AGENT_TASK_CHANNEL_ENTER_F_ALL) != 0 ||
2035         ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_DRAIN) != 0 &&
2036          enter->max_submit != 0) ||
2037         enter->max_submit > AGENT_TASK_CHANNEL_CAPACITY ||
2038         enter->min_complete > AGENT_TASK_CHANNEL_CAPACITY ||
2039         enter->reserved != 0 || enter->reserved_tail[0] != 0 ||
2040         enter->reserved_tail[1] != 0)
2041         return result->status;
2042     limit = (enter->flags & AGENT_TASK_CHANNEL_ENTER_F_DRAIN) != 0 ? 0 :
2043         (enter->max_submit == 0 ? AGENT_TASK_CHANNEL_CAPACITY :
2044          enter->max_submit);
2045     enabled = intr_save();
2046     state = agent_task_channel_find_locked(p);
2047     if (!agent_task_channel_issuer_valid_locked(state, p, issuer) ||
2048         state->ops != ops) {
2049         intr_restore(enabled);
2050         result->status = AGENT_TASK_CHANNEL_STALE;
2051         return result->status;
2052     }
2053     header = &state->private->header;
2054     if (enter->generation != header->generation) {
2055         agent_task_channel_enter_result_fill(
2056             state, result, AGENT_TASK_CHANNEL_STALE, 0, 0);
2057         intr_restore(enabled);
2058         return result->status;
2059     }
2060     status = agent_task_channel_ack_locked(state, enter->cq_head);
2061     if (status < 0) {
2062         agent_task_channel_enter_result_fill(state, result, status, 0, 0);
2063         intr_restore(enabled);
2064         return result->status;
2065     }

```

```

2066     header = &state->private->header;
2067     if ((header->flags & AGENT_TASK_CHANNEL_RING_F_RESYNC) != 0) {
2068         if ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_RESYNC) == 0 ||
2069             enter->sq_tail != 0) {
2070             agent_task_channel_enter_result_fill(
2071                 state, result,
2072                 AGENT_TASK_CHANNEL_RESYNC_REQUIRED, 0, 0);
2073             intr_restore(enabled);
2074             return result->status;
2075         }
2076         header->flags &= ~AGENT_TASK_CHANNEL_RING_F_RESYNC;
2077     } else if ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_RESYNC) != 0) {
2078         agent_task_channel_enter_result_fill(
2079             state, result, AGENT_TASK_CHANNEL_BAD_REQUEST, 0, 0);
2080         intr_restore(enabled);
2081         return result->status;
2082     }
2083     if (enter->sq_tail < header->sq_tail ||
2084         enter->sq_tail < header->sq_head ||
2085         enter->sq_tail - header->sq_head > AGENT_TASK_CHANNEL_CAPACITY) {
2086         status = agent_task_channel_protocol_fault_locked(state);
2087         agent_task_channel_enter_result_fill(state, result, status, 0, 0);
2088         intr_restore(enabled);
2089         return result->status;
2090     }
2091     header->sq_tail = enter->sq_tail;
2092     completed += agent_task_channel_flush_locked(state);
2093     agent_task_channel_publish_locked(state);
2094     intr_restore(enabled);
2095     if (ops != 0 && ops->expire != 0) {
2096         status = agent_task_channel_expire(p, agent_task_now(), ops);
2097         if (status < 0)
2098             goto finish;
2099         status = AGENT_TASK_CHANNEL_OK;
2100     }
2101     while (submitted < limit) {
2102         uint consumed = 0;
2103
2104         status = agent_task_channel_consume_one(
2105             p, issuer, enter->generation, ops, &consumed);
2106         submitted += consumed;
2107         if (status <= 0)
2108             break;
2109         if (consumed != 1)
2110             panic("Task Channel consumed result");
2111     }
2112 finish:
2113     enabled = intr_save();
2114     state = agent_task_channel_find_locked(p);
2115     if (!agent_task_channel_issuer_valid_locked(state, p, issuer) ||
2116         state->ops != ops) {
2117         intr_restore(enabled);
2118         result->status = AGENT_TASK_CHANNEL_STALE;
2119         return result->status;
2120     }
2121     completed += agent_task_channel_flush_locked(state);
2122     if (status >= 0)

```

```

2123     status = AGENT_TASK_CHANNEL_OK;
2124     if (status == AGENT_TASK_CHANNEL_OK && enter->min_complete != 0 &&
2125         state->private->header.cq_tail -
2126         state->private->header.cq_head < enter->min_complete)
2127         status = AGENT_TASK_CHANNEL_RETRY;
2128     agent_task_channel_publish_locked(state);
2129     agent_task_channel_enter_result_fill(state, result, status, submitted,
2130                                         completed);
2131     intr_restore(enabled);
2132     return result->status;
2133 }

```

资源导入先根据文件描述符取得可读普通文件，再从 offset 0 读取请求长度并额外探测一个字节。长度、精确 EOF、NUL 与 UTF-8 校验全部通过后，内核把内容、SHA-256、最新非零 Context 序号和 provenance 写入通道私有槽位。IMPORT 本身不追加 Context，不会推进共享文件 offset，也不会把文件对象指针保存到任务句柄中。

SQE 引用资源时仍要核对 slot、type 和 generation。BORROWED 任务只在执行期间取得引用，完成后资源继续可用；OWNED 任务则在终态 CQE（terminal CQE）发布后消费输入。显式释放、自动消费和槽位复用都会推进 generation，旧句柄因此返回 STALE。如果 IMPORT 的最终 copyout 失败，刚建立且尚未对 Guest 可见的资源会被撤销。

#### 代码 17: Task UTF-8 文件 snapshot 的校验与来源绑定

c

```

462 static int
463 agent_task_bridge_resource_import(
464     struct proc *p, struct file *file,
465     const struct agent_task_channel_resource *control,
466     struct agent_task_resource_import *imported)
467 {
468     struct agent_context_record context;
469     struct open_file_io_token lease = OPEN_FILE_IO_TOKEN_INIT;
470     struct vfs_cred cred;
471     uint64 context_hash;
472     uint64 context_sequence;
473     uint64 provenance_labels;
474     uint length;
475     int got;
476     int context_lane_held = 0;
477     int lease_held = 0;
478     int status = AGENT_TASK_CHANNEL_BAD_REQUEST;
479
480     if (imported == 0)
481         return status;
482     memset(imported, 0, sizeof(*imported));
483     if (p == 0 || control == 0 || !p->is_agent || p->pid <= 0 ||
484         p->agent_control_id == 0 ||
485         control->resource_type != AGENT_ARTIFACT_UTF8 ||
486         control->resource_flags != AGENT_TASK_HANDLE_F_OWNED ||
487         control->source_handle >= FD_BUFFER_SIZE || control->length == 0 ||
488         control->length > AGENT_TASK_RESOURCE_UTF8_MAX || file == 0)
489         return status;
490     length = (uint)control->length;
491     if (!file->readable || file->type != FD_INODE || file->ip == 0) {
492         status = AGENT_TASK_CHANNEL_BAD_REQUEST;

```

```

493     goto out;
494 }
495 got = ivalid(file->ip);
496 if (got < 0) {
497     status = got == FS_LOOKUP_BUSY ? AGENT_TASK_CHANNEL_RETRY :
498             AGENT_TASK_CHANNEL_EVIDENCE;
499     goto out;
500 }
501 if (file->ip->type != T_FILE) {
502     status = AGENT_TASK_CHANNEL_BAD_REQUEST;
503     goto out;
504 }
505 vfs_cred_from_proc(p, &cred);
506 if (!vfs_inode_authorize(file->ip, &cred, VFS_OP_READ)) {
507     status = AGENT_TASK_CHANNEL_DENIED;
508     goto out;
509 }
510 if (agent_lifecycle_context_lane_enter(p) < 0) {
511     status = AGENT_TASK_CHANNEL_RETRY;
512     goto out;
513 }
514 context_lane_held = 1;
515 context_sequence = p->context_path_latest;
516 if (context_sequence == 0 || p->context_path_count == 0 ||
517     p->context_path_capacity == 0 ||
518     context_sequence < p->context_path_oldest ||
519     agent_context_read_record(
520         p, (context_sequence - 1U) % p->context_path_capacity,
521         &context) < 0 ||
522     context.sequence != context_sequence || context.record_hash == 0 ||
523     context.record_hash != agent_context_record_hash(&context)) {
524     status = AGENT_TASK_CHANNEL_EVIDENCE;
525     goto out;
526 }
527 context_hash = context.record_hash;
528 provenance_labels = agent_provenance_current_labels(p);
529 if (open_file_io_lease_acquire(
530     file, VFS_OP_READ, &lease, &cred) < 0) {
531     status = AGENT_TASK_CHANNEL_RETRY;
532     goto out;
533 }
534 lease_held = 1;
535 got = readi_lease(file->ip, &cred, &lease, 0,
536     (uint64)imported->snapshot, 0, length + 1U);
537 if (got < 0) {
538     status = got == FS_LOOKUP_BUSY ? AGENT_TASK_CHANNEL_RETRY :
539             AGENT_TASK_CHANNEL_EVIDENCE;
540     goto out;
541 }
542 if ((uint)got != length) {
543     status = AGENT_TASK_CHANNEL_BAD_REQUEST;
544     goto out;
545 }
546 if (p->context_path_latest != context_sequence ||
547     p->context_path_count == 0 || p->context_path_capacity == 0 ||
548     context_sequence < p->context_path_oldest ||
549     agent_context_read_record(

```



```

550     p, (context_sequence - 1U) % p->context_path_capacity,
551     &context) < 0 ||
552     context.sequence != context_sequence ||
553     context.record_hash != context_hash ||
554     context.record_hash != agent_context_record_hash(&context) ||
555     agent_provenance_current_labels(p) != provenance_labels) {
556     status = AGENT_TASK_CHANNEL_RETRY;
557     goto out;
558 }
559 for (uint i = 0; i < length; i++) {
560     if (imported->snapshot[i] == 0) {
561         status = AGENT_TASK_CHANNEL_BAD_REQUEST;
562         goto out;
563     }
564 }
565 if (!agent_task_bridge_utf8_valid(imported->snapshot, length)) {
566     status = AGENT_TASK_CHANNEL_BAD_REQUEST;
567     goto out;
568 }
569 imported->snapshot[length] = 0;
570 status = AGENT_TASK_CHANNEL_OK;
571
572 out:
573     if (lease_held)
574         open_file_io_token_end(&lease);
575     if (context_lane_held)
576         agent_lifecycle_context_lane_leave(p);
577     if (status != AGENT_TASK_CHANNEL_OK) {
578         memset(imported, 0, sizeof(*imported));
579         return status;
580     }
581     imported->resource_type = AGENT_ARTIFACT_UTF8;
582     imported->resource_flags = AGENT_TASK_HANDLE_F_OWNED;
583     imported->producer_node_id = AGENT_EXECUTION_NODE_NONE;
584     imported->producer_pid = p->pid;
585     imported->producer_control_id = p->agent_control_id;
586     imported->source_handle = control->source_handle;
587     imported->length = length;
588     imported->provenance_labels =
589         provenance_labels | AGENT_PROVENANCE_UNTRUSTED_FILE_DATA;
590     imported->producer_context_sequence = context_sequence;
591     agent_sha256(imported->snapshot, length, imported->content_digest);
592     return AGENT_TASK_CHANNEL_OK;
593 }

```

### 3.8.6. 资源使用租约与 workflow EEVDF

资源控制器通过资源使用租约锁定并激活一组资源账户额度。租约记录创建者、生命周期、代次和账户对；激活时，内核将租约令牌写入线程冷状态，停用、结算和放弃时都必须匹配同一代次。这样，一段工具执行只能使用受限且可结算的资源额度。

## 代码 18: 资源使用租约的创建与账户对提交

c

```

1770 int resource_phase_lease_begin(
1771     struct resource_phase_lease *lease,
1772     struct workflow_lifecycle_key lifecycle,
1773     struct resource_account_handle exec_handle,
1774     struct resource_account_handle storage_handle,
1775     enum resource_charge_class charge_class,
1776     const uint64 exec_amounts[RESOURCE_KIND_COUNT],
1777     const uint64 storage_amounts[RESOURCE_KIND_COUNT], uint node_id,
1778     uint64 request_id)
1779 {
1780     struct resource_account *accounts[RESOURCE_PHASE_ACCOUNT_COUNT];
1781     struct resource_phase_account_lock
1782         *locks[RESOURCE_PHASE_ACCOUNT_COUNT];
1783     struct resource_account_handle handles[RESOURCE_PHASE_ACCOUNT_COUNT] = {
1784         [RESOURCE_PHASE_EXEC] = exec_handle,
1785         [RESOURCE_PHASE_STORAGE] = storage_handle,
1786     };
1787     uint64 amounts[RESOURCE_PHASE_ACCOUNT_COUNT][RESOURCE_KIND_COUNT];
1788     uint masks[RESOURCE_PHASE_ACCOUNT_COUNT], union_mask;
1789     struct resource_phase_registry_entry *entry;
1790     struct thread *owner;
1791     int registry_slot;
1792     int enabled;
1793
1794     if (lease == 0 || lease->state != RESOURCE_PHASE_LEASE_EMPTY ||
1795         exec_amounts == 0 || storage_amounts == 0 || request_id == 0 ||
1796         !workflow_lifecycle_key_valid(lifecycle) ||
1797         !resource_charge_class_valid(charge_class) ||
1798         resource_account_handle_equal(exec_handle, storage_handle))
1799         return -1;
1800     memmove(amounts[RESOURCE_PHASE_EXEC], exec_amounts,
1801             sizeof(amounts[RESOURCE_PHASE_EXEC]));
1802     memmove(amounts[RESOURCE_PHASE_STORAGE], storage_amounts,
1803             sizeof(amounts[RESOURCE_PHASE_STORAGE]));
1804     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1805         for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++)
1806             if (amounts[role][kind] > (uint)-1)
1807                 return -1;
1808         masks[role] = resource_phase_amount_mask(amounts[role]);
1809     }
1810     union_mask = masks[RESOURCE_PHASE_EXEC] |
1811                 masks[RESOURCE_PHASE_STORAGE];
1812     if (union_mask == 0)
1813         return -1;
1814
1815     enabled = intr_save();
1816     owner = curr_thread();
1817     registry_slot = resource_phase_registry_free_slot();
1818     accounts[RESOURCE_PHASE_EXEC] = resource_account_lookup(exec_handle);
1819     accounts[RESOURCE_PHASE_STORAGE] =
1820         resource_account_lookup(storage_handle);
1821     if (accounts[RESOURCE_PHASE_EXEC] == 0 ||
1822         accounts[RESOURCE_PHASE_STORAGE] == 0 ||
1823         accounts[RESOURCE_PHASE_EXEC]->kind != RESOURCE_ACCOUNT_EXEC ||

```

```

1824     accounts[RESOURCE_PHASE_STORAGE]->kind !=
1825         RESOURCE_ACCOUNT_STORAGE ||
1826     registry_slot < 0 || owner == 0 || owner->trapframe == 0 ||
1827     owner->identity_generation == 0 || owner->process == 0 ||
1828     !proc_tearardown_live(owner->process) ||
1829     resource_phase_registry_owner_busy(owner) ||
1830     charge_class !=
1831         (owner->process->resource_slot_reserved ?
1832             RESOURCE_CHARGE_RESERVED :
1833             RESOURCE_CHARGE_ORDINARY) ||
1834     resource_phase_generation == RESOURCE_LIMIT_UNBOUNDED ||
1835     !workflow_lifecycle_active(lifecycle) ||
1836     !resource_phase_owner_binding_matches(
1837         lifecycle, exec_handle, owner) ||
1838     !resource_phase_lifecycle_pair_admissible(
1839         lifecycle, handles) ||
1840     !resource_phase_lock_records_locked(accounts, handles, locks) ||
1841     !resource_phase_pair_admissible_locked(
1842         accounts, charge_class, amounts, masks, union_mask)) {
1843     intr_restore(enabled);
1844     return -1;
1845 }
1846 resource_phase_pair_reclaim_locked(
1847     accounts, charge_class, amounts, union_mask);
1848 resource_phase_pair_commit_locked(
1849     accounts, handles, locks, charge_class, amounts, masks);
1850 resource_phase_generation++;
1851 entry = &resource_phase_registry[registry_slot];
1852 memset(entry, 0, sizeof(*entry));
1853 entry->owner = owner;
1854 entry->lease.lifecycle = lifecycle;
1855 entry->lease.generation = resource_phase_generation;
1856 entry->lease.request_id = request_id;
1857 entry->lease.owner_thread_generation = owner->identity_generation;
1858 entry->lease.node_id = node_id;
1859 entry->lease.charge_class = charge_class;
1860 entry->lease.state = RESOURCE_PHASE_LEASE_ADMITTED;
1861 for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1862     entry->lease.account[role].handle = handles[role];
1863     entry->lease.account[role].kind_mask = masks[role];
1864     for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++)
1865         entry->lease.account[role].remaining[kind] =
1866             (uint)amounts[role][kind];
1867 }
1868 entry->used = 1;
1869 lease->generation = resource_phase_generation;
1870 lease->request_id = request_id;
1871 lease->node_id = node_id;
1872 lease->registry_slot = (uint)registry_slot;
1873 lease->state = RESOURCE_PHASE_LEASE_ADMITTED;
1874 intr_restore(enabled);
1875 return 0;
1876 }

```

## 代码 19：资源使用租约的线程激活与去激活

c

```

1878 int resource_phase_lease_activate(struct resource_phase_lease *lease,
1879                                uint64 expected_generation)
1880 {
1881     struct thread *thread;
1882     struct resource_account *account;
1883     struct resource_phase_registry_entry *entry;
1884     struct resource_phase_lease_record *record;
1885     int enabled;
1886
1887     if (lease == 0 || expected_generation == 0)
1888         return -1;
1889     enabled = intr_save();
1890     thread = curr_thread();
1891     entry = resource_phase_registry_lookup(
1892         lease, expected_generation);
1893     record = entry == 0 ? 0 : &entry->lease;
1894     if (entry == 0 || record->state != RESOURCE_PHASE_LEASE_ADMITTED ||
1895         thread == 0 || entry->owner != thread ||
1896         record->owner_thread_generation != thread->identity_generation ||
1897         thread->trapframe == 0 ||
1898         thread->identity_generation == 0 ||
1899         thread_trap_cold(thread)->resource_phase_lease_generation != 0 ||
1900         thread_trap_cold(thread)->resource_phase_claim_depth != 0 ||
1901         !proc_teardown_live(thread->process) ||
1902         !workflow_lifecycle_active(record->lifecycle) ||
1903         !resource_phase_owner_matches(record, thread))
1904         goto fail;
1905     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1906         account = resource_account_lookup(record->account[role].handle);
1907         if (account == 0 || account->state != RESOURCE_ACCOUNT_ACTIVE ||
1908             resource_phase_account_lock_lookup(
1909                 record->account[role].handle) == 0)
1910             goto fail;
1911     }
1912     record->owner_thread_generation = thread->identity_generation;
1913     record->state = RESOURCE_PHASE_LEASE_ACTIVE;
1914     lease->state = RESOURCE_PHASE_LEASE_ACTIVE;
1915     thread_trap_cold(thread)->resource_phase_lease_slot =
1916         lease->registry_slot;
1917     thread_trap_cold(thread)->resource_phase_lease_generation =
1918         expected_generation;
1919     intr_restore(enabled);
1920     return 0;
1921 fail:
1922     intr_restore(enabled);
1923     return -1;
1924 }
1925
1926 int resource_phase_lease_deactivate(struct resource_phase_lease *lease,
1927                                    uint64 expected_generation)
1928 {
1929     struct thread *thread;
1930     struct resource_phase_registry_entry *entry;
1931     struct resource_phase_lease_record *record;

```

```

1932     int enabled;
1933
1934     if (lease == 0 || expected_generation == 0)
1935         return -1;
1936     enabled = intr_save();
1937     thread = curr_thread();
1938     entry = resource_phase_registry_lookup(
1939         lease, expected_generation);
1940     record = entry == 0 ? 0 : &entry->lease;
1941     if (entry == 0 || record->state != RESOURCE_PHASE_LEASE_ACTIVE ||
1942         thread == 0 || entry->owner != thread ||
1943         thread->trapframe == 0 ||
1944         thread_trap_cold(thread)->resource_phase_lease_slot !=
1945             lease->registry_slot ||
1946         thread_trap_cold(thread)->resource_phase_lease_generation !=
1947             expected_generation ||
1948         record->owner_thread_generation != thread->identity_generation ||
1949         record->outstanding_claims != 0 ||
1950         thread_trap_cold(thread)->resource_phase_claim_depth != 0) {
1951         intr_restore(enabled);
1952         return -1;
1953     }
1954     thread_trap_cold(thread)->resource_phase_lease_slot =
1955         RESOURCE_PHASE_REGISTRY_SLOT_NONE;
1956     thread_trap_cold(thread)->resource_phase_lease_generation = 0;
1957     record->state = RESOURCE_PHASE_LEASE_DEACTIVATED;
1958     lease->state = RESOURCE_PHASE_LEASE_DEACTIVATED;
1959     intr_restore(enabled);
1960     return 0;
1961 }

```

调度器不会因为同一工作流拥有更多线程，就把它当作更多独立的公平实体。workflow\_scheduler\_select() 先收集可运行工作流，再依据滞后量、截止时间、权重和可运行负载选择实体；调度和记账分别记录服务开始时刻与已消耗时间。因此，调度公平性按工作流记账，而不是按偶然增加的线程数记账。

## 代码 20：工作流 EEVDF 的候选收集与选择

c

```

408 int workflow_scheduler_select(
409     const struct workflow_scheduler_candidate *candidates, uint count,
410     uint64 now_cycle, uint64 now_tick, int *selected_domain)
411 {
412     int slots[WORKFLOW_SCHEDULER_ENTITY_CAP];
413     uint planned_latency[WORKFLOW_SCHEDULER_ENTITY_CAP];
414     uint planned_request_ticks[WORKFLOW_SCHEDULER_ENTITY_CAP];
415     uint planned_decays[WORKFLOW_SCHEDULER_ENTITY_CAP];
416     uint64 planned_vruntime[WORKFLOW_SCHEDULER_ENTITY_CAP];
417     uint64 planned_remaining[WORKFLOW_SCHEDULER_ENTITY_CAP];
418     uint64 planned_deadline[WORKFLOW_SCHEDULER_ENTITY_CAP];
419     uint reserved = 0;
420     uint new_slots = 0;
421     uint wake_transitions = 0;
422     uint64 rebase_offset;
423     uint64 planned_vtime;
424     int selected = -1;

```

```

425     int enabled;
426
427     (void)now_cycle;
428     if (selected_domain == 0 || candidates == 0 || count == 0 ||
429         count > WORKFLOW_SCHEDULER_ENTITY_CAP)
430         return -1;
431     *selected_domain = -1;
432     for (uint i = 0; i < WORKFLOW_SCHEDULER_ENTITY_CAP; i++)
433         slots[i] = WORKFLOW_SCHEDULER_SLOT_NONE;
434     enabled = intr_save();
435     /* Phase one is read-only: validate every identity and reserve local slots. */
436     for (uint i = 0; i < count; i++) {
437         int is_new = 0;
438
439         if (!workflow_scheduler_identity_valid(
440             candidates[i].lifecycle, candidates[i].account,
441             candidates[i].domain_id) ||
442             candidates[i].runnable_threads == 0 ||
443             candidates[i].runnable_threads > 0xffffU ||
444             candidates[i].latency_class >
445             AGENT_WORKFLOW_LATENCY_BATCH)
446             goto fail;
447         for (uint j = 0; j < i; j++)
448             if (candidates[j].domain_id ==
449                 candidates[i].domain_id ||
450                 workflow_lifecycle_key_equal(
451                     candidates[j].lifecycle,
452                     candidates[i].lifecycle) ||
453                 resource_account_handle_equal(
454                     candidates[j].account,
455                     candidates[i].account))
456                 goto fail;
457         planned_request_ticks[i] = workflow_scheduler_request_ticks(
458             candidates[i].latency_class,
459             candidates[i].deadline_delta_ticks);
460         if (planned_request_ticks[i] == 0)
461             goto fail;
462         slots[i] = workflow_scheduler_plan_slot_locked(
463             &candidates[i], reserved, &is_new);
464         if (slots[i] < 0)
465             goto fail;
466         reserved |= 1U << slots[i];
467         if (is_new)
468             new_slots |= 1U << i;
469         else if (workflow_scheduler_entities[slots[i]].flags &
470             WORKFLOW_SCHEDULER_F_CACHE_INVALID)
471             goto fail;
472     }
473
474     /* Compute the entire post-selection state locally before any mutation. */
475     rebase_offset = workflow_scheduler_rebase_offset_locked();
476     planned_vtime = workflow_scheduler_vtime >= rebase_offset ?
477         workflow_scheduler_vtime - rebase_offset : 0;
478     for (uint i = 0; i < count; i++) {
479         struct workflow_scheduler_entity *entity =
480             &workflow_scheduler_entities[slots[i]];
481         uint64 request_cycles = workflow_scheduler_request_cycles(

```



```

482     planned_request_ticks[i]);
483
484     planned_decays[i] = 0;
485     if (new_slots & (1U << i)) {
486         planned_vruntime[i] = planned_vtime;
487         planned_remaining[i] = 0;
488         planned_latency[i] = AGENT_WORKFLOW_LATENCY_NORMAL;
489         wake_transitions |= 1U << i;
490     } else {
491         planned_vruntime[i] = entity->vruntime - rebase_offset;
492         planned_remaining[i] = entity->remaining_cycles;
493         planned_latency[i] = entity->latency_class;
494         planned_request_ticks[i] = entity->request_ticks;
495         if (!(entity->flags &
496             AGENT_WORKFLOW_SCHED_F_RUNNABLE)) {
497             wake_transitions |= 1U << i;
498             planned_remaining[i] = 0;
499             if (entity->flags &
500                 AGENT_WORKFLOW_SCHED_F_SLEEPING) {
501                 uint64 slept =
502                     now_tick >= entity->sleep_start_tick ?
503                     now_tick -
504                     entity->sleep_start_tick :
505                     0;
506
507                 planned_decays[i] = slept /
508                     WORKFLOW_SCHEDULER_SLEEP_DECAY_TICKS;
509                 if (planned_decays[i] >
510                     WORKFLOW_SCHEDULER_MAX_SLEEP_DECAYS)
511                     planned_decays[i] =
512                         WORKFLOW_SCHEDULER_MAX_SLEEP_DECAYS;
513                 planned_vruntime[i] =
514                     workflow_scheduler_decay_toward(
515                         planned_vruntime[i],
516                         planned_vtime,
517                         planned_decays[i]);
518             }
519         }
520     }
521     /* Urgency may shorten an open request, never mint more service. */
522     if (planned_remaining[i] == 0 ||
523         request_cycles < planned_remaining[i]) {
524         planned_remaining[i] = request_cycles;
525         planned_request_ticks[i] = workflow_scheduler_request_ticks(
526             candidates[i].latency_class,
527             candidates[i].deadline_delta_ticks);
528         planned_latency[i] = candidates[i].latency_class;
529     }
530     if (planned_vruntime[i] > ~0ULL - planned_remaining[i])
531         goto fail;
532     planned_deadline[i] =
533         planned_vruntime[i] + planned_remaining[i];
534 }
535 if (count == 1) {
536     if (planned_vruntime[0] > planned_vtime)
537         planned_vtime = planned_vruntime[0];
538 } else {

```

```

539     uint64 average = workflow_scheduler_average(
540         planned_vruntime, count);
541
542     if (average > planned_vtime)
543         planned_vtime = average;
544 }
545 for (uint i = 0; i < count; i++) {
546     if (planned_vruntime[i] <= planned_vtime) {
547         if (selected < 0 ||
548             workflow_scheduler_before(candidates,
549                                     planned_deadline,
550                                     planned_vruntime, i,
551                                     (uint)selected))
552             selected = i;
553     }
554 }
555 if (selected < 0)
556     goto fail;

```

## 代码 21： workflow EEVDF 的提交、调度与记账

c

```

558     /* Phase two commits a plan that can no longer fail. */
559     if (rebase_offset != 0)
560         workflow_scheduler_rebase_locked();
561     for (uint i = 0; i < count; i++) {
562         struct workflow_scheduler_entity *entity =
563             &workflow_scheduler_entities[slots[i]];
564
565         if (new_slots & (1U << i)) {
566             memset(entity, 0, sizeof(*entity));
567             entity->lifecycle = candidates[i].lifecycle;
568             entity->account = candidates[i].account;
569             entity->domain_id = candidates[i].domain_id;
570             entity->flags = AGENT_WORKFLOW_SCHED_F_ACTIVE;
571             entity->earliest_deadline_tick =
572                 WORKFLOW_SCHEDULER_NO_DEADLINE;
573             entity->cached_latency_class =
574                 (uchar)candidates[i].latency_class;
575             entity->schedulable_threads =
576                 (ushort)candidates[i].runnable_threads;
577         }
578         workflow_scheduler_domain_slot[candidates[i].domain_id] =
579             (signed char)slots[i];
580         entity->vruntime = planned_vruntime[i];
581         entity->remaining_cycles = planned_remaining[i];
582         entity->virtual_deadline = planned_deadline[i];
583         entity->request_ticks = (uchar)planned_request_ticks[i];
584         entity->latency_class = (uchar)planned_latency[i];
585         entity->runnable_threads =
586             (ushort)candidates[i].runnable_threads;
587         entity->mode = AGENT_WORKFLOW_SCHED_MODE_EEVDF;
588         if (wake_transitions & (1U << i)) {
589             entity->sleep_decays = workflow_scheduler_counter32_add(
590                 entity->sleep_decays, planned_decays[i]);
591             entity->wake_tick = now_tick;
592             entity->flags |= WORKFLOW_SCHEDULER_F_WAKE_PENDING;

```

```

593     }
594     entity->flags &= ~(AGENT_WORKFLOW_SCHED_F_SLEEPING |
595         AGENT_WORKFLOW_SCHED_F_ELIGIBLE |
596         AGENT_WORKFLOW_SCHED_F_FALLBACK);
597     entity->flags |= AGENT_WORKFLOW_SCHED_F_RUNNABLE;
598     if (planned_vruntime[i] <= planned_vtime)
599         entity->flags |= AGENT_WORKFLOW_SCHED_F_ELIGIBLE;
600     else
601         entity->eligibility_misses =
602             workflow_scheduler_counter32_add(
603                 entity->eligibility_misses, 1);
604     }
605     workflow_scheduler_vtime = planned_vtime;
606     if (candidates[selected].deadline_delta_ticks == 0) {
607         struct workflow_scheduler_entity *entity =
608             &workflow_scheduler_entities[slots[selected]];
609
610         if (!(entity->flags & WORKFLOW_SCHEDULER_F_DEADLINE_MISSED)) {
611             entity->flags |= WORKFLOW_SCHEDULER_F_DEADLINE_MISSED;
612             entity->deadline_misses = workflow_scheduler_counter32_add(
613                 entity->deadline_misses, 1);
614         }
615     }
616     *selected_domain = candidates[selected].domain_id;
617     intr_restore(enabled);
618     return 0;

```

#### 算法 4：资源与 CPU 的工作流级执行过程

1. 执行约定把节点的执行额度、存储额度和记账类别写入当前声明。
2. 资源使用租约锁定、激活并结算本阶段的账户额度；超限或状态失效时返回受控结果。
3. 可运行线程按工作流汇集，EEVDF 根据实体的滞后量和截止时间选择下一次服务。
4. 执行约定路径在上下文和执行记录中写入终止状态（terminal state），使资源拒绝、取消和超时能够回溯到具体节点和尝试号。

### 3.8.7. Guest runtime（客户机运行时）与结果文件发布

Nexus 用户态库会重新发现内核工具目录，逐项核对 ABI、编号、名称和 schema 是否一致；发起调用时还会检查当前智能体身份、产品角色和必要能力位。用户态校验不替代内核，而是让模型适配层尽早排除不可能成功的工具调用。

#### 代码 22：Nexus 对内核工具目录的发现与一致性检查

c

```

681     return agent_nexus_task_decode_n(text, length, task);
682 }
683
684 int agent_nexus_tools_discover(void)
685 {
686     unsigned char seen[AGENT_TOOL_COUNT];
687     int count;
688

```

```

689     memset(nexus_tool_catalog, 0, sizeof(nexus_tool_catalog));
690     memset(seen, 0, sizeof(seen));
691     nexus_tool_catalog_count = 0;
692     count = tool_list(nexus_tool_catalog, AGENT_TOOL_COUNT);
693     if (count != AGENT_TOOL_COUNT)
694         return -1;
695     for (int i = 0; i < count; i++) {
696         const struct agent_tool_desc_v2 *descriptor =
697             &nexus_tool_catalog[i];
698         const struct agent_nexus_tool_spec *spec;
699
700         if (descriptor->version != AGENT_CALL_VERSION_V2 ||
701             nexus_tool_catalog[i].size != sizeof(nexus_tool_catalog[i]) ||
702             descriptor->tool_id <= 0 ||
703             descriptor->tool_id > AGENT_TOOL_COUNT ||
704             strlen(descriptor->name, sizeof(descriptor->name)) ==
705                 sizeof(descriptor->name) ||
706             strlen(descriptor->params, sizeof(descriptor->params)) ==
707                 sizeof(descriptor->params) ||
708             seen[descriptor->tool_id - 1] != 0)
709             goto invalid;
710         spec = &nexus_tool_specs[descriptor->tool_id - 1];
711         if (spec->tool_id != descriptor->tool_id ||
712             !nexus_text_equal_bounded(spec->name, descriptor->name,
713                                     AGENT_TOOL_NAME_SIZE) ||
714             !nexus_text_equal_bounded(spec->parameters,
715                                     descriptor->params,
716                                     AGENT_TOOL_PARAMS_SIZE))
717             goto invalid;
718         seen[descriptor->tool_id - 1] = 1;
719     }
720     nexus_tool_catalog_count = count;
721     return count;
722
723 invalid:

```

### 代码 23: Nexus 带类型信息工具调用的用户态封装

c

```

894     return argument_index == argument_count;
895 }
896
897 int agent_nexus_tool_call(const char *name, unsigned long long request_id,
898                          const struct agent_nexus_tool_argument *arguments,
899                          unsigned int argument_count,
900                          struct agent_response_v2 *response)
901 {
902     const struct agent_tool_desc_v2 *tool = agent_nexus_tool_find(name);
903     const struct agent_nexus_tool_spec *spec = agent_nexus_tool_spec_find(name);
904     struct agent_request_v2 request;
905     struct agent_nexus_artifact_actor identity;
906     struct agent_info info;
907     unsigned int role_bit;
908
909     if (tool == 0 || spec == 0 || response == 0 || request_id == 0 ||
910         argument_count > AGENT_TOOL_PARAM_MAX ||
911         (argument_count != 0 && arguments == 0) ||

```

```

912     (tool->flags & AGENT_TOOL_F_CALLABLE) == 0 ||
913     argument_count > tool->param_count ||
914     !nexus_schema_arguments_valid(spec, arguments, argument_count))
915     return -1;
916     if (agent_nexus_identity_current(&identity) < 0)
917         return -1;
918     role_bit = AGENT_NEXUS_TOOL_ROLE(identity.product_role);
919     memset(&info, 0, sizeof(info));
920     if (agent_info(&info) != 0 || !info.is_agent ||
921         info.agent_id != (int)identity.agent_id ||
922         (spec->product_role_mask & role_bit) == 0 ||
923         (info.capability_mask & spec->required_capabilities) !=
924         spec->required_capabilities)
925         return -1;
926     memset(&request, 0, sizeof(request));
927     memset(response, 0, sizeof(*response));
928     memset(nexus_tool_params, 0, sizeof(nexus_tool_params));
929     request.version = AGENT_CALL_VERSION_V2;
930     request.size = sizeof(request);
931     request.tool_id = tool->tool_id;
932     request.param_count = argument_count;
933     request.request_id = request_id;
934     request.params = argument_count ?
935         (unsigned long long)nexus_tool_params : 0;
936     strcpy(request.tool_name, tool->name);
937     for (unsigned int i = 0; i < argument_count; i++) {
938         struct agent_param_v2 *param = &nexus_tool_params[i];
939
940         param->version = AGENT_PARAM_VERSION;
941         param->size = sizeof(*param);
942         param->type = arguments[i].type;
943         strcpy(param->key, arguments[i].key);
944         if (arguments[i].type == AGENT_PARAM_UINT64) {
945             param->value_size = sizeof(unsigned long long);
946             param->value.uint64_value = arguments[i].number;
947         } else {
948             unsigned int length = strlen(arguments[i].text,
949                 sizeof(arguments[i].text)) + 1U;
950
951             param->value_size = length;
952             strcpy(param->value.string_value, arguments[i].text);
953         }
954     }
955     if (tool_call(&request, response) != 0 ||
956         response->version != AGENT_CALL_VERSION_V2 ||
957         response->size != sizeof(*response) ||
958         response->request_id != request_id || response->tool_id != tool->tool_id ||
959         strlen(response->tool_name, sizeof(response->tool_name)) ==
960         sizeof(response->tool_name) ||
961         !nexus_text_equal_bounded(response->tool_name, tool->name,

```

agent\_file\_publish() 先把 header 与 payload 一并拷入按 workflow 账户计费的内核页，再把这份完整 snapshot 交给 VFS。路径必须是 workflow 访问范围中的单个正式文件名，已有文件不会被覆盖。用户态缓冲区在 syscall 返回前即与后续 I/O 解耦。

VFS 为结果分配未命名 inode，完成标签、空间预留、数据写入和第一次 checkpoint；

只有数据块和最终 `inode` 形态已经持久，才用一次 `dirlink` 接入正式目录并提交第二次 `checkpoint`。已知的接入失败会回收未命名 `inode`；I/O 状态不确定时则保留可能已经接入目录的 `inode`，并返回 `INDETERMINATE`。

`Nexus` 收到 `DUPLICATE` 或 `INDETERMINATE` 后不会直接重试写入。它打开正式路径，逐字节比较 `header`、`payload` 和 `EOF`；只有完整内容与本次请求一致才把发布收敛为成功。这样，其他 `Agent` 看见的是一份完整结果文件，而不是带无效 `header` 或缺少尾部 `payload` 的中间状态。

#### 代码 24：未命名 `inode` 的完整写入与单次目录接入

c

```

7701 int fs_agent_file_publish_atomic(char *path, const struct vfs_cred *cred,
7702                                const void *snapshot, uint total)
7703 {
7704     struct fs_storage_charge charge;
7705     struct inode *dp = 0;
7706     struct inode *existing = 0;
7707     struct inode *ip = 0;
7708     uint intent_owner;
7709     char key[DIRSIZ + 1];
7710     int lookup_status = FS_LOOKUP_ERROR;
7711     int root_status = FS_LOOKUP_ERROR;
7712     int result;
7713     int status;
7714
7715     if (!fs_agent_publish_name_valid(path) || cred == 0 || snapshot == 0 ||
7716         total == 0 || total > AGENT_FILE_PUBLISH_MAX_BYTES)
7717         return AGENT_STATUS_BAD_PARAM;
7718     if (fs_dirent_canonicalize(path, key) < 0)
7719         return AGENT_STATUS_BAD_PARAM;
7720     dp = root_dir_status(&root_status);
7721     if (dp == 0 || root_status != FS_LOOKUP_FOUND) {
7722         if (dp != 0)
7723             iput(dp);
7724         return fs_agent_publish_io_status(root_status);
7725     }
7726     if (!vfs_inode_authorize(dp, cred, VFS_OP_CREATE) ||
7727         !vfs_create_request_authorize(cred, VFS_POLICY_WORKFLOW, 0, 1, 0) ||
7728         cred->scope_id < VFS_SCOPE_FIRST_DYNAMIC) {
7729         iput(dp);
7730         return AGENT_STATUS_DENIED;
7731     }
7732     existing = dirlookup(dp, key, 0, VFS_POLICY_WORKFLOW,
7733                         cred->scope_id, &lookup_status);
7734     if (existing != 0) {
7735         iput(existing);
7736         iput(dp);
7737         return AGENT_STATUS_DUPLICATE;
7738     }
7739     if (lookup_status != FS_LOOKUP_ABSENT) {
7740         iput(dp);
7741         return fs_agent_publish_io_status(lookup_status);
7742     }
7743     if (fs_storage_charge_from_vfs(cred, &charge) < 0) {
7744         iput(dp);

```

```

7745     return AGENT_STATUS_DENIED;
7746 }
7747 ip = ialloc(dp->dev, T_FILE, &charge, &lookup_status);
7748 if (ip == 0) {
7749     iput(dp);
7750     if (lookup_status == FS_LOOKUP_BUSY)
7751         return AGENT_STATUS_RETRY;
7752     return fs_io_health == FS_IO_HEALTHY ? AGENT_STATUS_NO_SPACE :
7753         fs_agent_publish_io_status(lookup_status);
7754 }
7755 result = invalid(ip);
7756 if (result < 0)
7757     goto fail_inode;
7758 if (ip->type != T_FILE ||
7759     fs_qmap_transition_owner(ip->fs_owner_domain,
7760         FS_QMAP_ALLOCATING_FLAG,
7761         &intent_owner) < 0 ||
7762     intent_owner != charge.owner) {
7763     result = -1;
7764     goto fail_inode;
7765 }
7766 ip->fs_owner_domain = charge.owner;
7767 result = vfs_inode_init_label(ip, cred, VFS_POLICY_WORKFLOW);
7768 if (result < 0)
7769     goto fail_inode;
7770 result = iupdate(ip);
7771 if (result < 0)
7772     goto fail_inode;
7773 result = fs_preallocate_inode(ip, cred, total);
7774 if (result < 0) {
7775     if (result == FS_LOOKUP_BUSY ||
7776         result == VIRTIO_DISK_ERR_BUSY)
7777         status = AGENT_STATUS_RETRY;
7778     else
7779         status = fs_io_health == FS_IO_HEALTHY ?
7780             AGENT_STATUS_NO_SPACE :
7781             fs_agent_publish_io_status(result);
7782     goto fail_known;
7783 }
7784 result = writei(ip, cred, 0, (uint64)snapshot, 0, total);
7785 if (result != (int)total) {
7786     status = result > 0 && fs_io_health == FS_IO_HEALTHY ?
7787         AGENT_STATUS_RETRY : fs_agent_publish_io_status(result);
7788     goto fail_known;
7789 }
7790 /* The inode image and every data block must be durable before its only
7791  * namespace attachment can be staged. */
7792 result = fs_publish_checkpoint();
7793 if (result < 0) {
7794     status = AGENT_STATUS_INDETERMINATE;
7795     goto fail_known;
7796 }
7797 result = dirlink_publish(dp, key, ip->inum, cred);
7798 if (result == 0) {
7799     iput(ip);
7800     iput(dp);
7801     return AGENT_STATUS_OK;

```



```

7802     }
7803     if (result == FS_LOOKUP_INDETERMINATE ||
7804         fs_io_health == FS_IO_INDETERMINATE) {
7805         /* The official dirent may already exist; do not reclaim its inode. */
7806         iput(ip);
7807         iput(dp);
7808         return AGENT_STATUS_INDETERMINATE;
7809     }
7810     if (result == FS_DIRLINK_EXISTS)
7811         status = AGENT_STATUS_DUPLICATE;
7812     else if (result == FS_DIRLINK_NO_SPACE)
7813         status = AGENT_STATUS_NO_SPACE;
7814     else
7815         status = fs_agent_publish_io_status(result);
7816     goto fail_known;
7817
7818 fail_inode:
7819     status = fs_agent_publish_io_status(result);
7820 fail_known:
7821     iput(dp);
7822     return fs_agent_publish_discard(ip, status);
7823 }

```

## 代码 25: Nexus 对重复或不确定发布的精确核对

c

```

1506 static int nexus_artifact_store(
1507     struct agent_nexus_artifact_header *stored, const void *payload,
1508     unsigned int payload_size)
1509 {
1510     struct agent_nexus_artifact_header digest_header;
1511     char path[AGENT_NEXUS_ARTIFACT_PATH_SIZE];
1512     int publish_status;
1513     struct agent_nexus_artifact_header existing_header;
1514     unsigned char chunk[64];
1515     const unsigned char *expected = payload;
1516     unsigned int offset = 0;
1517     char extra;
1518     ssize_t tail;
1519     int close_status;
1520     int fd;
1521
1522     if (payload == 0 || payload_size == 0 ||
1523         payload_size > AGENT_NEXUS_ARTIFACT_MAX ||
1524         agent_nexus_artifact_path(stored->handle, path) < 0)
1525         return -1;
1526     agent_nexus_sha256(payload, payload_size, stored->payload_sha256);
1527     memset(stored->manifest_sha256, 0, sizeof(stored->manifest_sha256));
1528     digest_header = *stored;
1529     agent_nexus_sha256(&digest_header, sizeof(digest_header),
1530         stored->manifest_sha256);
1531     publish_status = agent_file_publish(
1532         path, stored, sizeof(*stored), payload, payload_size);
1533     if (publish_status == AGENT_STATUS_OK)
1534         return 0;
1535     if (publish_status != AGENT_STATUS_DUPLICATE &&
1536         publish_status != AGENT_STATUS_INDETERMINATE)

```

```
1537     return -1;
1538
1539     /* A duplicate or indeterminate attach converges only when the official
1540      * path contains the exact requested bytes. */
1541     fd = open(path, O_RDONLY);
1542
1543     if (fd < 0 ||
1544         nexus_read_all(fd, &existing_header, sizeof(existing_header)) < 0 ||
1545         !nexus_bytes_equal(&existing_header, stored, sizeof(*stored)))
1546         goto mismatch;
1547     while (offset < payload_size) {
1548         unsigned int take = payload_size - offset;
1549
1550         if (take > sizeof(chunk))
1551             take = sizeof(chunk);
1552         if (nexus_read_all(fd, chunk, take) < 0 ||
1553             !nexus_bytes_equal(chunk, expected + offset, take))
1554             goto mismatch;
1555         offset += take;
1556     }
1557     tail = read(fd, &extra, 1);
1558     close_status = close(fd);
1559     return tail == 0 && close_status == 0 ? 0 : -1;
1560
1561 mismatch:
1562     if (fd >= 0)
1563         (void)close(fd);
1564     return -1;
1565 }
```

## 4. 项目测试

测试先核对接口结构和构建结果，再进入 RISC-V Guest 检查各项内核机制，最后运行控制台、Nexus 和相应的性能负载。功能结果来自真实系统调用输出和协议摘要；性能数据保留逐样本来源，配对比较、分布图和统计表都能追溯到对应的启动记录。各组测量随所检查的机制展开，不另设脱离功能路径的展示性实验。

### 4.1. 功能与性能测试

#### 4.1.1. 测试组织

AgentOS-uCore 的测试从接口结构、内核镜像、Guest 行为和综合场景四个方向交叉检查。结构约定检查核对 UAPI 的尺寸、偏移和模块依赖；交叉编译把真实内核和用户程序链接为 RISC-V 镜像；Guest 场景通过系统调用观察 Agent 身份、Context、VFS、IPC、资源和调度状态；Host 测试则覆盖帧编码、Provider、N1、artifact 与验证脚本。性能测量直接读取同一 Guest 工作负载产生的时钟和计数，不另写一套只用于出图的模拟程序。

| 对应机制                | 主要程序                                                                               | 观察重点                                                 |
|---------------------|------------------------------------------------------------------------------------|------------------------------------------------------|
| Agent 进程与 Context 区 | agenteval_ucore,<br>agenttrust_ucore,<br>agentscope_ucore,<br>agentfinal_ucore     | 创建、派生、装入和退出；固定映射、直接读取、回滚、清空与 FIFO 淘汰                 |
| 结构化接口与工具协议          | agenteval_ucore,<br>agenttoolabi_ucore,<br>agentcontract_ucore,<br>agenttask_ucore | 目录与 schema；V2/V3；UTF-8 资源导入；重试、取消、截止时间；SQ/CQ 背压与重新同步 |
| 文件查询与实时事件           | agenteval_ucore,<br>agentfs_ucore,<br>agentbench_ucore                             | 元数据与摘要；遍历和索引结果一致性；ENTER、UPDATE、LEAVE 与缺口恢复           |
| Agent Loop 与 EEVDF  | agenteval_ucore,<br>agentloop_ucore,<br>agentsched_ucore,<br>agenteevdf_ucore      | 原子等待、消息与 TIMER 唤醒、超时、线程放大、工作流服务份额                    |
| 综合场景                | labdemo_ucore,<br>agentpublish_ucore,<br>agentnexus_ucore                          | 恢复流程、三轮 Nexus 交互、四个业务角色、结果文件原子发布、失败后重排、审批拒绝和会话关闭     |

表 4.1: 系统机制、测试程序与观察重点

自动化检查依次覆盖智能体模块与 ABI 结构约定、Nexus 协议约定和 Host 测试程序。严格的 Nexus ReplayProvider 还会在一轮全新 QEMU 启动中产生 11 个请求摘要，连续进行 3 轮交互，并走过研究句柄失效后的任务重排、报告审批拒绝和完整关闭路径。

```

1 make agent-module-check
2 make agentos-build TOOLPREFIX=riscv64-linux-gnu-
3 make agentos-nexus-check TOOLPREFIX=riscv64-linux-gnu-
4 make local-host-selftests

```

### 4.1.2. 任务一至任务五的 Guest 综合验收

agenteval\_ucore 把赛题任务一至任务五放在同一个 RV64 Guest 中检查。运行命令如下：

```
1 AGENT_TEST_CASE=agenteval_ucore \
2 make agentos-test TOOLPREFIX=riscv64-linux-gnu-
```

这项测试关注系统调用之后真正可见的状态，而不是在 Host 侧搜索“成功”字符串。五段场景的观察内容见表4.2。

| 场景                | Guest 中可观察的行为                                                                 |
|-------------------|-------------------------------------------------------------------------------|
| Agent 进程创建与地址空间   | Agent 与普通进程可以并存；身份和配额可查询；固定高地址的七页 Context 区中，六页内核镜像保持只读，用户缓存页可写。              |
| 内核结构化交互接口         | Guest 能列举工具并发起多种带类型信息的调用；未知工具、错误参数类型和小返回缓冲区得到不同的结构化状态。                        |
| 上下文路径管理           | 六轮以上连续调用的直接映射读取与系统调用查询一致；回滚和清空改变后续可见路径；连续写入 133 条记录后只保留最新 128 条，系统继续运行且未耗尽内存。 |
| 面向 Agent 的文件查询    | 文件按属性和摘要被找到，不要求调用者预先知道完整路径；遍历与索引返回同一结果；结构化结果可以写入第七页用户缓存。                      |
| Agent Loop 内核运行机制 | 等待线程在无事件时睡眠，消息或 TIMER 到达后继续运行；多个 Agent 同时活动时，工作流调度统计持续推进。                     |

表 4.2: agenteval\_ucore 的五段可观察行为

这组综合验收把创建、工具调用、Context、文件查询和 Loop 串在同一次启动中，能够发现单项测试不易暴露的状态衔接问题。例如，Context 映射若只在创建时存在、exec 后丢失，后续的查询缓存场景便无法继续；文件事件若没有正确唤醒等待者，Loop 场景也不会仅凭查询结果“看起来正常”。

### 4.1.3. 结构化工具与任务通道测量

任务通道测量让紧凑 Batch、单项 V3 和 SQ/CQ 分别执行 16 次等价 ECHO，并轮换三条路径的先后顺序。每条操作序列记录完整墙钟时间，同时核对工具、状态、Context 序号、执行记录票号和结果指纹，避免把少做检查或少执行操作误判为性能提升。4 次启动共保留 96 条操作序列。

图4.1中，紧凑 Batch、单项 V3 和 SQ/CQ 完成 16 项等价操作的中位数分别为 561 微秒、2051 微秒和 1620.5 微秒。三条路径分别使用 1、16、2 次系统调用：Batch 在一次进入内顺序执行短操作；V3 为每一项绑定执行约定、节点和尝试号；SQ/CQ 通过共享队列提交并回收结果。每次启动的首轮中位数依次为 5.747、7.098、6.1435 ms，后续轮次降至 0.548、1.9965、1.5615 ms，说明刚进入该路径时的建立和缓存开销会明显抬高延迟，不能只报后续轮次的中位数。

这一结果表明，短小、同质且需要同步返回的操作适合 Batch，系统没有必要为了接口形式统一而强制走更重的路径。V3 多付出的时间对应 Execution Contract、节点、尝试号和执行记录检查；SQ/CQ 的价值则在于有界提交、背压、取消和唯一 terminal CQE，而不是压低 16 个短调用的最低延迟。当前 Provider 延迟很短，Task Bridge 也按同步方式处理，本组结果还不能说明队列面对长 Provider 延迟和异步 backlog 时的表现。

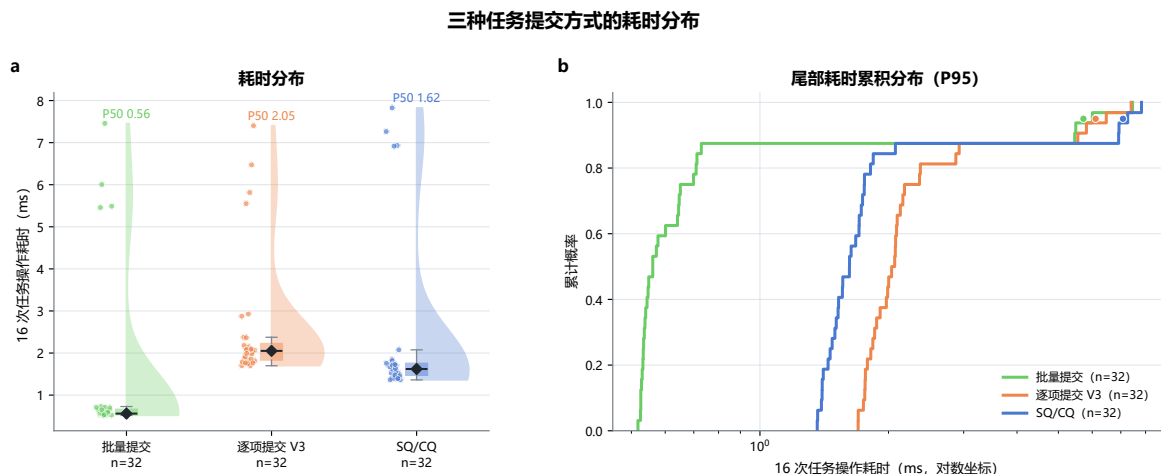


图 4.1: 三种任务提交方式的耗时分布

#### 4.1.4. 结果文件发布与冲突处理

结果文件发布测试直接在 Guest 中调用 `agent_file_publish()`，同时检查正常写入、同名竞争、错误参数和 Nexus 收敛逻辑。运行命令如下：

```
1 AGENT_TEST_CASE=agentpublish_ucore \
2 make agentos-test TOOLPREFIX=riscv64-linux-gnu-
```

正常路径读取到完整 header、payload 和 EOF。同一访问范围内的两个调用并发发布同名文件时，结果恰好为一个 OK 和一个 DUPLICATE，已经发布的内容没有被覆盖。坏指针、非法路径、错误结构大小、错误版本和非零保留字段均未留下正式目录项；失败与重复请求也没有增加 workflow 资源计数，删除测试生成的两份文件后，块和 inode 计数回到基线。

Nexus 随后按正式路径逐字节回读。已有内容与本次 header、payload 和 EOF 完全一致时，发布结果可以收敛为成功；内容不同则明确拒绝。这组结果表明，未命名 inode 与单次正式目录接入避免了“文件名已经可见、内容尚未写完”的中间状态，同名竞争也没有演变为静默覆盖。失败和重复发布同样沿用 workflow 资源账户的记账与回收路径。

通用 FS epoch 动态回归在 dirty、inflight 和 durable 三种状态下均通过，覆盖有序批提交、fsync、重启重放和既有 power-cut 窗口。作为共用的分配、回收和恢复基础，分配器故障与重启矩阵的 36 个用例也全部得到预期状态；去掉关键 FLUSH 的变异会被测试捕获。结果文件发布沿用这条 checkpoint 与恢复路径。

#### 4.1.5. 文件查询路径的 I/O 观测

查询测量同时记录文件读写、系统调用、扫描记录数、任务完成次数和 workflow 服务次数。图 4.2 上半部分把核心路径的 bytes\_read 和完整流程的全局 I/O 计数分别按核心时间窗内的系统调用次数归一化；下半部分按已完成操作数展示三种任务提交方式的传输成本。

图 4.2 的颜色只能在同一列、同一指标和同一统计窗口内比较。原始中位数显示，indexed 路径把 physical reads 从 143.5 降到 58、VirtIO notify 从 247.5 降到 165、requests 从 402.5 降到 332。这与选择性索引先缩小候选集的设计相符，说明查询优化已经减少了读路径上的实际工作量。与此同时，writes 从 213 增到 225.5、flush 从 47 增到 48，目录探测和检查记录也没有下降；系统当前只是把核心查询做得更省，并没有使完整流程中的写入、刷新和收尾同步变轻，这与图 3.5 中的端到端结果一致。

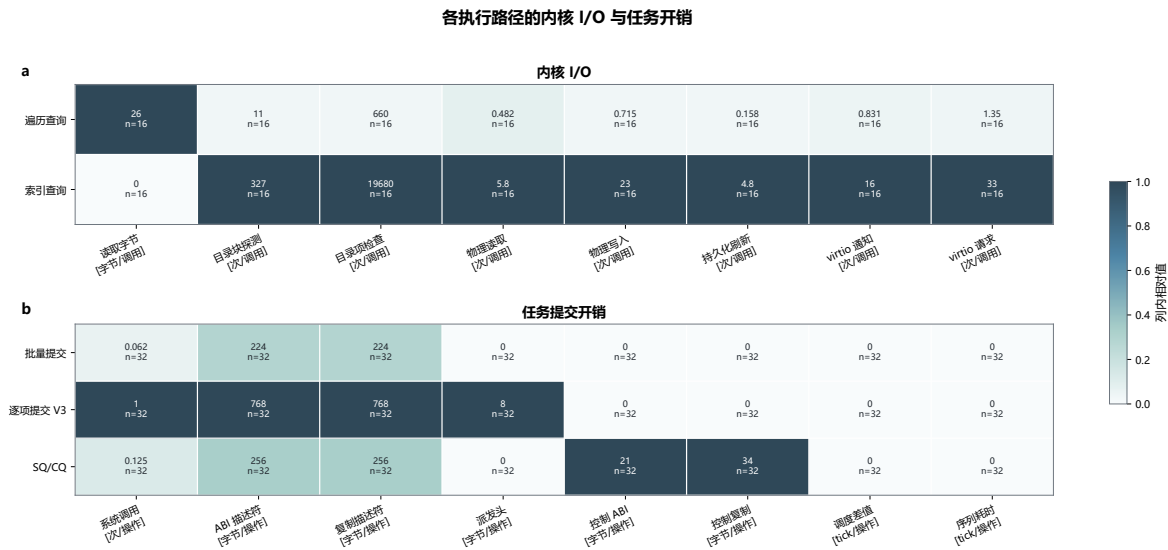


图 4.2: 各查询与任务路径的内核 I/O 观测

任务面板的 tick 中位数为 0 也不代表没有耗时：Batch、单项 V3、SQ/CQ 分别有 7/32、9/32、6/32 轮跨过 1 个 tick。10 ms 的 tick 适合观察是否跨越调度节拍，却不足以分辨这些微秒级路径的细小差异，因此延迟结论仍以墙钟测量为准。

4.1.6. 测量数据与可重复性

性能测量使用冻结在源码提交 2b14fb1f74b9bd093e6de939a16554620835699e 的 30 次相互独立全新 QEMU 启动，保存 33 份原始串口输出、19 个 CSV 数据表和 7,498 条结构化记录。每条记录都带有来源文件、来源 SHA-256 和原文序号。同一次启动中的内部配对用于观察相同工作量下的波动；跨启动汇总时，以源启动记录作为独立单位。

| 对应机制               | 测量设计                   | 独立单位        | 样本量             |
|--------------------|------------------------|-------------|-----------------|
| 综合恢复与文件查询          | 16 个 AB/BA 均衡配对启动      | 启动记录        | 16 组核心路径和完整流程配对 |
| 元数据目录与索引           | 3 × 4 参数网格，每格 15 个内部配对 | 源启动记录       | 180 组查询配对       |
| 工具协议与任务通道          | 4 次启动，8 组轮换顺序          | 启动记录/区段     | 96 条 16 次操作序列   |
| Agent Loop 与 EEVDF | 6 次启动，并发度为 1–4         | 启动记录/时间窗和探针 | 504 条精确唤醒探针     |

表 4.3: 各内核机制的测量数据构成

数据清单记录源码提交、工具版本、工作量计划、Guest 与构建哈希，并汇总原始输出、CSV 数据表和图像文件哈希；清单中的 75 项均通过哈希校验。保存这些材料的目的是，是让图表中的每个点都能回到对应的 Guest 输出。当前功能改动的正确性由日常回归另行验证；性能数据不替代功能验收，也不表示这些改动之后已重新测量。



## 4.2. 测试路径与结果

测试不只比对用户态输出文本，还通过真实 RV64 Guest 调用观察内核对象的状态、返回码、上下文序号、代次和运行统计。功能场景、机制探针和性能测量共用同一构建产物，因此表4.1中的项目都能追溯到实际执行路径。

### 4.2.1. Guest 原生工作负载

labdemo\_ucore 在 Guest 中创建并使用 AgentOS 对象，收集查询、文件写入、消息等待和调度路径上的计数与延迟。遍历路径和索引路径共用业务工作量、结果指纹和报告结构，因此比较不会把 Host 预处理、日志格式或测试框架差异误认为内核性能差异。

#### 代码 26: Guest 原生 AgentOS 工作负载的执行入口

c

```

745 static void run_native_workload(struct labdemo_workload_metrics *metrics)
746 {
747     static struct agent_file_query query;
748     static struct agent_file_query_result result;
749     static struct agent_file_meta target;
750     struct labdemo_lane_measurement_state *state = &native_measurement;
751     static char name[6];
752
753     memset(metrics, 0, sizeof(*metrics));
754     demo_quiescence_fence("native", "E2E_START", 1, &state->e2e_start);
755     metrics->end_to_end_started_us = demo_now_us();
756     seed_native_workload(&target);
757     memset(&query, 0, sizeof(query));
758     query.flags = AGENT_FILE_QUERY_USE_INDEX;
759     query.max_hits = AGENT_FILE_QUERY_MAX_HITS;
760     strcpy(query.project, DEMO_PROJECT);
761     strcpy(query.run_id, DEMO_BENCH_RUN);
762     strcpy(query.status, "failed");
763     demo_quiescence_fence("native", "CORE_START", 2, &state->core_start);
764     metrics->started_us = demo_now_us();
765     DEMO_DIAG_BEGIN();
766     check(agent_file_query(&query, &result) == 1,
767           "native failed-stage query");
768     DEMO_DIAG_END("native", "failed_index_query");
769     metrics->records_examined += result.scanned_records;
770     check(result.returned == 1 && result.used_index == 1 &&
771           strcmp(result.hits[0].stage, "align") == 0 &&
772           strcmp(result.hits[0].status, "failed") == 0,
773           "native failed-stage result");
774     metrics->discovered_us = demo_now_us();
775     demo_corpus_name(name, 'n', DEMO_BENCH_TARGET);
776     DEMO_DIAG_BEGIN();
777     {
778         int fd = open(name, O_WRONLY | O_TRUNC);
779
780         check(fd >= 0, "native recovery open");
781         check(write(fd, DEMO_BENCH_RECOVERED_BODY,
782                   strlen(DEMO_BENCH_RECOVERED_BODY)) ==
783               (ssize_t)strlen(DEMO_BENCH_RECOVERED_BODY),
784               "native recovery write");

```



```

785     target.update_mask = AGENT_FILE_META_UPDATE_STATUS |
786         AGENT_FILE_META_UPDATE_SUMMARY;
787     strcpy(target.status, "recovered");
788     strcpy(target.summary, "memory_limit recovered");
789     check(agent_file_meta_set(&target) == 0,
790         "native recovery metadata stage");
791     check(demo_quiescence_fsync(fd) == 0,
792         "native recovery primary ack");
793     check(close(fd) == 0, "native recovery close");
794 }
795 DEMO_DIAG_END("native", "grouped_recovery_primary_ack");
796 metrics->committed_us = demo_now_us();
797 strcpy(query.status, "recovered");
798 memset(&result, 0, sizeof(result));
799 DEMO_DIAG_BEGIN();
800 check(agent_file_query(&query, &result) == 1,
801     "native recovered-stage query");
802 DEMO_DIAG_END("native", "recovered_index_query");
803 metrics->records_examined += result.scanned_records;
804 check(result.returned == 1 && result.used_index == 1 &&
805     strcmp(result.hits[0].stage, "align") == 0 &&
806     strcmp(result.hits[0].status, "recovered") == 0,
807     "native recovered-stage result");
808 metrics->result_items = 1;
809 metrics->outcome_hash = demo_outcome_hash(result.hits[0].stage,
810     result.hits[0].status);
811 metrics->finished_us = demo_now_us();
812 take_performance_snapshot(&state->core_ack);
813 metrics->workload_syscalls = performance_delta(
814     state->core_start.performance.snapshot.observer_workload_syscalls,
815     state->core_ack.snapshot.observer_workload_syscalls);
816 check(metrics->workload_syscalls != 0, "native workload syscall receipt");
817 demo_quiescence_fence("native", "ACK_SETTLED", 3,
818     &state->ack_settled);
819 DEMO_DIAG_BEGIN();
820 for (int i = 0; i < LABDEMO_CORPUS_SIZE; i++) {
821     demo_corpus_name(name, 'n', i);
822     check(unlink(name) == 0, "native corpus cleanup");
823 }
824 DEMO_DIAG_END("native", "cleanup_unlink");
825 demo_quiescence_fence("native", "E2E_END", 4, &state->e2e_end);
826 metrics->end_to_end_finished_us = demo_now_us();
827 print_workload_lane("native", metrics);
828 print_mechanism_delta("native", "core", &state->core_start.performance,
829     &state->core_ack);
830 print_mechanism_delta("native", "end_to_end",
831     &state->e2e_start.performance,
832     &state->e2e_end.performance);
833 }
834
835 static void make_op(struct agent_op *op, int tool, uint64 id, uint64 arg0,
836     const char *payload)
837 {
838     memset(op, 0, sizeof(*op));
839     op->version = AGENT_CALL_VERSION;
840     op->tool_id = tool;
841     op->request_id = id;

```

```

842     op->arg0 = arg0;
843     if (payload)
844         strcpy(op->payload, payload);
845 }

```

查询图表由当前测量目录中的原始数据生成。核心路径与完整流程窗口的对比见第3.6章图3.5；该图同时保留机制路径上的时间和资源观测值。所有数值对应当前 QEMU/u-Core 配置、给定工作负载和该轮运行，不将其外推为其他硬件平台的固定吞吐量。

#### 4.2.2. 工具、执行约定与任务通道测试

Nexus Runtime 发起工具调用时，会把任务文本编码为固定结构，并通过 `send_message` 进入带类型信息的工具路径。该路径先完成任务结构校验、参数编码和回包状态检查，因此测试能区分用户态编码失败、内核 `schema` 拒绝和工具副作用失败。

##### 代码 27: Nexus 任务编码与 `send_message` 调用

c

```

992         argument_count, response);
993 }
994
995 int agent_nexus_task_send(int target_pid, unsigned long long task_id,
996     const struct agent_nexus_task *task,
997     struct agent_response_v2 *response)
998 {
999     struct agent_nexus_tool_argument arguments[2];
1000     char message[AGENT_EVENT_PAYLOAD_SIZE];
1001
1002     if (target_pid <= 0 || task_id == 0 || response == 0 ||
1003         agent_nexus_task_encode(task, message) < 0)
1004         return -1;
1005     memset(arguments, 0, sizeof(arguments));
1006     strcpy(arguments[0].key, "target_pid");
1007     arguments[0].type = AGENT_PARAM_UINT64;
1008     arguments[0].number = (unsigned long long)target_pid;
1009     strcpy(arguments[1].key, "message");
1010     arguments[1].type = AGENT_PARAM_STRING;
1011     strcpy(arguments[1].text, message);
1012     if (agent_nexus_tool_call("send_message", task_id, arguments, 2,
1013         response) < 0)

```

执行约定测试覆盖过期生命周期、过期执行约定、未知节点、工具不匹配、截止时间和依赖终止状态（`terminal state`）等分支。副作用前检查产生的完成记录关联执行记录票号和输出来源；对于直接调用或 V3 调用的拒绝，如果无法建立关键记录，调用会返回 `NO_SPACE/RETRY`，测试同样覆盖这条记录失败语义。因此，拒绝或取消不是孤立的错误码，而是能够与上下文、执行记录或明确的记录失败状态关联的结果。

##### 代码 28: 执行约定完成时的执行记录与 `terminal state` 不变量

c

```

1834 void
1835 agent_execution_contract_complete(struct agent_execution_claim *claim,
1836     const struct agent_result *result,
1837     const struct agent_execution_outcome *outcome)

```

```

1838 {
1839     struct agent_execution_contract_record *record;
1840     struct agent_execution_node_internal *node;
1841     struct agent_execution_node_runtime *runtime;
1842     struct agent_execution_completion *cached;
1843     struct agent_execution_producer_metadata *producer;
1844     int cache_index;
1845     int terminal_failure;
1846     int enabled;
1847
1848     if (claim == 0 || result == 0 || !claim->active || claim->legacy ||
1849         claim->slot < 0 ||
1850         claim->slot >= WORKFLOW_LIFECYCLE_CAP ||
1851         claim->node_id >= AGENT_EXECUTION_CONTRACT_MAX_NODES)
1852         return;
1853     enabled = intr_save();
1854     record = &agent_execution_contracts[claim->slot];
1855     if (!agent_execution_record_matches(record, claim->lifecycle) ||
1856         record->generation != claim->contract_generation ||
1857         claim->node_id >= record->node_count) {
1858         intr_restore(enabled);
1859         return;
1860     }
1861     runtime = &record->runtime[claim->node_id];
1862     node = &record->nodes[claim->node_id];
1863     if (runtime->state != AGENT_EXECUTION_NODE_RUNNING ||
1864         runtime->attempt_id != claim->attempt_id ||
1865         !agent_execution_digest_equal(runtime->request_digest,
1866             claim->request_digest)) {
1867         intr_restore(enabled);
1868         return;
1869     }
1870     if ((runtime->flags &
1871         AGENT_EXECUTION_RUNTIME_F_CANCEL_REQUESTED) != 0 &&
1872         result->status != AGENT_STATUS_CANCELLED)
1873         panic("execution contract cancel winner");
1874     if ((runtime->flags &
1875         AGENT_EXECUTION_RUNTIME_F_TIMEOUT_REQUESTED) != 0 &&
1876         result->status != AGENT_STATUS_TIMEOUT)
1877         panic("execution contract timeout winner");
1878     if (result->sequence == 0 || outcome == 0 ||
1879         outcome->evidence_ticket == 0 ||
1880         outcome->output_provenance_labels == 0)
1881         panic("execution contract terminal without evidence");
1882     runtime->sequence = result->sequence;
1883     runtime->status = result->status;
1884     runtime->decision_reason = claim->decision_reason;
1885     record->running_mask &= ~(1ULL << claim->node_id);
1886     if (record->running_count == 0)
1887         panic("execution contract running underflow");
1888     record->running_count--;
1889     producer = &agent_execution_producers[claim->slot][claim->node_id];
1890     if (claim->producer_control_id == 0 || claim->producer_pid <= 0)
1891         panic("execution contract producer identity");
1892     producer->control_id = claim->producer_control_id;
1893     producer->pid = claim->producer_pid;
1894     producer->context_sequence = result->sequence;

```

```

1895     producer->provenance_labels = outcome->output_provenance_labels;
1896     producer->valid = 1;
1897     if (result->status == AGENT_STATUS_OK) {
1898         runtime->state = AGENT_EXECUTION_NODE_SUCCEEDED;
1899         record->completed_mask |= 1ULL << claim->node_id;
1900         record->failed_mask &= ~(1ULL << claim->node_id);
1901     } else {
1902         runtime->state = result->status == AGENT_STATUS_CANCELLED ?
1903             AGENT_EXECUTION_NODE_CANCELLED :
1904             AGENT_EXECUTION_NODE_FAILED;
1905         record->completed_mask &= ~(1ULL << claim->node_id);
1906         terminal_failure = claim->retry_forbidden ||
1907             claim->attempt_id >= node->max_attempts ||
1908             !agent_execution_retry_allowed(node, result->status);
1909         if (terminal_failure) {
1910             record->failed_mask |= 1ULL << claim->node_id;

```

任务通道在提交后维护已接纳和运行状态、生命周期引用以及回调引用；硬截止时间触发过期，完成或拒绝进入统一的完成队列。该状态机的测试同时验证“不丢失生命周期引用”和“不在过期后伪装完成”两项性质。

#### 代码 29：任务提交后的回调、截止时间与完成分支

c

```

1900     agent_task_channel_publish_locked(state);
1901     intr_restore(enabled);
1902     return 0;
1903 }
1904 lifecycle = agent_task_lifecycle_key(p);
1905 if (workflow_lifecycle_operation_enter(lifecycle) < 0) {
1906     intr_restore(enabled);
1907     return AGENT_TASK_CHANNEL_RETRY;
1908 }
1909 agent_task_callback_get_locked(state);
1910 intr_restore(enabled);
1911 if ((sqe.flags & AGENT_TASK_SQE_F_HARD_DEADLINE) != 0 &&
1912     (ops == 0 || ops->expire == 0)) {
1913     agent_task_validation_abort(p, ops);
1914     return AGENT_TASK_CHANNEL_EVIDENCE;
1915 }
1916 hook_status = ops->validate(
1917     p, &sqe, &input_view, &validation, &completion);
1918 if (hook_status != AGENT_TASK_HOOK_PENDING) {
1919     agent_task_validation_abort(p, ops);
1920     return AGENT_TASK_CHANNEL_RETRY;
1921 }
1922 if (!agent_task_validation_valid(&validation)) {
1923     agent_task_validation_abort(p, ops);
1924     return AGENT_TASK_CHANNEL_EVIDENCE;
1925 }
1926 enabled = intr_save();
1927 state = agent_task_channel_find_locked(p);
1928 if (!agent_task_channel_issuer_valid_locked(state, p, issuer) ||
1929     state->ops != ops ||
1930     state->private->header.generation != generation ||
1931     state->private->header.sq_head != position) {

```

```

1932     if (!agent_task_channel_owner_valid_locked(state, p) ||
1933         state->ops != ops ||
1934         state->private->header.callback_refs == 0)
1935         panic("Task Channel validate cleanup");
1936     agent_task_callback_put_locked(state);
1937     workflow_lifecycle_operation_leave(state->lifecycle);
1938     intr_restore(enabled);
1939     agent_task_reclaim_deferred(p, ops);
1940     return AGENT_TASK_CHANNEL_STALE;
1941 }
1942 private = state->private;
1943 request = agent_task_request_alloc_locked(private);
1944 if (request == 0) {
1945     private->header.backpressure++;
1946     if (private->header.callback_refs == 0)
1947         panic("Task Channel validate pin");
1948     agent_task_callback_put_locked(state);
1949     workflow_lifecycle_operation_leave(lifecycle);
1950     intr_restore(enabled);
1951     return 0;
1952 }
1953 memset(request, 0, sizeof(*request));
1954 request->sqe = sqe;
1955 request->accepted_tick = agent_task_now();
1956 request->expected_output_type = validation.output_artifact_type;
1957 request->provenance_labels = validation.output_provenance_labels;
1958 request->state = AGENT_TASK_REQUEST_ACCEPTED;
1959 request->flags = AGENT_TASK_REQUEST_F_LIFECYCLE_HELD;
1960 if (private->header.callback_refs == 0)
1961     panic("Task Channel submit transfer");
1962 agent_task_callback_put_locked(state);
1963 if (agent_task_request_input_acquire_locked(
1964     private, state->resource_private, request) < 0) {
1965     request->flags &= ~AGENT_TASK_REQUEST_F_LIFECYCLE_HELD;
1966     workflow_lifecycle_operation_leave(lifecycle);
1967     memset(request, 0, sizeof(*request));
1968     status = agent_task_channel_protocol_fault_locked(state);
1969     intr_restore(enabled);
1970     return status;
1971 }
1972 request->state = AGENT_TASK_REQUEST_RUNNING;
1973 private->header.sq_head++;
1974 private->header.submitted++;
1975 private->header.last_accepted_request_id = sqe.request_id;
1976 *consumed = 1;
1977 private->header.live_requests++;
1978 agent_task_callback_get_locked(state);
1979 agent_task_channel_publish_locked(state);
1980 intr_restore(enabled);
1981 hook_status = ops->submit(
1982     p, &sqe, &input_view, &validation, &completion);
1983 enabled = intr_save();
1984 state = agent_task_channel_find_locked(p);
1985 if (!agent_task_channel_owner_valid_locked(state, p) ||
1986     state->ops != ops || state->private->header.callback_refs == 0)
1987     panic("Task Channel submit callback");
1988 agent_task_callback_put_locked(state);

```

```

1989     intr_restore(enabled);
1990     agent_task_reclaim_deferred(p, ops);
1991     if (hook_status == AGENT_TASK_HOOK_PENDING &&
1992         (sqe.flags & AGENT_TASK_SQE_F_HARD_DEADLINE) != 0 &&
1993         agent_task_now() >= sqe.deadline_tick) {
1994         status = agent_task_channel_expire(p, agent_task_now(), ops);
1995         return agent_task_consumed_after_expire(status);
1996     }
1997     if (hook_status == AGENT_TASK_HOOK_COMPLETE ||
1998         hook_status == AGENT_TASK_HOOK_DENIED) {
1999         status = agent_task_channel_complete(
2000             p, sqe.ring_generation, sqe.request_id,
2001             sqe.slot_generation, &completion, ops);
2002         if (status == AGENT_TASK_CHANNEL_OK)
2003             return 1;
2004         return status;
2005     }
2006     if (hook_status != AGENT_TASK_HOOK_PENDING)
2007         return AGENT_TASK_CHANNEL_EVIDENCE;
2008     return 1;

```

### 4.2.3. Nexus 会话与 Host Relay

Nexus 使用带序号的帧，把 Guest 的会话、工具事件、审批和运行指标接入 Host Relay。用户态解码器会校验会话、序号、类别和长度，再由中继程序完成 Provider 适配和本地端点转发。网络和模型协议留在 Host 侧，智能体身份、工具权限和任务状态仍由 Guest 和内核验证。

#### 代码 30: Nexus V2 帧的编码入口

c

```

903 static int live_frame_encode(const char *session, uint64 sequence,
904                             const char *kind, const char *payload,
905                             char *output, uint capacity)
906 {
907     struct live_builder builder;
908     unsigned char digest[LIVE_SHA_SIZE];
909     char digest_hex[LIVE_SHA_HEX_SIZE + 1];
910     uint payload_length = strlen(payload);
911     uint encoded_length;
912
913     if (!live_session_valid(session) || sequence == 0 ||
914         !live_kind_valid_v2(kind) ||
915         payload_length == 0 ||
916         payload_length > LIVE_MAX_JSON ||
917         !live_utf8_valid((const unsigned char *)payload, payload_length))
918         return -1;
919     live_sha256(payload, payload_length, digest);
920     live_digest_hex(digest, digest_hex);
921     encoded_length = live_base64_encode(
922         (const unsigned char *)payload, payload_length,
923         live_base64_buffer, sizeof(live_base64_buffer));
924     if (encoded_length == 0)
925         return -1;
926     live_builder_init(&builder, output, capacity);

```

```

927     live_builder_text(&builder, LIVE_PREFIX_V2);
928     live_builder_text(&builder, session);
929     live_builder_char(&builder, ' ');
930     live_builder_u64(&builder, sequence);
931     live_builder_char(&builder, ' ');
932     live_builder_text(&builder, kind);
933     live_builder_char(&builder, ' ');
934     live_builder_u64(&builder, payload_length);
935     live_builder_char(&builder, ' ');
936     live_builder_text(&builder, digest_hex);
937     live_builder_char(&builder, ' ');
938     live_builder_text(&builder, live_base64_buffer);
939     live_builder_char(&builder, '\n');
940     return builder.ok && builder.length <= LIVE_MAX_FRAME ?
941         (int)builder.length : -1;
942 }

```

### 代码 31: Host Relay 的串口写入与本地消息分发

Python

```

2154     def _reader(self, stream, source: str) -> None:
2155         def run() -> None:
2156             while True:
2157                 try:
2158                     chunk = stream.read(4096)
2159                     except OSError:
2160                         chunk = b""
2161                     if not chunk:
2162                         self.events.put(("eof", source.encode("ascii")))
2163                     return
2164                     self.events.put((source, chunk))
2165
2166                 threading.Thread(target=run, name=f"agentos-{source}", daemon=True).start()
2167
2168     def _write_serial(self, line: bytes) -> None:
2169         relay._write_process_before_deadline(
2170             self.process,
2171             line,
2172             deadline_monotonic=time.monotonic() + SERIAL_WRITE_TIMEOUT_SECONDS,
2173         )
2174
2175     def _drain_local(self) -> None:
2176         # Bound each pass so local traffic cannot defer provider polling or
2177         # Guest serial processing. The role-aware queue also prevents a ping
2178         # flood from placing a controller command behind every observer item.
2179         for _ in range(MAX_LOCAL_DRAIN):
2180             try:
2181                 peer, message = self.endpoints.inbound.get_nowait()
2182             except queue.Empty:
2183                 return
2184             try:
2185                 # A peer may disconnect after its reader queued commands.
2186                 # Identity and liveness are checked at execution time so a
2187                 # replacement controller never inherits the old queue.
2188                 if not self.endpoints.is_current(peer):
2189                     continue
2190             try:

```



```

2191         self._handle_local(peer, message)
2192     except relay.RelayError as error:
2193         peer.send(
2194             {
2195                 "type": "error",
2196                 "code": error.code,
2197                 "message": error.public_message,
2198             }
2199         )
2200     finally:
2201         consumed = getattr(peer, "inbound_consumed", None)
2202         if consumed is not None:
2203             consumed()
2204
2205     def _handle_local(self, peer: _Peer, message: Mapping[str, object]) -> None:
2206         message_type = message.get("type")
2207         if peer.role == "observer":
2208             if message_type == "ping" and set(message) == {"type"}:
2209                 peer.send({"type": "pong"})
2210             return
2211             raise relay.WireProtocolError("ROLE_DENIED", "observer is read-only")
2212         if message_type == "user_message":
2213             if set(message) != {"type", "content"}:
2214                 raise relay.WireProtocolError("BAD_LOCAL_MESSAGE", "user message is malformed")
2215             self.session.submit_user(message.get("content"))
2216         elif message_type == "command":
2217             if set(message) != {"type", "command"}:
2218                 raise relay.WireProtocolError("BAD_LOCAL_MESSAGE", "control message is
malformed")
2219             command = message.get("command")
2220             self.session.request_control(command)

```

测试运行还检查关闭、取消和过期后的清理过程。Host Relay 可以请求停止，Guest 把等待和工具事件转换为可观察的完成记录；内核则利用生命周期代次，阻止回收后的延迟消息、任务完成通知或订阅引用落到新的 workflow。

#### 4.2.4. 失效状态与恢复分支

测试不仅执行正常流程，也主动构造过期代次、错误 schema、资源不足、SQ/CQ 协议失步、实时查询缺口和失效 artifact 句柄。预期结果不是笼统的“调用失败”，而是由对应接口返回 STALE、DENIED、NO\_SPACE、RETRY 等状态，并保持尚未提交的副作用不可见。任务通道在协议失步后要求显式 RESYNC；实时查询在出现缺口后要求以替代订阅和完整快照恢复基线；Nexus 收到失效研究结果时重新安排任务，审批被拒绝时保留可读报告并继续关闭会话。这些分支用于检查错误能否被上层识别和处理，而不是只验证正常路径能够跑通。

## 5. 总结与展望

### 5.1. 阶段性结论

本项目在 RISC-V uCore 中实现了内核中的智能体运行机制。AgentOS-uCore 以现有的进程、页表、VFS、系统调用和调度器为基础，把身份、工具、上下文、文件查询、事件等待、资源和调度纳入同一工作流。六组机制让智能体在发生副作用时都具有明确的身份、能力位、访问范围、代次、数据来源和资源账户。

项目取得了三方面进展。第一，Agent 已从应用层标签变为内核能够识别的进程身份，普通进程和 Agent 进程可以在同一个 uCore 中共存，高频 Context 数据也能从固定用户地址直接读取。第二，工具调用协议、V2/V3、上下文路径、实时查询、消息等待、资源账户和 EEVDF 贯通了 Agent 从调用、等待到取得资源服务的主要过程。第三，控制台和 Nexus 把这些内核机制用于科研恢复场景和四角色常驻协作，使确定性测试与模型驱动交互走同一条 Guest 路径。

实验展示了各条路径的性能特征。索引核心路径在 16 个独立配对中均取得更低延迟，配对加速比中位数为 3.118 倍；12 组文件目录网格的中位加速比介于 1.164–2.808 倍；504 条 EEVDF 唤醒探针均在 0–1 个 tick 内得到服务。端到端窗口中，索引路径相对遍历路径的中位差值为 +13.452 ms，说明核心窗口外的总开销抵消了索引收益；现有数据尚未把外围步骤逐段计时，后续应先定位主导项再优化。

下一阶段将沿三条主线推进：为实时查询补充分页游标，降低宽条件下反复收紧查询的成本；在长 Provider 延迟和密集事件下扩大压力测试，观察 Task Channel 与事件队列的背压；将工作流 EEVDF 扩展到 SMP 和真实 RISC-V 硬件，分析多核实体迁移、唤醒和资源记账。与此同时，完整流程会按准备、查询、写入、刷新和清理分别计时，先定位索引收益被外围开销抵消的具体环节，再决定优化顺序。

### 5.2. 当前技术限制

当前测试环境是 RV64 QEMU 单 Hart，工作流 EEVDF 的实体迁移和多核竞争尚未测量。实时查询单次最多返回 8 个结果，ABI 还没有分页游标；事件队列、Context 路径、任务通道和调度实体表也采用固定容量。现有负载以短时、可重复的 Guest 场景为主，尚未覆盖长 Provider 延迟下的大规模异步 backlog。调度观测使用 10 ms 的 tick，适合判断唤醒是否跨越节拍，不足以分辨微秒级差异。

### 5.3. 后续工作

下一阶段将补充分页查询和长时间压力场景，重点观察长 Provider 延迟、任务积压、订阅事件密集到达时的背压行为；随后把工作流 EEVDF 扩展到 SMP 和真实 RISC-V 硬件，分析实体迁移、唤醒和资源记账。性能测量还会进一步拆分场景准备、查询、写入、刷新和清理阶段，并使用更细粒度的时钟定位完整流程中的额外开销。

## 6. 参考说明

### 6.1. uCore 与系统基础

AgentOS-uCore 建立在 LearningOS/uCore 教学内核之上，继承 RISC-V 启动、进程、虚拟内存、VFS、异常处理、系统调用和基础调度框架。项目在这些现有对象上增加智能体状态和用户内核接口（UAPI），同时保持原有组件的职责边界。相关基础资料见参考文献中的 uCore 课程材料。

### 6.2. 协议、工具与 Runtime

结构化工具和任务通道使用定长结构、版本和大小字段、共享头文件以及 Guest 与内核布局检查。Nexus 的 Host Relay 管理本地套接字、TLS、Provider 适配器和帧转发；Guest 保存业务状态、artifact 以及与内核的交互。系统中的外部依赖及许可信息以仓库中的许可证和源码声明为准。

### 6.3. 测量资料

性能图来自 docs/figures/performance/ 的当前导出文件。测量目录保留原始串口输出、CSV 和逐样本来源信息，用于复核配对延迟、扫描组、任务操作序列和 EEVDF 唤醒探针。图表叙述依据这些保存的测量结果，不以模型推断替代原始记录。

### 6.4. 文献使用

系统设计参考操作系统、能力安全、结构化接口、事件驱动 Runtime 和公平调度等公开资料。

## 7. 参考文献

- [1] LearningOS Community. ucore-tutorial-code-2025s. <https://github.com/LearningOS/uCore-Tutorial-Code-2025S>, 2025. AgentOS-uCore 的教学内核基础.
- [2] Linux Kernel Documentation. Cpu accounting controller. <https://docs.kernel.org/6.11/admin-guide/cgroup-v1/cpuacct.html>, 2024.
- [3] Linux Kernel Project. rstat.c. <https://github.com/torvalds/linux/blob/master/kernel/cgroup/rstat.c>, 2026.
- [4] Linux Kernel Documentation. Bpf ring buffer. <https://docs.kernel.org/6.6/bpf/ringbuf.html>, 2023.
- [5] Linux Kernel Documentation. Eevdf scheduler. <https://docs.kernel.org/scheduler/sched-eevdf.html>, 2026.
- [6] Jens Axboe. Efficient io with io\_uring. [https://kernel.dk/io\\_uring.pdf](https://kernel.dk/io_uring.pdf), 2019.
- [7] Haiku Project. Practical file system design with the be file system. <https://www.haiku-os.org/legacy-docs/practical-file-system-design.pdf>, 1999.
- [8] Anthropic. How tool use works. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>, 2026.
- [9] Anthropic. How claude code works: The agentic loop. <https://code.claude.com/docs/en/how-claude-code-works>, 2026.